LẬP TRÌNH PYTHON CHO MACHINE LEARNING

Từ Python cơ bản đến Machine Learning

Ngô Hoàng Dũng - Lê Hoàng Ân - Nguyễn Thị Hà - Lê Ngọc Ánh Dương

Ngày 1 tháng 11 năm 2026

Mục lục

1 Giới thiệu về Python **1**
1.1 Python là gì? . . . 1
1.2 Lịch sử phát triển của Python . . . 1
1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991) . . . 1
1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999) . . . 1
1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000) . . . 2
1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008) . . . 2
1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay) . . . 2
1.3 Guido van Rossum – tác giả của Python . . . 2
1.4 Tên gọi Python . . . 2
1.5 Hướng dẫn cài đặt Python và Anaconda . . . 2
1.5.1 Cài đặt Python . . . 2
1.5.2 Cài đặt Anaconda/Miniconda . . . 3
1.5.3 Tạo môi trường Conda . . . 4
1.6 Tóm tắt chương . . . 4

2 Cú pháp Python cơ bản **5**
2.1 Biến và Kiểu dữ liệu . . . 5
2.1.1 Các kiểu dữ liệu cơ bản . . . 5
2.2 Toán tử trong Python . . . 6
2.2.1 Toán tử cơ bản . . . 6
2.2.2 Toán tử gán mở rộng . . . 6
2.3 Hàm toán học trong Python . . . 6
2.3.1 Các hằng số toán học . . . 7
2.3.2 Các hàm toán học thường dùng . . . 7
2.4 Cấu trúc dữ liệu nâng cao . . . 7
2.4.1 List (Danh sách) . . . 7
2.5 Cấu trúc điều khiển và Vòng lặp . . . 7
2.5.1 Câu lệnh điều kiện if-else . . . 7
2.6 Tóm tắt chương . . . 8

3 Lập trình hướng đối tượng trong Python **9**
3.1 Lớp và Đối tượng (Class and Object) . . . 9
3.2 Các phương thức đặc biệt (Magic Methods) . . . 9
3.2.1 Phương thức `__init__` và `__call__` . . . 9
3.3 Tính Kế thừa (Inheritance) . . . 10
3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters) . . . 11

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn . . . 11
3.6 Mẫu thiết kế Pipeline . . . 12
3.7 Tóm tắt chương . . . 12

4 Xử Lý Dữ Liệu Với NumPy Và Pandas (NumPy & Pandas) 13
4.1 Thư viện NumPy - Tính toán Khoa học . . . 13
4.1.1 Khởi tạo mảng và kiểm tra thuộc tính . . . 13
4.1.2 Thao tác trên mảng: Thêm phần tử, xóa, sửa . . . 14
4.1.3 Slicing và Boolean Indexing (Lọc dữ liệu) . . . 14
4.1.4 Đại số tuyến tính (Linear Algebra) . . . 15
4.2 Thư viện Pandas - Xử lý và Phân tích Dữ liệu . . . 15
4.2.1 Tạo và Đọc dữ liệu . . . 15
4.2.2 Truy xuất & Điều kiện lọc dữ liệu . . . 16
4.2.3 Xử lý dữ liệu khuyết & Thống kê mô tả . . . 16
4.2.4 Thay thế và Áp dụng hàm . . . 17
4.2.5 Gom nhóm (GroupBy) & Gộp dữ liệu (Concat/Merge) . . . 17

5 Tiền xử lý dữ liệu (Data Preprocessing) 18
5.1 Khởi nguồn và vai trò của tiền xử lý dữ liệu . . . 19
5.1.1 "Rác vào, rác ra" . . . 19
5.1.2 Tiền xử lý trong quy trình khai phá tri thức (KDD) . . . 19
5.1.3 Phân loại các tác vụ tiền xử lý . . . 20
5.1.4 Thư viện scikit-learn và giao diện `fit` / `transform` . . . 20
5.2 Các kiểu dữ liệu và lựa chọn phép biến đổi . . . 21
5.3 Xử lý dữ liệu thiếu (Missing Values) . . . 21
5.3.1 Cơ chế sinh dữ liệu thiếu . . . 22
5.3.2 Simple Imputer: trung bình, trung vị, giá trị phổ biến . . . 22
5.3.3 KNN Imputer và khoảng cách nan-Euclid . . . 23
5.3.4 Iterative Imputer . . . 25
5.4 Phát hiện ngoại lai (Outlier Detection) . . . 28
5.4.1 Phương pháp một biến: z-score và quy tắc IQR . . . 28
5.4.2 Phương pháp nhiều biến: Local Outlier Factor (LOF) . . . 29
5.5 Mã hóa dữ liệu phân loại . . . 31
5.5.1 Label Encoder . . . 31
5.5.2 One Hot Encoder . . . 33
5.6 Vector hóa văn bản . . . 35
5.6.1 Mô hình túi từ và Count Vectorizer . . . 35
5.6.2 TF-IDF . . . 37
5.7 Chuẩn hóa thang đo đặc trưng (Feature Scaling) . . . 40
5.7.1 Vì sao cần chuẩn hóa thang đo? . . . 40
5.7.2 Min Max Scaler . . . 40
5.7.3 Max Abs Scaler . . . 40
5.7.4 Standard Scaler (chuẩn hóa z-score) . . . 41
5.7.5 Robust Scaler . . . 41
5.7.6 Ví dụ so sánh bốn bộ chuẩn hóa . . . 41
5.8 Dữ liệu mất cân bằng và SMOTE . . . 42

5.9 Kết hợp các bước: Pipeline và ColumnTransformer . . . 44
5.10 Nghiên cứu tình huống: bộ dữ liệu Ames Housing . . . 45
5.10.1 Giới thiệu bộ dữ liệu . . . 45
5.10.2 Những "cái bẫy" khi đọc dữ liệu . . . 46
5.10.3 Đọc và khảo sát dữ liệu . . . 46
5.10.4 Xây dựng quy trình tiền xử lý hoàn chỉnh . . . 47
5.11 Tóm tắt chương . . . 48
5.12 Bài tập . . . 49
Tài liệu tham khảo của chương . . . 50

6 Hồi quy tuyến tính (Linear Regression) 52
6.1 Mô hình Hồi quy tuyến tính đơn biến . . . 52
6.2 Hàm mất mát (Loss Function) . . . 52
6.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn . . . 52
6.4 Tóm tắt chương . . . 53

7 Phân lớp (Classification) 54
7.1 Bài toán phân lớp . . . 54
7.2 K láng giềng gần nhất (KNN) . . . 55
7.2.1 KNN có trọng số . . . 56
7.2.2 KNN cho bài toán hồi quy . . . 57
7.3 Naive Bayes . . . 58
7.4 Hồi quy logistic (Logistic Regression) . . . 59
7.5 Cây quyết định (Decision Tree) . . . 60
7.5.1 Độ đo độ hỗn tạp . . . 61
7.6 Máy vector hỗ trợ (SVM) . . . 62
7.6.1 Lề cực đại . . . 62
7.6.2 Lề mềm và tham số C . . . 62
7.6.3 Thủ thuật nhân . . . 62
7.7 So sánh các thuật toán . . . 63
7.8 Tóm tắt chương . . . 64
7.9 Bài tập . . . 64
Tài liệu tham khảo của chương . . . 65

8 Học kết hợp (Ensemble Learning) 66
8.1 Trí tuệ đám đông . . . 66
8.1.1 Độ chệch và phương sai . . . 67
8.2 Bagging và Random Forest . . . 67
8.2.1 Bagging . . . 67
8.2.2 Random Forest . . . 68
8.3 AdaBoost . . . 69
8.4 Gradient Boosting và XGBoost . . . 70
8.4.1 Gradient Boosting . . . 70
8.4.2 XGBoost . . . 70
8.5 Voting và Stacking . . . 72
8.6 Tổng kết so sánh . . . 73

8.7 Tóm tắt chương . . . 73
8.8 Bài tập . . . 73
Tài liệu tham khảo của chương . . . 74

9 Phân cụm (Clustering) 75
9.1 Bài toán phân cụm . . . 75
9.2 K-Means . . . 76
9.2.1 Chọn số cụm K . . . 77
9.3 Phân cụm phân cấp (Hierarchical Clustering) . . . 78
9.4 DBSCAN . . . 80
9.5 Mean-shift . . . 81
9.6 So sánh và ứng dụng . . . 82
9.7 Tóm tắt chương . . . 83
9.8 Bài tập . . . 83
Tài liệu tham khảo của chương . . . 84

10 Giảm chiều dữ liệu (Dimensionality Reduction) 85
10.1 Lời nguyền số chiều . . . 85
10.2 Kiến thức nền: phương sai, hiệp phương sai và trị riêng . . . 86
10.2.1 Phương sai . . . 86
10.2.2 Hiệp phương sai và hệ số tương quan . . . 86
10.2.3 Trị riêng và vector riêng . . . 87
10.3 Phân tích giá trị suy biến (SVD) . . . 88
10.3.1 Lịch sử . . . 88
10.3.2 Tính SVD thông qua $\mathbf{X}^\top\mathbf{X}$ . . . 89
10.3.3 Ứng dụng: nén ảnh và TruncatedSVD . . . 90
10.4 Phân tích thành phần chính (PCA) . . . 91
10.4.1 Lịch sử và ý tưởng . . . 91
10.4.2 Chọn số thành phần . . . 94
10.4.3 PCA trong pipeline học máy . . . 95
10.5 Tóm tắt chương . . . 96
10.6 Bài tập . . . 96
Tài liệu tham khảo của chương . . . 97

11 Đánh giá và lựa chọn mô hình (Validation) 98
11.1 Lỗi tổng quát hóa . . . 98
11.2 Chiến lược chia dữ liệu . . . 99
11.2.1 Hold-out: train / validation / test . . . 99
11.2.2 Kiểm định chéo k-fold . . . 100
11.3 Độ đo cho bài toán phân lớp . . . 101
11.3.1 Ma trận nhầm lẫn . . . 101
11.3.2 Đường ROC và AUC . . . 102
11.4 Độ đo cho bài toán hồi quy . . . 103
11.5 Tìm siêu tham số . . . 104
11.5.1 Đường cong học và đường cong kiểm định . . . 105
11.6 Những sai lầm thường gặp . . . 106

11.7 Tóm tắt chương 106
11.8 Bài tập 106
Tài liệu tham khảo của chương 107

12 Mạng nơ-ron nhân tạo (Neural Network) 108
12.1 Lịch sử phát triển 109
12.2 Perceptron 109
12.3 Mạng nơ-ron nhiều lớp (MLP) 111
12.3.1 Kiến trúc 111
12.3.2 Hàm kích hoạt 112
12.3.3 Hàm mất mát và lớp đầu ra 112
12.4 Lan truyền ngược (Backpropagation) 112
12.5 Huấn luyện mạng nơ-ron trong thực tế 114
12.5.1 Bộ tối ưu 114
12.5.2 Khởi tạo và chuẩn hóa 114
12.5.3 Chống quá khớp 114
12.6 MLP với scikit-learn 115
12.7 Giới thiệu thư viện học sâu 116
12.8 Tóm tắt chương 117
12.9 Bài tập 117
Tài liệu tham khảo của chương 118

Chương 1: Giới thiệu về Python

1.1 Python là gì?

Python là một ngôn ngữ lập trình được sử dụng rộng rãi trong các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và máy học (ML). Các nhà phát triển sử dụng Python vì nó hiệu quả, dễ học và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và tăng tốc độ phát triển. (Nguồn: `https://aws.amazon.com/vi/what-is/python/`)

Python hỗ trợ nhiều mô hình lập trình, trong đó có lập trình thủ tục, lập trình hướng đối tượng và lập trình hàm.

Một chương trình Python đơn giản:

```
print("Hello, Python!")
```

Cú pháp của Python có xu hướng sử dụng khoảng trắng để thể hiện cấu trúc của chương trình. Ví dụ:

```
if age >= 18:
    print("Adult")
else:
    print("Under 18")
```

Cách tổ chức này giúp mã nguồn Python có cấu trúc rõ ràng và dễ đọc.

1.2 Lịch sử phát triển của Python

1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991)

Vào cuối năm 1989, khi đang làm việc tại Viện Nghiên cứu Quốc gia về Toán học và Khoa học Máy tính (CWI) ở Hà Lan, Guido van Rossum quyết định viết một ngôn ngữ lập trình mới trong kỳ nghỉ Giáng sinh để làm dự án phụ. Ông muốn tạo ra một ngôn ngữ kết hợp tính đơn giản của ngôn ngữ ABC với khả năng tương tác tốt với hệ điều hành Amoeba.

Tháng 2/1991, Guido đăng tải mã nguồn lên nhóm tin alt.sources với phiên bản Python 0.9.0. Dù chỉ là bản phôi thai, nó đã có những nền móng rất hiện đại: Mô hình OOP, Hệ thống xử lý ngoại lệ, Các kiểu dữ liệu cốt lõi (list, dict, str) và Hệ thống module.

1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999)

Tháng 1/1994, Python 1.0 chính thức ra mắt, tích hợp mạnh mẽ các công cụ của Lập trình chức năng (Functional Programming) như lambda, map, filter và reduce.

Năm 1995 (Python 1.2), Guido tiếp tục phát triển Python tại Tập đoàn Sáng kiến Nghiên cứu Quốc gia (CNRI) ở Mỹ, bổ sung khả năng truyền tham số theo tên (Keyword arguments). Trong giai đoạn này, Guido được cộng đồng trao danh hiệu BDFL (Benevolent Dictator for Life).

1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000)

Python 2.0 đánh dấu sự chuyển dịch lớn sang mô hình phát triển mã nguồn mở hoàn toàn. Các tính năng đột phá bao gồm: List Comprehension, Garbage Collector (GC), và ra đời quy trình PEP (Python Enhancement Proposal). Phiên bản Python 2.7 (2010) trở thành phiên bản ổn định nhất của dòng Python 2.

1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008)

Sau nhiều năm phát triển, Python 2 bị tích tụ nhiều lỗi thiết kế. Guido quyết định phát hành Python 3.0 với đặc điểm cốt lõi là Không tương thích ngược (Backwards Incompatible) với Python 2. Những cải tiến quan trọng bao gồm:

- Đồng bộ Unicode cho mọi chuỗi ký tự.
- Thay đổi câu lệnh `print` thành hàm `print()`.
- Tối ưu hóa bộ nhớ bằng cách chuyển các hàm như `range()`, `zip()`, `map()` sang dạng trả về Iterator.

1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay)

Cú pháp đơn giản giúp Python trở thành lựa chọn hàng đầu trong Khoa học dữ liệu và AI với các thư viện như NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch.

Ngày 01/01/2020, Python 2 chính thức bị khai tử (EOL), đưa Python 3 trở thành tiêu chuẩn duy nhất. Giai đoạn hiện tại (Python 3.8 – 3.13+) tập trung vào bổ sung các toán tử mới và dự án Faster CPython nhằm tối ưu hóa hiệu năng.

1.3 Guido van Rossum – tác giả của Python

Guido van Rossum là một nhà khoa học máy tính người Hà Lan. Ông là người giữ quyền quyết định tối cao về định hướng kiến trúc của ngôn ngữ (BDFL) cho đến khi từ chức vào năm 2018.

1.4 Tên gọi Python

Tên gọi ”Python” không xuất phát từ loài trăn, mà lấy cảm hứng từ chương trình hài kịch yêu thích của ông trên đài BBC: ”Monty Python’s Flying Circus”.

1.5 Hướng dẫn cài đặt Python và Anaconda

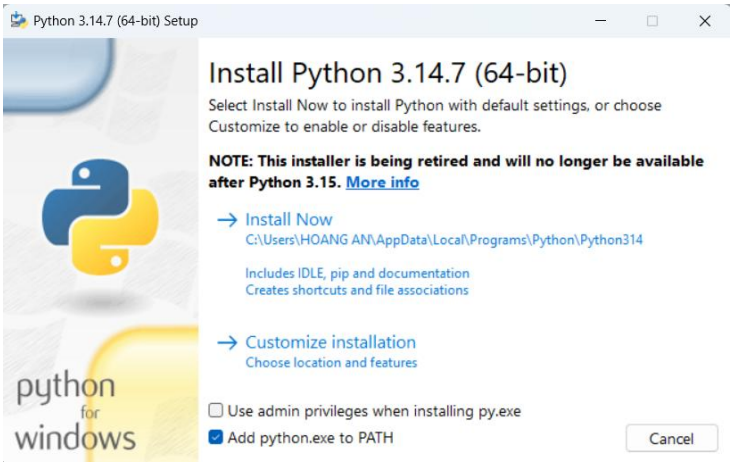
1.5.1 Cài đặt Python

1. Truy cập trang chủ Python: `https://www.python.org/downloads/`



Hình 1.1: Guido van Rossum - Tác giả sáng lập Python. (Nguồn: Sauria Associates, LLC / FlickrVision)

2. Tải phiên bản cài đặt (Ví dụ: Python 3.14.7 cho Windows).
3. Mở file cài đặt. **Lưu ý quan trọng:** Nhớ tick vào ô ”Add python.exe to PATH” trước khi bấm ”Install Now”.
4. Bấm ”Close” khi cài đặt thành công. Có thể chọn ”Disable path length limit” nếu cần.



Hình 1.2: Tick chọn Add python.exe to PATH trong quá trình cài đặt

1.5.2 Cài đặt Anaconda/Miniconda

Anaconda là một nền tảng phân phối Python và R phục vụ cho khoa học dữ liệu.

1. Truy cập trang tải Anaconda: `https://www.anaconda.com/download`
2. Tải bản cài đặt Anaconda Distribution hoặc Miniconda.
3. Mở file cài đặt, chọn ”Just Me”, chọn đường dẫn lưu (ví dụ: `C:\miniconda3`).

4. Trong phần Advanced Installation Options, có thể tick chọn ”Register Miniconda3 as my default Python”.

5. Nhấn ”Install” và đợi quá trình cài đặt hoàn tất.

1.5.3 Tạo môi trường Conda

Sau khi cài đặt Anaconda/Miniconda, mở Anaconda Prompt và sử dụng lệnh sau để tạo một môi trường ảo (ví dụ tên là `ai`) với phiên bản Python 3.14:

```
conda create -n ai python=3.14 anaconda
```

Xác nhận tiến trình cài đặt các gói bằng cách gõ y (yes) khi được yêu cầu.

1.6 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu:

- Khái niệm cơ bản về Python và ứng dụng của nó.
- Lịch sử hình thành chi tiết qua từng giai đoạn (từ Python 0.9.0 đến Python 3.x).
- Tiểu sử của tác giả Guido van Rossum và nguồn gốc tên gọi thú vị của Python.
- Các bước cài đặt môi trường lập trình cơ bản với Python và Anaconda.

Chương 2: Cú pháp Python cơ bản

Chương này trình bày các khái niệm nền tảng trong lập trình Python, bao gồm biến, kiểu dữ liệu, toán tử, cấu trúc điều khiển và các hàm toán học. Đây là các kiến thức bắt buộc trước khi đi vào xử lý dữ liệu và Machine Learning.

2.1 Biến và Kiểu dữ liệu

Trong Python, biến được khởi tạo tự động khi gán giá trị. Python là ngôn ngữ *dynamically typed* (định kiểu động).

Định nghĩa 2.1 (Biến trong Python)**.** *Biến là tên gọi đại diện cho một vùng nhớ chứa dữ liệu. Giá trị của biến có thể thay đổi trong quá trình thực thi chương trình.*

2.1.1 Các kiểu dữ liệu cơ bản

Dựa trên nền tảng Python Basic, dưới đây là bảng tổng hợp các kiểu dữ liệu cốt lõi thường xuyên được sử dụng:

Tên	Kiểu dữ liệu	Ví dụ
`str`	Kiểu chuỗi (string)	`a = 'Thien Thanh'`
`int`	Kiểu số nguyên	`a = 79`
`float`	Kiểu số thực	`a = 7.6`
`NoneType`	Kiểu None	`a = None`
`bool`	Kiểu luận lý (True, False)	`a = True`
`list`	Kiểu danh sách	`a = [3, 2, 1, 5]`
`tuple`	Kiểu bộ	`a = (1, 5, 3)`
`dict`	Kiểu từ điển	`b = {'one': 1, 'four': 4}`
`set`	Kiểu tập hợp	`a = {1, 3, 5}`

Bảng 2.1: Các kiểu dữ liệu cơ bản trong Python

Để kiểm tra kiểu dữ liệu của một biến trong Python, ta sử dụng hàm `type()`:

Listing 2.1: Kiểm tra kiểu dữ liệu

```
a = 7
print(type(a))  # Output: <class 'int'>
```

2.2 Toán tử trong Python

Toán tử là thành phần không thể thiếu để thực hiện các phép tính từ cơ bản đến phức tạp trong các thuật toán.

2.2.1 Toán tử cơ bản

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+`	`3 + 7`	`10`
Trừ	`-`	`7 - 2`	`5`
Nhân	`*`	`2 * 7`	`14`
Chia	`/`	`7 / 5`	`1.4`
Chia lấy phần nguyên	`//`	`10 // 4`	`2`
Chia lấy phần dư	`%`	`9 % 5`	`4`
Lấy mũ	`**`	`2 ** 3`	`8`

Bảng 2.2: Các toán tử số học cơ bản

2.2.2 Toán tử gán mở rộng

Để mã nguồn ngắn gọn hơn, Python hỗ trợ các toán tử gán mở rộng kết hợp phép tính và phép gán:

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+=`	`a = 7; a += 3`	`10`
Trừ	`-=`	`b = 9; b -= 4`	`5`
Nhân	`*=`	`a = 2; a *= 7`	`14`
Chia	`/=`	`a = 7; b /= 5`	`1.4`
Chia lấy phần nguyên	`//=`	`a = 10; a //= 4`	`2`
Chia lấy phần dư	`%=`	`a = 9; a %= 5`	`4`
Lấy mũ	`**=`	`a = 2; a **= 3`	`8`

Bảng 2.3: Các toán tử gán mở rộng

2.3 Hàm toán học trong Python

Đối với các ứng dụng khoa học dữ liệu, thư viện `math` cung cấp các hằng số và hàm tính toán nâng cao. Để sử dụng, ta cần import thư viện:

Listing 2.2: Import thư viện math

```
1 import math
```

2.3.1 Các hằng số toán học

- Hằng số Euler (E): `math.e` ≈ 2.718
- Hằng số Pi (π): `math.pi` ≈ 3.14159

2.3.2 Các hàm toán học thường dùng

Tên hàm (Toán học)	Mã lệnh Python
$\ln(x)$	`math.log(math.e)`
$\log_2(x)$	`math.log2(3)`
e^x	`math.exp(3)`
$\log_{10}(x)$	`math.log10(3)`
$\sqrt{x}$	`math.sqrt(3)`
a^x	`math.pow(2, 3)`

Bảng 2.4: Một số hàm toán học từ thư viện math

2.4 Cấu trúc dữ liệu nâng cao

Đối với Machine Learning, ngoài mảng 1 chiều, ta thường sử dụng 3 cấu trúc dữ liệu cơ bản bao gồm List, Tuple và Dictionary.

2.4.1 List (Danh sách)

List là tập hợp các phần tử có thứ tự và có thể thay đổi (mutable).

Listing 2.3: Thao tác cơ bản trên List

```
# Khoi tao list
features = [1.2, 3.4, 5.1, 2.8]

# Truy cap va thay doi phan tu
features[0] = 0.9
features.append(4.5)

print("List sau khi cap nhat:", features)
```

2.5 Cấu trúc điều khiển và Vòng lặp

2.5.1 Câu lệnh điều kiện if-else

Listing 2.4: Sử dụng cấu trúc điều kiện

```
score = 0.85

```

```
if score >= 0.8:
    print("Mo hinh dat do chinh xac cao")
else:
    print("Can can thiep hoac toi uu them")
```

2.6 Tóm tắt chương

Trong chương này, chúng ta đã nắm được:

- Cách khai báo biến và chi tiết các kiểu dữ liệu cơ bản trong Python.
- Các toán tử số học và toán tử gán mở rộng.
- Cách sử dụng thư viện `math` cho các hàm toán học phức tạp.
- Cấu trúc dữ liệu nâng cao: List, Tuple, Dictionary.
- Cấu trúc rẽ nhánh trong Python.

Chương 3: Lập trình hướng đối tượng trong Python

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là phương pháp luận lập trình đóng vai trò then chốt trong việc thiết kế các thư viện Machine Learning hiện đại như Scikit-Learn, PyTorch và TensorFlow. Chương này trình bày các khái niệm OOP cốt lõi và hướng dẫn cách ứng dụng OOP để tự đóng gói một mô hình Machine Learning.

3.1 Lớp và Đối tượng (Class and Object)

Định nghĩa 3.1 (Lớp và Đối tượng)**.** ***Lớp (Class)*** *là khuôn mẫu định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi).* ***Đối tượng (Object)*** *là một thể hiện cụ thể được khởi tạo từ lớp đó.*

Ví dụ 3.1. *Khai báo một lớp cơ bản lưu trữ mẫu dữ liệu:*

Listing 3.1: Khai báo Class và tạo Object trong Python

```
class DataSample:
    def __init__(self, feature_name, value):
        self.feature_name = feature_name  # Thuoc tinh
        self.value = value

    def display_info(self):               # Phuong thuc
        print(f"Feature: {self.feature_name}, Value: {self.value}")

# Khoi tao doi tuong
sample = DataSample(feature_name="Age", value=25)
sample.display_info()
```

3.2 Các phương thức đặc biệt (Magic Methods)

Các phương thức có cú pháp hai dấu gạch dưới (`__method__`) giúp tùy biến hành vi của đối tượng trong Python.

3.2.1 Phương thức `__init__` và `__call__`

- `__init__`: Hàm khởi tạo (constructor), tự động thực thi khi tạo mới một đối tượng.

- `__call__`: Cho phép gọi trực tiếp một đối tượng giống như một hàm, thường dùng khi thiết kế các Layer trong Neural Network.

Listing 3.2: Sử dụng phương thức __call__

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

# Su dung nhu mot ham
double = Multiplier(factor=2)
print(double(5))  # Output: 10
```

3.3 Tính Kế thừa (Inheritance)

Tính kế thừa cho phép tạo một lớp mới dựa trên lớp đã có, giúp tái sử dụng mã nguồn và chuẩn hóa giao diện của các bộ biến đổi dữ liệu (Transformers).

Listing 3.3: Kế thừa lớp BaseTransformer trong tiền xử lý dữ liệu

```
class BaseTransformer:
    def fit(self, arr):
        raise NotImplementedError("Phuong thuc fit() bat buoc phai
            duoc trien khai o lop con.")

    def transform(self, arr):
        raise NotImplementedError("Phuong thuc transform() bat buoc
            phai trien khai o lop con.")

    def fit_transform(self, arr):
        self.fit(arr)
        return self.transform(arr)

class MinMaxScaler(BaseTransformer):
    def fit(self, arr):
        self.min_ = min(arr)
        self.max_ = max(arr)
        return self

    def transform(self, arr):
        return [(x - self.min_) / (self.max_ - self.min_) for x in
            arr]

    def inverse_transform(self, arr):
        return [x * (self.max_ - self.min_) + self.min_ for x in arr]
```

3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)

Trong thiết kế Machine Learning, ta cần phân biệt rõ giữa siêu tham số (cấu hình trước khi huấn luyện) và tham số học được (được sinh ra sau khi gọi hàm `fit`).

Listing 3.4: Bắt lỗi an toàn khi quản lý tham số học được

```
class MinMaxScaler(BaseTransformer):
    def __init__(self, feature_range=(0, 1)):
        self.feature_range = feature_range  # Hyperparameter

    def fit(self, arr):
        self.min = float(min(arr))          # Learned Parameter
        self.max = float(max(arr))          # Learned Parameter
        return self

    def transform(self, arr):
        # Kiem tra an toan: Đam bao mo hinh da duoc fit()
        if not hasattr(self, 'min') or not hasattr(self, 'max'):
            raise RuntimeError("Loi: Ban phai goi ham fit() truoc khi
                goi transform()!")

        low, high = self.feature_range
        std = [(x - self.min) / (self.max - self.min) for x in arr]
        return [x * (high - low) + low for x in std]
```

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn

Trọng tâm ứng dụng OOP trong Machine Learning là đóng gói thuật toán thành một Class có các giao diện chuẩn (`fit`, `predict`, `evaluate`). Dưới đây là triển khai mô hình Hồi quy tuyến tính đơn biến (Simple Linear Regression) bằng công thức đại số.

Listing 3.5: Đóng gói thuật toán Simple Linear Regression

```
class SimpleLinearRegression:
    def __init__(self):
        self.w0 = 0
        self.w1 = 0

    def fit(self, X, y):
        n = len(X)
        mean_x = sum(X) / n
        mean_y = sum(y) / n

        numerator = sum((X[i] - mean_x) * (y[i] - mean_y) for i in
           range(n))
        denominator = sum((X[i] - mean_x) ** 2 for i in range(n))

        if denominator == 0:
```

```
            raise ValueError("Bao loi: Phuong sai cua X bang 0, khong the tinh toan.")

        self.w1 = numerator / denominator
        self.w0 = mean_y - (self.w1 * mean_x)
        return self

    def predict(self, X):
        return [(self.w1 * x_i + self.w0) for x_i in X]

    def evaluate(self, X, y):
        predictions = self.predict(X)
        mse = sum((predictions[i] - y[i]) ** 2 for i in range(len(y))) / len(y)
        return mse
```

3.6 Mẫu thiết kế Pipeline

Pipeline là lớp quản lý chuỗi các bước biến đổi dữ liệu nối tiếp nhau, cho phép tự động hóa toàn bộ quy trình tiền xử lý trước khi đưa vào mô hình dự đoán.

Listing 3.6: Triển khai cấu trúc Pipeline cơ bản

```
class Pipeline:
    def __init__(self, steps):
        self.steps = steps

    def fit_transform(self, arr):
        data = arr
        for transformer in self.steps:
            data = transformer.fit_transform(data)
        return data

    def transform(self, arr):
        data = arr
        for transformer in self.steps:
            data = transformer.transform(data)
        return data
```

3.7 Tóm tắt chương

- Khái niệm Lớp, Đối tượng và tư duy lập trình hướng đối tượng.
- Ứng dụng các Magic Methods (`__init__`, `__call__`) trong xây dựng Framework.
- Nguyên lý Kế thừa giúp tái sử dụng và quản lý mã nguồn linh hoạt.
- Đóng gói thuật toán Machine Learning thành Class theo giao diện tiêu chuẩn.
- Phân biệt rõ ràng siêu tham số và tham số học được thông qua các thuộc tính của đối tượng

Chương 4: Xử Lý Dữ Liệu Với NumPy Và Pandas (NumPy & Pandas)

Trong khoa học dữ liệu và học máy, việc xử lý dữ liệu đóng vai trò nền tảng. Dữ liệu thực tế thường lộn xộn, thiếu sót hoặc không đúng định dạng. Để giải quyết vấn đề này, Python cung cấp hệ sinh thái các thư viện mạnh mẽ, trong đó nổi bật nhất là NumPy và Pandas. Chương này sẽ hướng dẫn bạn cách sử dụng hai thư viện này để thực hiện các thao tác tiền xử lý, phân tích và biến đổi dữ liệu một cách hiệu quả.

4.1 Thư viện NumPy - Tính toán Khoa học

NumPy (Numerical Python) là thư viện cốt lõi cho tính toán khoa học trong Python. Nó cung cấp đối tượng mảng đa chiều hiệu suất cao và các công cụ để làm việc với các mảng này.

4.1.1 Khởi tạo mảng và kiểm tra thuộc tính

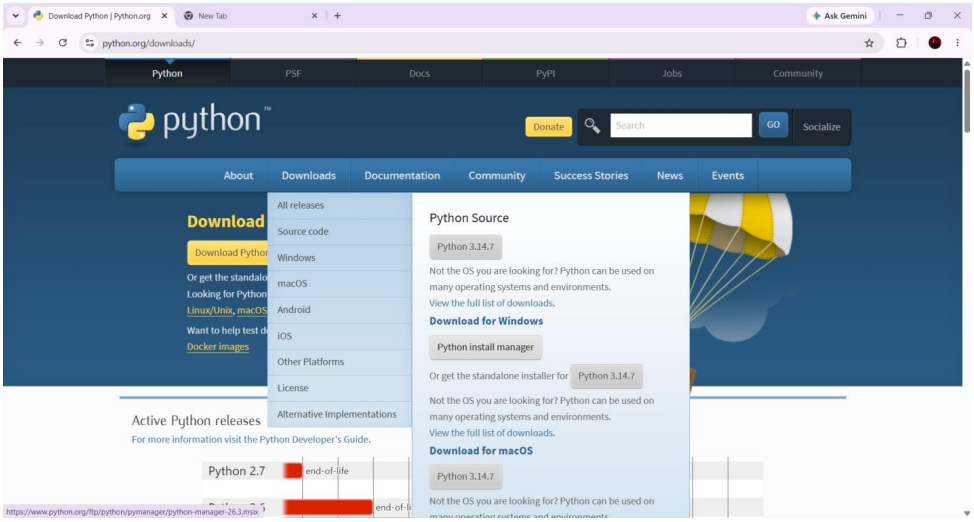
NumPy hỗ trợ tạo mảng đa chiều từ list của Python hoặc sử dụng các hàm có sẵn để khởi tạo các ma trận đặc biệt.

```
import numpy as np

arr1D = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Shape 1D:", arr1D.shape, "| Dtype:", arr1D.dtype)

arr2D = np.array([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])
print("Shape 2D:", arr2D.shape, "| So chieu (ndim):", arr2D.ndim)


arr_zeros = np.zeros((3, 4))
arr_ones = np.ones((2, 5))
arr_eye = np.identity(3)
arr_random = np.random.randn(3, 4)
```



Hình 4.1: Khởi tạo và kiểm tra thuộc tính mảng NumPy

4.1.2 Thao tác trên mảng: Thêm phần tử, xóa, sửa

Trong quá trình xử lý, việc thay đổi cấu trúc mảng bằng cách thêm, xóa, sửa phần tử là rất thường xuyên. NumPy cung cấp các hàm như `append`, `insert`, và `delete` để thao tác với phần tử mảng.

```
arr1D[0] = 99

arr_appended = np.append(arr1D, [11, 12])
arr_inserted = np.insert(arr1D, 2, 999)

arr_deleted = np.delete(arr1D, 0)
```

4.1.3 Slicing và Boolean Indexing (Lọc dữ liệu)

Slicing cho phép lấy một phần của mảng, trong khi Boolean Indexing cho phép lọc các phần tử thỏa mãn điều kiện nhất định.

```

sub_arr = arr2D[:2, :3]
print("Cat mang [:2, :3]:\n", sub_arr)


city_data = np.random.randn(5, 3)
pos_values = city_data[city_data > 0]
print("Cac gia tri > 0:", pos_values)

city_data[city_data > 0] = 0
print("Ma tran sau khi thay the:\n", city_data)
```

4.1.4 Đại số tuyến tính (Linear Algebra)

NumPy tích hợp thư viện `linalg` cung cấp các hàm mạnh mẽ cho đại số tuyến tính như tính tích ma trận, ma trận nghịch đảo, giải hệ phương trình tuyến tính, v.v.

```
A = np.array([[7, 5, -3],
              [3, -5, 2],
              [5, 3, -7]])
b = np.array([16, -8, 0])


x = np.linalg.solve(A, b)
print("Nghiem x, y, z:", x)
print("Kiem tra nghiem dung:", np.allclose(np.dot(A, x), b))
```

4.2 Thư viện Pandas - Xử lý và Phân tích Dữ liệu

Pandas được xây dựng dựa trên NumPy, cung cấp các cấu trúc dữ liệu linh hoạt là `Series` (1D) và `DataFrame` (2D), đặc biệt phù hợp cho dữ liệu dạng bảng.

4.2.1 Tạo và Đọc dữ liệu

Pandas cung cấp hàm `read_csv()` để đọc dữ liệu từ tệp CSV và các hàm như `head()`, `tail()` để quan sát dữ liệu. Ngoài ra, ta có thể khởi tạo DataFrame từ Dictionary hoặc List.

```
import pandas as pd

data_dict = [
    {'city': 'Delhi', 'data': 1000},
    {'city': 'Bangalore', 'data': 2000},
    {'city': 'Mumbai', 'data': 1500}
]
df_dict = pd.DataFrame(data_dict)

data_list = [[1, 'Apple'], [2, 'Banana']]
df_list = pd.DataFrame(data_list, columns=['ID', 'Fruit'])

data_vnm = {
    'Date': ['10-01-2023', '10-02-2023', '10-03-2023', '10-04-2023',
       '10-05-2023', '10-06-2023', '10-07-2023'],
    'Ticker': ['VNM', 'VNM', 'VNM', 'VNM', 'VNM', 'VNM', 'VNM'],
    'Open': [75.5, 76.0, 75.8, 77.2, 76.5, 78.0, 77.8],
    'Close': [76.0, 75.5, 77.0, 76.5, 78.0, 77.5, 78.2],
    'Volume': [12000, 15000, 13500, 18000, 16000, 20000, 19500]
}
df_mock = pd.DataFrame(data_vnm)


df_mock.to_csv('vinamilk_stock.csv', index=False)
```

```
print("Đã ạto thành công file 'vinamilk_stock.csv' trong ưth ụmc ệhin
    ạti!")

df_stock = pd.read_csv('vinamilk_stock.csv')

print("--- 5 DONG DAU CUA DU LIEU ---")
print(df_stock.head())

print("\n--- 3 DONG CUOI CUA DU LIEU ---")
print(df_stock.tail(3))
```



Hình 4.2: Tạo và Đọc dữ liệu với Pandas

4.2.2 Truy xuất & Điều kiện lọc dữ liệu

Hai thuộc tính quan trọng để truy cập dữ liệu trong Pandas là `loc` (theo nhãn) và `iloc` (theo vị trí số nguyên).

```
data_fict = {
    'price': [100, 200, 300],
    'quantity_purchased': [10, 20, 32],
    'user_type': ['A', 'B', 'C'],
    'user_class': ['X', 'Y', 'Z']
}
df = pd.DataFrame(data_fict)
print(df.iloc[:5, :4])
df_filtered = df.loc[df['quantity_purchased'] > 25]
```

4.2.3 Xử lý dữ liệu khuyết & Thống kê mô tả

Pandas cung cấp các công cụ để phát hiện và xử lý dữ liệu bị thiếu (`NaN`) cũng như các hàm để mô tả nhanh bộ dữ liệu như `describe()` hay `info()`.

```
print(df.describe())

mean_price = df['price'].mean()
df['price'].fillna(value=mean_price, inplace=True)
```

4.2.4 Thay thế và Áp dụng hàm

Các phương thức `replace()` và `apply()` là những công cụ mạnh mẽ để thay đổi hoặc biến đổi dữ liệu trên cột.

```
df['user_type'] = df['user_type'].replace({'A': 'Premium', 'B': '
    Standard'})

df['discounted_price'] = df['price'].apply(lambda x: x * 0.9)
```

4.2.5 Gom nhóm (GroupBy) & Gộp dữ liệu (Concat/Merge)

Hàm `groupby()` rất hữu ích để nhóm các dòng theo giá trị của một cột và thực hiện các phép toán thống kê.

```
grouped = df.groupby('user_class')['quantity_purchased'].agg(['sum',
    'mean', 'count'])
print(grouped)
```

Chương 5: Tiền xử lý dữ liệu (Data Preprocessing)

Các thuật toán học máy (Machine Learning – ML) cổ điển như hồi quy tuyến tính, KNN, K-Means, SVM hay PCA đều được xây dựng trên nền tảng đại số tuyến tính và giải tích: chúng nhận đầu vào là **các vector số thực** và thực hiện phép cộng, phép nhân, phép tính khoảng cách giữa các vector đó. Trong khi đó, dữ liệu ngoài đời thực lại rất đa dạng: tên màu sắc, tên khu phố, đoạn văn bản bình luận, mức đánh giá "tốt – trung bình – kém", diện tích nhà tính bằng feet vuông bên cạnh số phòng tắm chỉ từ 0 đến 4… Khoảng cách giữa "dữ liệu thô" và "dữ liệu mà mô hình hiểu được" chính là đối tượng nghiên cứu của chương này.

> *Nguyên tắc cốt lõi:* Mô hình ML không làm việc được trực tiếp trên văn bản hay nhãn chữ — **mọi thứ đều phải được biến đổi thành số**, và phải biến đổi sao cho **không làm sai lệch ý nghĩa** của dữ liệu.

Sau khi học xong chương này, người học có khả năng:

- Trình bày được nguồn gốc lịch sử và vị trí của tiền xử lý dữ liệu trong quy trình khai phá tri thức (KDD) và CRISP-DM.
- Phân biệt các kiểu dữ liệu (định danh, thứ bậc, số, văn bản) và chọn phép biến đổi phù hợp cho từng kiểu.
- Xử lý dữ liệu thiếu bằng **Simple Imputer**, **KNN Imputer** (khoảng cách nan-Euclid) và **Iterative Imputer**; phát hiện ngoại lai bằng z-score, quy tắc IQR và **Local Outlier Factor**.
- Mã hóa dữ liệu phân loại bằng **Label Encoder** và **One Hot Encoder**, hiểu rõ ưu nhược điểm của từng cách.
- Vector hóa văn bản bằng **Count Vectorizer** và **TF-IDF Vectorizer**, tự tính tay được ma trận TF-IDF.
- Chuẩn hóa thang đo đặc trưng bằng **Min Max Scaler**, **Max Abs Scaler**, **Standard Scaler** và **Robust Scaler**.
- Xử lý dữ liệu mất cân bằng bằng **SMOTE** và đặt bước lấy mẫu đúng vị trí trong pipeline.
- Xây dựng quy trình tiền xử lý hoàn chỉnh, tránh rò rỉ dữ liệu, trên bộ dữ liệu thực tế **Ames Housing**.

5.1 Khởi nguồn và vai trò của tiền xử lý dữ liệu

5.1.1 "Rác vào, rác ra"

Ngay từ những ngày đầu của ngành máy tính, giới lập trình viên đã lưu truyền nguyên lý *Garbage In, Garbage Out* (GIGO – "rác vào, rác ra"), thường được gán cho George Fuechsel, một kỹ sư IBM vào cuối thập niên 1950. Nguyên lý này nói rằng một chương trình dù đúng đến đâu cũng chỉ cho ra kết quả tốt khi dữ liệu đầu vào tốt. Đối với học máy, nguyên lý này còn nghiêm trọng hơn: mô hình *học* trực tiếp từ dữ liệu, nên mọi sai lệch, nhiễu hay cách mã hóa không phù hợp đều được mô hình "học thuộc" và khuếch đại thành dự đoán sai.

5.1.2 Tiền xử lý trong quy trình khai phá tri thức (KDD)

Năm 1996, Fayyad, Piatetsky-Shapiro và Smyth công bố bài báo kinh điển *From Data Mining to Knowledge Discovery in Databases* trên tạp chí *AI Magazine*, chính thức hóa quy trình **khám phá tri thức từ cơ sở dữ liệu** (Knowledge Discovery in Databases – KDD). Trong quy trình này, "khai phá dữ liệu" (tức bước chạy thuật toán) chỉ là **một** bước, đứng sau hàng loạt bước chuẩn bị:

1. **Lựa chọn** (Selection): chọn tập dữ liệu, biến quan tâm.
2. **Tiền xử lý** (Preprocessing): làm sạch, xử lý nhiễu, xử lý giá trị thiếu.
3. **Biến đổi** (Transformation): mã hóa, chuẩn hóa, giảm chiều.
4. **Khai phá dữ liệu** (Data Mining): áp dụng thuật toán.
5. **Diễn giải / Đánh giá** (Interpretation/Evaluation).

[Chỗ chèn hình]
`images/ch-preprocessing-kdd.jpg`

Hình 5.1: Quy trình KDD theo Fayyad và cộng sự (1996). Tiền xử lý và biến đổi dữ liệu đứng trước bước khai phá dữ liệu.(https://onlinelibrary.wiley.com/doi/epdf/10.1609/aimag.v17i3.1230)

Cùng thời điểm, một liên minh công nghiệp gồm DaimlerChrysler, SPSS và NCR phát triển chuẩn **CRISP-DM** (Cross-Industry Standard Process for Data Mining, công bố bản 1.0 năm 2000) gồm sáu pha: *Hiểu nghiệp vụ – Hiểu dữ liệu – Chuẩn bị dữ liệu – Mô hình hóa – Đánh giá – Triển khai*. Pha *Chuẩn bị dữ liệu* được nhiều tài liệu thực hành ước tính chiếm phần lớn thời gian của một dự án (con số thường được trích dẫn là khoảng 60–80%). Cuốn *Data Preparation*

for Data Mining của Dorian Pyle (1999) là một trong những chuyên khảo đầu tiên dành riêng cho chủ đề này.

[Chỗ chèn hình]

`images/ch-preprocessing-crispdm.jpg`

Hình 5.2: Sáu pha của CRISP-DM. Pha "Chuẩn bị dữ liệu" là nơi diễn ra phần lớn các kỹ thuật trong chương này.(https://rdsic.edu.vn/blog/blog-6/1549734giai-doan-nao-trong-crisp-dm-xem-xet-cac-dinh-dang-du-lieu-cua-nguon-du-lieu-trong-a-to-vi-cb.html)

5.1.3 Phân loại các tác vụ tiền xử lý

Theo Han, Kamber và Pei (*Data Mining: Concepts and Techniques*), tiền xử lý dữ liệu gồm bốn nhóm tác vụ chính:

Nhóm tác vụ	**Nội dung**
Làm sạch dữ liệu	Xử lý giá trị thiếu (**Simple / KNN / Iterative Imputer**), dữ liệu nhiễu, **ngoại lai** (IQR, LOF), dữ liệu sai định dạng.
Tích hợp dữ liệu	Gộp dữ liệu từ nhiều nguồn, xử lý trùng lặp và mâu thuẫn.
Biến đổi dữ liệu	**Mã hóa** (Label/One Hot Encoder), **vector hóa văn bản** (Count/TF-IDF), **chuẩn hóa thang đo** (các Scaler).
Rút gọn dữ liệu	Giảm số chiều (PCA, SVD – xem chương Giảm chiều dữ liệu), giảm số mẫu.

Bảng 5.1: Bốn nhóm tác vụ tiền xử lý dữ liệu

Chương này đi theo đúng trình tự của quy trình KDD: trước hết là nhóm **làm sạch dữ liệu** (điền giá trị thiếu, phát hiện ngoại lai), sau đó là nhóm **biến đổi dữ liệu** — nhóm kỹ thuật mà gần như mọi bài toán ML đều phải dùng — rồi đến kỹ thuật **cân bằng dữ liệu** SMOTE, và kết thúc bằng một nghiên cứu tình huống trên dữ liệu thực.

5.1.4 Thư viện scikit-learn và giao diện `fit` / `transform`

Toàn bộ kỹ thuật trong chương được cài đặt bằng thư viện **scikit-learn**. Dự án bắt đầu năm 2007 như một dự án Google Summer of Code của David Cournapeau, sau đó được nhóm nghiên cứu tại INRIA (Pháp) phát triển và phát hành công khai năm 2010; bài báo giới thiệu chính thức của Pedregosa và cộng sự đăng trên *Journal of Machine Learning Research* năm 2011.

Điểm mạnh của scikit-learn là **giao diện thống nhất** (Buitinck và cộng sự, 2013): mọi bộ biến đổi (transformer) đều có ba phương thức:

Phương thức	Ý nghĩa
`fit(X)`	**Học** các tham số từ dữ liệu (từ điển nhãn, min/max, trung bình, độ lệch chuẩn…).
`transform(X)`	**Áp dụng** các tham số đã học để biến đổi dữ liệu.
`fit_transform(X)`	Gộp hai bước trên (chỉ dùng trên tập huấn luyện).
`inverse_transform(X)`	Biến đổi ngược về giá trị gốc (khi phép biến đổi khả nghịch).

Bảng 5.2: Giao diện chung của các bộ biến đổi trong scikit-learn

Định nghĩa 5.1 (Rò rỉ dữ liệu – Data leakage)**.** *Rò rỉ dữ liệu xảy ra khi thông tin từ tập kiểm tra (test) được sử dụng, trực tiếp hay gián tiếp, trong quá trình huấn luyện. Với tiền xử lý, lỗi điển hình là gọi* `fit` *trên* ***toàn bộ*** *dữ liệu trước khi chia train/test. Quy tắc đúng:* ***chỉ*** `fit` ***trên tập huấn luyện****, sau đó dùng* `transform` *cho cả tập huấn luyện và tập kiểm tra.*

5.2 Các kiểu dữ liệu và lựa chọn phép biến đổi

Trước khi chọn kỹ thuật, cần xác định **thang đo** của từng biến. Stevens (1946) phân biệt bốn thang đo: định danh (nominal), thứ bậc (ordinal), khoảng (interval) và tỉ lệ (ratio). Trong thực hành ML, ta gom thành các nhóm sau:

Kiểu dữ liệu	Đặc điểm	Ví dụ	Phép biến đổi
Định danh (nominal)	Các giá trị chỉ khác nhau, **không có thứ tự**	màu sắc, khu phố	One Hot Encoder
Thứ bậc (ordinal)	Có **thứ tự** nhưng khoảng cách không rõ ràng	kém $<$ TB $<$ tốt	Ordinal Encoder
Nhãn đầu ra (target)	Biến cần dự đoán trong phân lớp	“spam”/“ham”	Label Encoder
Số (numeric)	Có giá trị và khoảng cách thực	diện tích, giá	Các Scaler
Văn bản (text)	Chuỗi từ có độ dài thay đổi	bình luận, tài liệu	Count / TF-IDF Vectorizer

Bảng 5.3: Kiểu dữ liệu và kỹ thuật tiền xử lý tương ứng

5.3 Xử lý dữ liệu thiếu (Missing Values)

Dữ liệu thiếu xuất hiện trong hầu hết các bộ dữ liệu thực tế: người được khảo sát bỏ trống câu hỏi, cảm biến mất tín hiệu, dữ liệu ghép từ nhiều nguồn không khớp nhau… Trong NumPy và pandas, giá trị thiếu được biểu diễn bằng `NaN` (*Not a Number* — một giá trị đặc biệt của chuẩn số thực dấu phẩy động IEEE 754). Phần lớn mô hình của scikit-learn (hồi quy tuyến tính, SVM, KNN, PCA…) sẽ **báo lỗi** khi gặp `NaN`, vì vậy xử lý dữ liệu thiếu là bước bắt buộc trước khi huấn luyện.

5.3.1 Cơ chế sinh dữ liệu thiếu

Donald Rubin (1976) đặt nền móng lý thuyết cho bài toán dữ liệu thiếu khi phân biệt ba **cơ chế thiếu**:

Cơ chế	Ý nghĩa	Ví dụ
MCAR – Missing Completely At Random	Việc thiếu hoàn toàn ngẫu nhiên, không phụ thuộc vào biến nào	Phiếu khảo sát bị thất lạc ngẫu nhiên
MAR – Missing At Random	Việc thiếu chỉ phụ thuộc vào các biến **quan sát được**	Người trẻ ít khai thu nhập hơn (tuổi đã biết)
MNAR – Missing Not At Random	Việc thiếu phụ thuộc vào chính **giá trị bị thiếu**	Người thu nhập rất cao từ chối khai thu nhập

Bảng 5.4: Ba cơ chế thiếu dữ liệu theo Rubin (1976)

Xóa các dòng có ô trống (`df.dropna()`) chỉ an toàn khi dữ liệu thiếu kiểu MCAR và chiếm tỉ lệ nhỏ; trong các trường hợp còn lại, xóa dòng vừa lãng phí dữ liệu vừa làm **lệch** mẫu. Giải pháp phổ biến hơn là **điền giá trị thiếu** (imputation). scikit-learn cung cấp ba bộ điền với mức độ phức tạp tăng dần: `SimpleImputer`, `KNNImputer` và `IterativeImputer`. Giống mọi bộ biến đổi khác, bộ điền phải được `fit` **chỉ trên tập huấn luyện**.

5.3.2 Simple Imputer: trung bình, trung vị, giá trị phổ biến

Cách đơn giản nhất là thay mọi ô thiếu của một cột bằng một **thống kê của chính cột đó**, tính trên các giá trị quan sát được:

- `strategy="mean"` – trung bình cộng: phù hợp với dữ liệu gần đối xứng, ít ngoại lai.
- `strategy="median"` – trung vị: bền vững với ngoại lai và dữ liệu lệch.
- `strategy="most_frequent"` – giá trị xuất hiện nhiều nhất (mode): dùng được cho cả cột chữ.
- `strategy="constant"` – một hằng số do người dùng chọn (ví dụ `"NA"` hoặc 0).

Ví dụ 5.1 (Điền bằng trung bình và trung vị)**.** *Hai cột dữ liệu, mỗi cột thiếu một giá trị:*

$$A = (7,\ 3,\ 4,\ 6,\ 2,\ ?), \qquad B = (6,\ 8,\ 9,\ 8,\ 7,\ ?).$$

Cách 1 – theo trung bình:

$$\bar{A} = \frac{7+3+4+6+2}{5} = \frac{22}{5} = 4{,}4, \qquad \bar{B} = \frac{6+8+9+8+7}{5} = \frac{38}{5} = 7{,}6.$$

Cách 2 – theo trung vị: *sắp xếp các giá trị quan sát được rồi chọn phần tử ở giữa:* $A \to (2, 3, \mathbf{4}, 6, 7)$*, điền 4;* $B \to (6, 7, \mathbf{8}, 8, 9)$*, điền 8.*

Khi số giá trị quan sát được là ***chẵn****, trung vị là trung bình cộng của hai phần tử ở giữa. Ví dụ với* $(6, 9, 7, 3, 2, 5)$*: sắp xếp thành* $(2, 3, \mathbf{5}, \mathbf{6}, 7, 9)$*, trung vị bằng* $\frac{5+6}{2} = 5{,}5$.

Listing 5.1: SimpleImputer với ví dụ trên

```
import numpy as np
from sklearn.impute import SimpleImputer

X = np.array([[7, 6], [3, 8], [4, 9], [6, 8], [2, 7], [np.nan, np.nan
    ]])
for s in ["mean", "median", "most_frequent"]:
    imp = SimpleImputer(strategy=s).fit(X)
    print(s, imp.statistics_)
# mean          [4.4 7.6]
# median        [4.  8. ]
# most_frequent [2.  8. ]  (cot A khong co gia tri lap -> lay gia tri
    nho nhat)
print(np.median([6, 9, 7, 3, 2, 5]))     # 5.5
```

Hạn chế. Điền bằng một hằng số làm **giảm phương sai** của cột và **làm yếu tương quan** giữa các cột, vì mọi ô thiếu nhận cùng một giá trị bất kể các thuộc tính còn lại của bản ghi. Tham số `add_indicator=True` bổ sung cột nhị phân đánh dấu ô nào đã được điền — bản thân sự kiện "bị thiếu" nhiều khi cũng mang thông tin có ích cho mô hình.

5.3.3 KNN Imputer và khoảng cách nan-Euclid

Troyanskaya và cộng sự (2001) đề xuất điền giá trị thiếu trong dữ liệu biểu hiện gen bằng các **láng giềng gần nhất**: hai bản ghi giống nhau ở các cột quan sát được thì nhiều khả năng cũng giống nhau ở cột bị thiếu. Thuật toán KNN được trình bày đầy đủ ở Chương 7; ở đây chỉ cần ý tưởng: *giá trị điền là trung bình của cột đó trên k bản ghi gần nhất*.

Khó khăn là chính các bản ghi cũng có ô trống, nên không tính được khoảng cách Euclid thông thường. Ta dùng khoảng cách **nan-Euclid**: chỉ cộng trên những tọa độ mà **cả hai** điểm đều có giá trị, rồi nhân với một hệ số để bù cho các tọa độ bị bỏ qua.

Định nghĩa 5.2 (Khoảng cách nan-Euclid)**.** *Với hai vector $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ có thể chứa giá trị thiếu, gọi P là tập các tọa độ mà cả hai vector cùng có giá trị:*

$$d_{nan}(\mathbf{x}, \mathbf{y}) = \sqrt{\frac{d}{|P|} \sum_{j \in P} (x_j - y_j)^2}, \tag{5.1}$$

*trong đó d là **tổng số chiều** của bài toán và $|P|$ là **số chiều tính được**.*

Ví dụ 5.2 (Tính khoảng cách nan-Euclid)**.**

- $\mathbf{x} = (5, \mathit{nan})$ *và* $\mathbf{y} = (4, 5)$*: chỉ tọa độ thứ nhất tính được, $d = 2$, $|P| = 1$:*

$$d_{nan} = \sqrt{\tfrac{2}{1}(5-4)^2} = \sqrt{2} \approx 1{,}414.$$

- $\mathbf{A} = (3, \mathit{nan}, \mathit{nan}, 6)$ *và* $\mathbf{B} = (1, \mathit{nan}, 4, 5)$*: tính được tọa độ 1 và 4, $d = 4$, $|P| = 2$:*

$$d_{nan} = \sqrt{\tfrac{4}{2}\left[(3-1)^2 + (6-5)^2\right]} = \sqrt{10} \approx 3{,}162.$$

Ví dụ 5.3 (Chạy tay KNN Imputer với $k = 3$)**.** *Điền ba ô thiếu trong Bảng 5.5.*

STT	x_1	x_2	x_3
1	7	6	4
2	**?**	4	5
3	1	5	**?**
4	3	9	7
5	8	**?**	8

Bảng 5.5: Dữ liệu ví dụ cho KNN Imputer

Bước 1 – xác định X và y. Để điền x_1 ở dòng 2, coi cột x_1 là "nhãn" y, hai cột còn lại (x_2, x_3) là đặc trưng X. Điểm cần dự đoán là $\mathbf{q} = (4, 5)$; các dòng được làm láng giềng phải **có** giá trị x_1 (dòng 1, 3, 4, 5).
Bước 2 – tính khoảng cách nan-Euclid trên hai cột (x_2, x_3), $d = 2$:

$$
\begin{aligned}
d(\mathbf{q}, \text{dòng } 1) &= \sqrt{(4-6)^2 + (5-4)^2} = \sqrt{5} \approx 2{,}236, \\
d(\mathbf{q}, \text{dòng } 3) &= \sqrt{\tfrac{2}{1}(4-5)^2} = \sqrt{2} \approx 1{,}414 && \text{(dòng 3 thiếu } x_3), \\
d(\mathbf{q}, \text{dòng } 4) &= \sqrt{(4-9)^2 + (5-7)^2} = \sqrt{29} \approx 5{,}385, \\
d(\mathbf{q}, \text{dòng } 5) &= \sqrt{\tfrac{2}{1}(5-8)^2} = \sqrt{18} \approx 4{,}243 && \text{(dòng 5 thiếu } x_2).
\end{aligned}
$$

Bước 3 – lấy trung bình của $k = 3$ láng giềng gần nhất (dòng 3, 1, 5): $x_1 = \frac{1+7+8}{3} \approx 5{,}333$.
Làm tương tự cho hai ô còn lại, ta được bảng tổng hợp:

Ô cần điền	**Điểm q**	**Khoảng cách đến các dòng**	**3 láng giềng**	**Giá trị điền**
x_1, dòng 2	$(x_2, x_3) = (4, 5)$	$d_1 = 2{,}236$; $d_3 = 1{,}414$; $d_4 = 5{,}385$; $d_5 = 4{,}243$	3, 1, 5	$\frac{1+7+8}{3} \approx 5{,}333$
x_3, dòng 3	$(x_1, x_2) = (1, 5)$	$d_1 = 6{,}083$; $d_2 = 1{,}414$; $d_4 = 4{,}472$; $d_5 = 9{,}899$	2, 4, 1	$\frac{5+7+4}{3} \approx 5{,}333$
x_2, dòng 5	$(x_1, x_3) = (8, 8)$	$d_1 = 4{,}123$; $d_2 = 4{,}243$; $d_3 = 9{,}899$; $d_4 = 5{,}099$	1, 2, 4	$\frac{6+4+9}{3} \approx 6{,}333$

Bảng 5.6: Kết quả chạy tay KNN Imputer ($k = 3$)

Lưu ý.

- Hệ số $d/|P|$ phải được áp dụng **mỗi khi** có tọa độ bị thiếu. Quên hệ số này — chẳng hạn viết $d(\mathbf{q}, \text{dòng } 5) = \sqrt{(5-8)^2} = 3$ thay vì $\sqrt{18} \approx 4{,}243$ — là lỗi hay gặp khi tính tay. Ở ví dụ này lỗi đó tình cờ không làm đổi tập láng giềng, nhưng trong trường hợp tổng quát sẽ cho kết quả sai.

- `KNNImputer` của scikit-learn đo khoảng cách trên **toàn bộ** 3 cột. Tọa độ đang cần điền của dòng nhận vốn bị thiếu nên tự động bị loại, và hệ số trở thành $3/|P|$ thay vì $2/|P|$. Mọi khoảng cách từ dòng nhận vì thế cùng được nhân với $\sqrt{3/2}$, nên thứ tự láng giềng và giá trị điền **không đổi**.

- KNN Imputer dựa trên khoảng cách nên cần **chuẩn hóa thang đo** (mục 5.7) trước khi dùng; tham số `weights="distance"` cho láng giềng gần hơn trọng số lớn hơn.

Listing 5.2: Khoảng cách nan-Euclid và KNNImputer

```
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.metrics.pairwise import nan_euclidean_distances

nan = np.nan
print(nan_euclidean_distances([[5, nan]], [[4, 5]]))
                    # [[1.4142]]
print(nan_euclidean_distances([[3, nan, nan, 6]], [[1, nan, 4, 5]]))
    # [[3.1623]]

X = np.array([[7, 6, 4], [nan, 4, 5], [1, 5, nan], [3, 9, 7], [8, nan
   , 8]])
print(KNNImputer(n_neighbors=3).fit_transform(X).round(3))
# [[7.    6.    4.   ]
#  [5.333 4.    5.   ]
#  [1.    5.    5.333]
#  [3.    9.    7.   ]
#  [8.    6.333 8.   ]]
```

5.3.4 Iterative Imputer

Simple Imputer bỏ qua quan hệ giữa các cột; KNN Imputer khai thác quan hệ đó một cách cục bộ. **Iterative Imputer** đi xa hơn: *mô hình hóa mỗi cột có giá trị thiếu như một hàm hồi quy của các cột còn lại*. Ý tưởng này bắt nguồn từ kỹ thuật **MICE** (Multivariate Imputation by Chained Equations) của van Buuren và Groothuis-Oudshoorn (2011), vốn phát triển từ khung *điền khuyết bội* (multiple imputation) của Rubin (1987). Trong scikit-learn, lớp `IterativeImputer` vẫn được xếp vào nhóm thử nghiệm, nên cần khai báo `from sklearn.experimental import enable_iterative_imputer` trước khi dùng.

Thuật toán:

1. **Khởi tạo**: điền tạm mọi ô thiếu bằng trung bình cột.
2. Lần lượt với từng cột x_j có ô thiếu: coi x_j là biến mục tiêu y, các cột còn lại (với giá trị đang điền tạm) là X; huấn luyện mô hình hồi quy trên các dòng mà x_j được quan sát; dự đoán và **cập nhật** các ô thiếu của x_j.
3. Lặp lại bước 2 cho đến khi các giá trị điền ổn định hoặc đạt số vòng tối đa `max_iter`.

Bộ hồi quy ở bước 2 có thể là bất kỳ mô hình nào. Trong ví dụ dưới đây ta dùng hồi quy tuyến tính, giải bằng phương trình chuẩn $\boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ (Chương **??**); cột đầu tiên của $\mathbf{X}$ toàn số 1 ứng với hệ số chặn β_0.

Ví dụ 5.4 (Chạy tay một vòng Iterative Imputer)**.** *Điền ba ô thiếu trong Bảng 5.7 theo thứ tự cột $x_1 \to x_2 \to x_3$.*

STT	x_1	x_2	x_3
1	3	7	1
2	**?**	4	8
3	6	3	7
4	5	**?**	4
5	1	9	**?**

Bảng 5.7: Dữ liệu ví dụ cho Iterative Imputer

Bước 1 – khởi tạo bằng trung bình cột:

$$\bar{x}_1 = \frac{3+6+5+1}{4} = 3{,}75, \qquad \bar{x}_2 = \frac{7+4+3+9}{4} = 5{,}75, \qquad \bar{x}_3 = \frac{1+8+7+4}{4} = 5.$$

Bước 2 – dự đoán x_1 từ (x_2, x_3). Dùng bốn dòng có x_1 (dòng 4 đang mang giá trị tạm $x_2 = 5{,}75$, dòng 5 mang $x_3 = 5$):

$$\mathbf{X} = \begin{pmatrix} 1 & 7 & 1 \\ 1 & 3 & 7 \\ 1 & 5{,}75 & 4 \\ 1 & 9 & 5 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 6 \\ 5 \\ 1 \end{pmatrix}, \quad \mathbf{X}^\top\mathbf{X} = \begin{pmatrix} 4 & 24{,}75 & 17 \\ 24{,}75 & 172{,}06 & 96 \\ 17 & 96 & 91 \end{pmatrix}, \quad \mathbf{X}^\top\mathbf{y} = \begin{pmatrix} 15 \\ 76{,}75 \\ 70 \end{pmatrix},$$

$$(\mathbf{X}^\top\mathbf{X})^{-1} \approx \begin{pmatrix} 5{,}9563 & -0{,}5735 & -0{,}5077 \\ -0{,}5735 & 0{,}0693 & 0{,}0340 \\ -0{,}5077 & 0{,}0340 & 0{,}0700 \end{pmatrix}, \qquad \boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \approx \begin{pmatrix} 9{,}78895 \\ -0{,}90153 \\ -0{,}10841 \end{pmatrix}.$$

Thay dòng 2 ($x_2 = 4$, $x_3 = 8$): $x_1 = 9{,}78895 - 0{,}90153 \times 4 - 0{,}10841 \times 8 \approx 5{,}3155$.
Bước 3 – dự đoán x_2 từ (x_1, x_3), thế giá trị vừa điền $x_1 = 5{,}3155$ vào dòng 2 (dòng 5 vẫn mang $x_3 = 5$):

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 1 \\ 1 & 5{,}3155 & 8 \\ 1 & 6 & 7 \\ 1 & 1 & 5 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 7 \\ 4 \\ 3 \\ 9 \end{pmatrix} \Rightarrow \boldsymbol{\beta} \approx \begin{pmatrix} 10{,}4972 \\ -1{,}1578 \\ -0{,}0598 \end{pmatrix}.$$

Thay dòng 4 ($x_1 = 5$, $x_3 = 4$): $x_2 = 10{,}4972 - 1{,}1578 \times 5 - 0{,}0598 \times 4 \approx 4{,}469$.
Bước 4 – dự đoán x_3 từ (x_1, x_2) với các giá trị vừa điền (tính với đầy đủ chữ số):

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 7 \\ 1 & 5{,}3155 & 4 \\ 1 & 6 & 3 \\ 1 & 5 & 4{,}4689 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 8 \\ 7 \\ 4 \end{pmatrix} \Rightarrow \boldsymbol{\beta} \approx \begin{pmatrix} 9{,}7807 \\ 0{,}3487 \\ -1{,}4001 \end{pmatrix}.$$

Thay dòng 5 ($x_1 = 1$, $x_2 = 9$): $x_3 = 9{,}7807 + 0{,}3487 \times 1 - 1{,}4001 \times 9 \approx -2{,}47$.

Kết thúc vòng thứ nhất: $x_1 \approx 5{,}32$; $x_2 \approx 4{,}47$; $x_3 \approx -2{,}47$.

Phân tích kết quả: bài toán điều kiện xấu. Giá trị $x_3 \approx -2{,}47$ nằm **ngoài** miền quan sát $[1;\ 8]$ của cột x_3 — một dấu hiệu cảnh báo. Nguyên nhân nằm ở bốn dòng dùng để hồi quy ở bước 4: tổng $x_1 + x_2$ của chúng lần lượt là 10; $9{,}32$; 9; $9{,}47$, tức hai đặc trưng gần như phụ thuộc tuyến tính (**đa cộng tuyến**). Khi đó $\mathbf{X}^\top\mathbf{X}$ gần suy biến: số điều kiện của nó vào khoảng $1{,}8 \times 10^6$ và phần tử lớn nhất của $(\mathbf{X}^\top\mathbf{X})^{-1}$ lên tới khoảng 9505. Hệ quả là nghiệm cực kỳ nhạy với sai số làm tròn ở các bước trước:

x_1 **(dòng 2)**	x_2 **(dòng 4)**	β_0	β_1	β_2	x_3 **(dòng 5)**
5,31553… (đủ chữ số)	4,46889…	9,781	0,349	$-1{,}400$	$-2{,}47$
5,316	4,469	8,852	0,460	$-1{,}316$	$-2{,}53$
5,32	4,47	1,242	1,375	$-0{,}624$	$-3{,}00$
5,306	4,47	32,894	$-2{,}430$	$-3{,}501$	$-1{,}04$
5,3445	4,47	$-41{,}058$	6,445	3,224	$-5{,}59$

Bảng 5.8: Độ nhạy của bước 4 khi các giá trị điền trước đó bị làm tròn

Chỉ cần x_1 lệch $0{,}01$ là hệ số chặn β_0 đã nhảy từ khoảng 9,8 lên 32,9 hay xuống -41. Vì vậy, khi tính tay mỗi người làm tròn một kiểu sẽ cho những hệ số rất khác nhau, dù **không ai tính sai phép tính**. Lặp thêm nhiều vòng cũng không giúp gì: với `max_iter=10`, giá trị điền của x_3 còn trôi tới khoảng $-28{,}5$.

Bài học thực hành:

1. Luôn **kiểm tra tính hợp lý** của giá trị được điền (so với miền quan sát, ý nghĩa nghiệp vụ).
2. Giới hạn miền giá trị bằng `min_value`, `max_value`.
3. Ưu tiên bộ hồi quy **có điều chuẩn** — mặc định của `IterativeImputer` là `BayesianRidge` — vì hệ số ổn định hơn khi có đa cộng tuyến (xem Ridge ở Chương **??**). Tuy vậy, điều chuẩn không thay thế được bước kiểm tra: trên chính ví dụ này, `BayesianRidge` không giới hạn miền vẫn cho $x_3 \approx -2{,}51$.
4. Chỉ dùng Iterative Imputer khi có **đủ dữ liệu** để ước lượng quan hệ giữa các cột; với bốn dòng như ví dụ, Simple Imputer hoặc KNN Imputer an toàn hơn.

Listing 5.3: IterativeImputer tái hiện ví dụ tính tay

```
import numpy as np
from sklearn.experimental import enable_iterative_imputer  # bat tinh nang thu nghiem
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

nan = np.nan
X = np.array([[3, 7, 1], [nan, 4, 8], [6, 3, 7], [5, nan, 4], [1, 9, nan]])

# Mot vong, hoi quy tuyen tinh, thu tu cot x1 -> x2 -> x3 (giong tinh tay)
imp = IterativeImputer(estimator=LinearRegression(), initial_strategy="mean",
                       imputation_order="roman", max_iter=1)
print(imp.fit_transform(X).round(4))
# dong 2: x1 = 5.3155 ; dong 4: x2 = 4.4689 ; dong 5: x3 = -2.4715

# Mac dinh BayesianRidge, 10 vong, gioi han mien gia tri theo du lieu quan sat
lo, hi = np.nanmin(X, axis=0), np.nanmax(X, axis=0)
```

```
imp = IterativeImputer(max_iter=10, min_value=lo, max_value=hi,
    random_state=0)
print(imp.fit_transform(X).round(3))
# dong 2: x1 = 4.833 ; dong 4: x2 = 4.5 ; dong 5: x3 = 1.0 (bi chan
    duoi)
```

Bộ điền	Ý tưởng	Ưu điểm	Nhược điểm
SimpleImputer	Thống kê của chính cột (mean, median, mode)	Nhanh, đơn giản, ổn định	Bỏ qua quan hệ giữa các cột; làm giảm phương sai
KNNImputer	Trung bình của k láng giềng theo khoảng cách nan-Euclid	Tận dụng sự tương đồng giữa các bản ghi	Chậm với dữ liệu lớn; cần chuẩn hóa thang đo
IterativeImputer	Hồi quy lặp từng cột theo các cột còn lại (MICE)	Khai thác quan hệ đa biến	Chậm; có thể ngoại suy phi thực tế; cần kiểm tra kết quả

Bảng 5.9: So sánh ba bộ điền giá trị thiếu

5.4 Phát hiện ngoại lai (Outlier Detection)

Theo định nghĩa kinh điển của Hawkins (1980), **ngoại lai** (outlier) là *"một quan sát lệch khỏi các quan sát khác nhiều đến mức gợi nghi ngờ rằng nó được sinh ra bởi một cơ chế khác"*. Ngoại lai có thể là lỗi nhập liệu (năm xây garage 2207 trong Ames Housing), lỗi đo đạc, hoặc là hiện tượng có thật nhưng hiếm (một giao dịch gian lận). Vì vậy **phát hiện** và **xử lý** là hai quyết định khác nhau: chỉ loại bỏ khi chắc chắn đó là lỗi; trong các trường hợp khác có thể giữ nguyên, biến đổi (lấy logarit), cắt ngưỡng (winsorizing), hoặc dùng mô hình và bộ chuẩn hóa bền vững như `RobustScaler`.

5.4.1 Phương pháp một biến: z-score và quy tắc IQR

Quy tắc z-score. Coi điểm có $|z| = |x - \bar{x}|/s > 3$ là ngoại lai. Điểm yếu của quy tắc này là chính ngoại lai kéo lệch $\bar{x}$ và làm s phình to (**hiệu ứng che lấp** – masking). Với cột $x = (2, 4, 6, 8, 30)$ sẽ dùng lại ở mục 5.7: $\bar{x} = 10$, $s = 10{,}20$ nên $z(30) = 1{,}96 < 3$ — giá trị 30 **không** bị phát hiện. Thực ra có thể chứng minh $|z| \le \sqrt{n-1}$ (Shiffler, 1988), nên với $n = 5$ không quan sát nào có thể vượt ngưỡng 3.

Quy tắc IQR (Tukey, 1977). Dựa trên các tứ phân vị Q_1, Q_3 và khoảng tứ phân vị $\text{IQR} = Q_3 - Q_1$; điểm nằm ngoài khoảng

$$\left[\, Q_1 - 1{,}5\,\text{IQR},\;\; Q_3 + 1{,}5\,\text{IQR} \,\right] \tag{5.2}$$

là ngoại lai — đây cũng chính là "râu" của biểu đồ hộp (boxplot). Với cùng dữ liệu: $Q_1 = 4$, $Q_3 = 8$, $\text{IQR} = 4$, khoảng hợp lệ là $[-2;\ 14]$, nên 30 bị phát hiện. Tứ phân vị bền vững với ngoại lai nên quy tắc IQR không bị che lấp.

Listing 5.4: z-score và quy tắc IQR

```
import numpy as np

x = np.array([2, 4, 6, 8, 30])
z = (x - x.mean()) / x.std()
print(z.round(2))                                          # [-0.78
   -0.59 -0.39 -0.2   1.96]
q1, q3 = np.percentile(x, [25, 75]); iqr = q3 - q1
print(x[(x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr)])  # [30]
```

5.4.2 Phương pháp nhiều biến: Local Outlier Factor (LOF)

Trong không gian nhiều chiều, một điểm có thể "bình thường" trên từng biến riêng lẻ nhưng bất thường khi xét đồng thời. Breunig, Kriegel, Ng và Sander (2000) đề xuất **LOF** (Local Outlier Factor): so sánh **mật độ cục bộ** của một điểm với mật độ của các láng giềng; điểm nằm trong vùng thưa hơn hẳn các láng giềng của nó là ngoại lai. LOF cùng tư tưởng "dựa trên mật độ" với thuật toán phân cụm DBSCAN (Chương 9); hai công trình có chung hai tác giả là Hans-Peter Kriegel và Jörg Sander.

Định nghĩa 5.3 (Các đại lượng của LOF)**.** *Với số láng giềng k và $N_k(p)$ là tập k láng giềng gần nhất của điểm p:*

- ***k-khoảng cách**: k-dist(p) là khoảng cách từ p đến láng giềng thứ k của nó.*
- ***Khoảng cách tiếp cận** (reachability distance):*

$$\textit{reach-dist}_k(p, o) = \max\{\textit{k-dist}(o),\ d(p, o)\}.$$

- ***Mật độ tiếp cận cục bộ** (local reachability density):*

$$\textit{lrd}_k(p) = \left(\frac{1}{k} \sum_{o \in N_k(p)} \textit{reach-dist}_k(p, o) \right)^{-1}.$$

- ***Hệ số ngoại lai cục bộ**:*

$$\textit{LOF}_k(p) = \frac{1}{k} \sum_{o \in N_k(p)} \frac{\textit{lrd}_k(o)}{\textit{lrd}_k(p)}.$$

$\textit{LOF} \approx 1$: mật độ tương đương các láng giềng (điểm bình thường); $\textit{LOF} \gg 1$: điểm thưa hơn hẳn láng giềng (ngoại lai).

Phép lấy max trong khoảng cách tiếp cận có tác dụng **làm trơn**: các điểm nằm sâu trong cùng một cụm dày đặc có khoảng cách tiếp cận bằng nhau, nhờ đó LOF không dao động vì những chênh lệch khoảng cách rất nhỏ.

Ví dụ 5.5 (Tính tay LOF với $k = 3$)**.** *Cho 5 điểm $A(0, 0)$, $B(1, 0)$, $C(0, 1)$, $D(1, 1)$ và $E(5, 4)$ với ma trận khoảng cách Euclid trong Bảng 5.10.*

	A	B	C	D	E
A	0	1	1	1,414	6,403
B	1	0	1,414	1	5,657
C	1	1,414	0	1	5,831
D	1,414	1	1	0	5
E	6,403	5,657	5,831	5	0

Bảng 5.10: Khoảng cách giữa 5 điểm của ví dụ LOF

Bước 1 – k-khoảng cách. Mỗi điểm A, B, C, D có hai láng giềng ở khoảng cách 1 và một láng giềng ở khoảng cách $\sqrt{2}$, nên k-dist $= 1{,}414$. Ba láng giềng của E là D, B, C, nên k-dist$(E) = 5{,}831$.
Bước 2 – khoảng cách tiếp cận. Chẳng hạn reach-dist$(A, B) = \max\{\text{k-dist}(B),\ d(A, B)\} = \max\{1{,}414;\ 1\} = 1{,}414$: dù A chỉ cách B một đơn vị, khoảng cách tiếp cận vẫn là $1{,}414$. Với E, các khoảng cách thật đều lớn hơn k-khoảng cách của láng giềng nên được giữ nguyên.
Bước 3 – mật độ và hệ số LOF được tổng hợp trong Bảng 5.11.

Điểm	**3 láng giềng**	**reach-dist**	**TB reach-dist**	**lrd**	**LOF**	**Nhãn**
A	B, C, D	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
B	A, D, C	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
C	A, D, B	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
D	B, C, A	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
E	D, B, C	5; 5,657; 5,831	5,496	0,1820	**3,886**	-1

Bảng 5.11: Bảng tính LOF ($k = 3$); nhãn theo quy ước của scikit-learn: 1 là điểm bình thường, -1 là ngoại lai

Chẳng hạn $\text{LOF}(E) = \dfrac{(0{,}7071 + 0{,}7071 + 0{,}7071)/3}{0{,}1820} \approx 3{,}886 \gg 1$, nên E là ngoại lai; các điểm còn lại có LOF $= 1$. Với `contamination="auto"`, scikit-learn coi điểm có LOF $> 1{,}5$ là ngoại lai.

[Chỗ chèn hình]
`images/ch-preprocessing-lof.png`

Hình 5.3: Minh họa LOF: bốn điểm A, B, C, D tạo thành vùng dày đặc; điểm E nằm trong vùng thưa với LOF $\approx 3{,}89$.

Listing 5.5: Local Outlier Factor với scikit-learn

```
1 import numpy as np
```

```
from sklearn.neighbors import LocalOutlierFactor

X = np.array([[0, 0], [1, 0], [0, 1], [1, 1], [5, 4]])
lof = LocalOutlierFactor(n_neighbors=3)
print(lof.fit_predict(X))                          # [ 1  1  1  1 -1]
print(-lof.negative_outlier_factor_.round(4))      # [1.     1.     1.
       1.     3.8862]
```

Để phát hiện ngoại lai trên **dữ liệu mới** (sau khi đã huấn luyện), đặt tham số `novelty=True` cho `LocalOutlierFactor`. Một lựa chọn khác mở rộng tốt cho dữ liệu lớn là `IsolationForest` (Liu, Ting & Zhou, 2008): ngoại lai là những điểm bị cô lập sau rất ít lần chia ngẫu nhiên.

5.5 Mã hóa dữ liệu phân loại

5.5.1 Label Encoder

Định nghĩa 5.4 (Label Encoding)**.** *Cho tập K nhãn phân biệt $\mathcal{C} = \{c_0, c_1, \ldots, c_{K-1}\}$. Label Encoding là một song ánh*

$$\phi : \mathcal{C} \to \{0, 1, \ldots, K-1\}, \qquad \phi(c_k) = k, \tag{5.3}$$

*gán cho mỗi nhãn một số nguyên duy nhất. Kết quả là một **từ điển** (dictionary) ánh xạ nhãn $\to$ số.*

Ví dụ 5.6. *Cho dãy đầu vào gồm 7 phần tử: `red`, `green`, `blue`, `red`, `green`, `red`, `orange`. Duyệt dãy từ trái sang phải, mỗi khi gặp một nhãn **mới** ta cấp cho nó số tiếp theo:*

Input	red	green	blue	red	green	red	orange
Output	0	1	2	0	1	0	3

Dictionary	
red	0
green	1
blue	2
orange	3

Bảng 5.12: Label Encoding theo thứ tự xuất hiện

[Chỗ chèn hình]

`images/ch-preprocessing-label-encoder.jpg`

Hình 5.4: Minh họa Label Encoder trên bảng tính: dòng input, dòng output và từ điển ánh xạ.

Để hiểu bản chất thuật toán, ta tự cài đặt Label Encoder chỉ bằng các cấu trúc đã học ở chương Python cơ bản: **list, vòng lặp `for` và dictionary**.

Listing 5.6: Tự cài đặt Label Encoder bằng list, for và dictionary

```
data = ["red", "green", "blue", "red", "green", "red", "orange"]

# Buoc 1: xay dung tu dien nhan -> so (theo thu tu xuat hien)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
print(dictionary)
# {'red': 0, 'green': 1, 'blue': 2, 'orange': 3}

# Buoc 2: ma hoa du lieu
encoded = [dictionary[label] for label in data]
print(encoded)          # [0, 1, 2, 0, 1, 0, 3]

# Buoc 3: giai ma (inverse) bang tu dien dao
inverse = {v: k for k, v in dictionary.items()}
print([inverse[v] for v in encoded])
```

Trong scikit-learn, lớp `LabelEncoder` làm việc tương tự nhưng có một điểm khác quan trọng: các nhãn được **sắp xếp theo thứ tự từ điển (alphabet)** trước khi đánh số, chứ không theo thứ tự xuất hiện.

Listing 5.7: Label Encoder với scikit-learn

```
from sklearn.preprocessing import LabelEncoder

data = ["red", "green", "blue", "red", "green", "red", "orange"]
le = LabelEncoder()
encoded = le.fit_transform(data)

print(le.classes_)    # ['blue' 'green' 'orange' 'red']
print(encoded)        # [3 1 0 3 1 3 2]
print(le.inverse_transform([0, 3]))   # ['blue' 'red']
```

Hạn chế của Label Encoder. Label Encoder tạo ra một **thứ tự giả** giữa các nhãn. Với từ điển ở Bảng 5.12, khoảng cách Euclid giữa các màu là:

$$d(\text{red}, \text{green}) = |0 - 1| = 1, \qquad d(\text{red}, \text{orange}) = |0 - 3| = 3. \tag{5.4}$$

Nghĩa là mô hình sẽ "hiểu" rằng màu đỏ giống màu xanh lá hơn màu cam ba lần — một kết luận hoàn toàn vô nghĩa, chỉ xuất phát từ việc ta đánh số tùy ý. Với các thuật toán dựa trên khoảng cách như **KNN, K-Means** hoặc các mô hình tuyến tính, đây là nguồn sai lệch nghiêm trọng.

> **Lưu ý chuẩn mực:** Trong scikit-learn, `LabelEncoder` được thiết kế để mã hóa **biến mục tiêu** y (nhãn lớp trong bài toán phân lớp). Với các **đặc trưng đầu vào** có thứ bậc, dùng `OrdinalEncoder` (cho phép tự khai báo thứ tự); với đặc trưng định danh, dùng `OneHotEncoder`.

Listing 5.8: Ordinal Encoder với thứ tự do người dùng khai báo

```
from sklearn.preprocessing import OrdinalEncoder

quality = [["Po"], ["TA"], ["Gd"], ["Ex"], ["TA"], ["Fa"]]
oe = OrdinalEncoder(categories=[["Po", "Fa", "TA", "Gd", "Ex"]])
print(oe.fit_transform(quality).ravel())
# [0. 2. 3. 4. 2. 1.]   -> Po < Fa < TA < Gd < Ex
```

5.5.2 One Hot Encoder

Ý tưởng mã hóa "một bit bật" (*one-hot*) xuất phát từ thiết kế mạch số, nơi mỗi trạng thái của máy trạng thái được biểu diễn bằng đúng một flip-flop ở mức 1. Trong thống kê, kỹ thuật tương đương là **biến giả** (dummy variable), được Suits (1957) phổ biến cho hồi quy với biến phân loại.

Định nghĩa 5.5 (One Hot Encoding)**.** *Với tập K nhãn $\mathcal{C} = \{c_0, \ldots, c_{K-1}\}$, One Hot Encoding ánh xạ nhãn c_k thành vector đơn vị thứ k trong $\mathbb{R}^K$:*

$$\psi(c_k) = \mathbf{e}_k = (0, \ldots, 0, \underbrace{1}_{\text{vị trí } k}, 0, \ldots, 0) \in \{0,1\}^K. \tag{5.5}$$

*Một cột phân loại được thay bằng K cột nhị phân; mỗi hàng có **đúng một** số 1.*

Ưu điểm then chốt: mọi cặp nhãn khác nhau đều **cách đều nhau**:

$$\|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}, \qquad \forall\, i \neq j, \tag{5.6}$$

nên không còn thứ tự giả. Đây là lý do One Hot Encoder là lựa chọn mặc định cho đặc trưng định danh khi dùng **K-Means, KNN**, SVM hay hồi quy tuyến tính.

Ví dụ 5.7. *Cho dãy 9 phần tử (chỉ số 0–8): `red, blue, red, red, blue, white, red, white, orange`. Từ điển theo thứ tự xuất hiện là {red: 0, blue: 1, white: 2, orange: 3}, do đó mỗi phần tử trở thành một vector 4 chiều:*

Chỉ số	Input	red (0)	blue (1)	white (2)	orange (3)
0	red	1	0	0	0
1	blue	0	1	0	0
2	red	1	0	0	0
3	red	1	0	0	0
4	blue	0	1	0	0
5	white	0	0	1	0
6	red	1	0	0	0
7	white	0	0	1	0
8	orange	0	0	0	1

Bảng 5.13: Ma trận One Hot (9×4) cho dãy màu sắc

[Chỗ chèn hình]

images/ch-preprocessing-one-hot-encoder.jpg

Hình 5.5: Minh họa One Hot Encoder trên bảng tính: từ điển (Key – Value) và ma trận đầu ra; ô được tô vàng là vị trí số 1 tương ứng với nhãn của từng hàng.

Listing 5.9: Tự cài đặt One Hot Encoder bằng list, for và dictionary

```
data = ["red", "blue", "red", "red", "blue",
        "white", "red", "white", "orange"]

# Buoc 1: tu dien (Key -> Value)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
K = len(dictionary)          # so cot = so nhan phan biet = 4

# Buoc 2: moi phan tu -> vector co dung mot so 1
one_hot = []
for label in data:
    row = [0] * K
    row[dictionary[label]] = 1
    one_hot.append(row)

for i, row in enumerate(one_hot):
    print(i, data[i], row)
# 0 red    [1, 0, 0, 0]
# 1 blue   [0, 1, 0, 0]
# ...
# 8 orange [0, 0, 0, 1]
```

Listing 5.10: One Hot Encoder với scikit-learn và pandas

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

data = np.array(["red", "blue", "red", "red", "blue",
                 "white", "red", "white", "orange"]).reshape(-1, 1)

ohe = OneHotEncoder(handle_unknown="ignore")
X = ohe.fit_transform(data)       # ket qua la ma tran thua (sparse)
```

```
print(ohe.get_feature_names_out())
# ['x0_blue' 'x0_orange' 'x0_red' 'x0_white']   (sap xep alphabet)
print(X.toarray()[:3])
# [[0. 0. 1. 0.]
#  [1. 0. 0. 0.]
#  [0. 0. 1. 0.]]

# Nhan moi chua tung gap -> vector toan 0 thay vi bao loi
print(ohe.transform([["green"]]).toarray())   # [[0. 0. 0. 0.]]

# Cach nhanh voi pandas
print(pd.get_dummies(pd.Series(data.ravel()), dtype=int).head(3))
```

Một số vấn đề cần lưu ý.

- **Bẫy biến giả** (dummy variable trap): K cột one-hot luôn có tổng bằng 1, tức phụ thuộc tuyến tính hoàn toàn với cột hệ số chặn. Với hồi quy tuyến tính không chính quy hóa, nên bỏ một cột bằng `OneHotEncoder(drop="first")` hoặc `pd.get_dummies(..., drop_first=True)`.
- **Bùng nổ số chiều**: biến có hàng nghìn giá trị (mã bưu chính, mã sản phẩm) sinh ra hàng nghìn cột. Khi đó cần gộp các nhãn hiếm, dùng ma trận thưa hoặc các kỹ thuật như target encoding, hashing.
- **Nhãn chưa từng gặp** ở tập kiểm tra: dùng `handle_unknown="ignore"` để tránh lỗi khi triển khai.

Tiêu chí	Label / Ordinal Encoder	One Hot Encoder
Số cột sinh ra	1	K (số nhãn)
Tạo thứ tự giả	Có	Không
Phù hợp với	Biến mục tiêu; đặc trưng thứ bậc; mô hình cây	Đặc trưng định danh; KNN, K-Means, SVM, mô hình tuyến tính
Nhược điểm	Sai lệch khoảng cách	Tăng số chiều, ma trận thưa

Bảng 5.14: So sánh Label Encoder và One Hot Encoder

5.6 Vector hóa văn bản

5.6.1 Mô hình túi từ và Count Vectorizer

Ý tưởng biểu diễn một văn bản bằng **tần suất các từ** có từ rất sớm: nhà ngôn ngữ học Zellig Harris (1954) trong *Distributional Structure* đã nhắc đến "túi từ" (bag of words), và Hans Peter Luhn (1957) tại IBM đề xuất dùng tần suất từ để đánh chỉ mục tài liệu tự động. Năm 1975, Salton, Wong và Yang hệ thống hóa thành **mô hình không gian vector** (Vector Space Model): mỗi văn bản là một vector trong không gian có số chiều bằng kích thước từ vựng.

Định nghĩa 5.6 (Count Vectorizer)**.** *Cho kho văn bản (corpus) $D = \{d_1, \ldots, d_N\}$ và từ vựng $V = \{t_1, \ldots, t_M\}$ gồm mọi từ xuất hiện trong D. Count Vectorizer biểu diễn văn bản d bằng vector*

$$\mathbf{x}_d = \big(\text{tf}(t_1, d), \text{tf}(t_2, d), \ldots, \text{tf}(t_M, d)\big), \tag{5.7}$$

trong đó $\text{tf}(t, d)$ *(term frequency) là số lần từ t xuất hiện trong d. Cả kho văn bản trở thành ma trận văn bản – từ (document-term matrix) kích thước $N \times M$.*

Ví dụ 5.8. *Xét kho văn bản gồm 3 câu:*

- d_1*: "the cats are in the house"*
- d_2*: "the dogs are in the house and outside"*
- d_3*: "the cats and dogs are friends"*

Từ vựng (sắp xếp alphabet) gồm $M = 9$ từ: and, are, cats, dogs, friends, house, in, outside, the.

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	1	1	0	0	1	1	0	2
d_2	1	1	0	1	0	1	1	1	2
d_3	1	1	1	1	1	0	0	0	1
$\text{df}(t)$	2	3	2	2	1	2	2	1	3

Bảng 5.15: Ma trận đếm từ (Count Vectorizer) và tần suất văn bản $\text{df}(t)$

Listing 5.11: Count Vectorizer với scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]
cv = CountVectorizer()
X = cv.fit_transform(corpus)

print(cv.vocabulary_)
# {'the': 8, 'cats': 2, 'are': 1, 'in': 6, 'house': 5,
#  'dogs': 3, 'and': 0, 'outside': 7, 'friends': 4}
print(X.toarray())
# [[0 1 1 0 0 1 1 0 2]
#  [1 1 0 1 0 1 1 1 2]
#  [1 1 1 1 1 0 0 0 1]]
```

Mặc định `CountVectorizer` chuyển chữ về chữ thường và tách từ bằng biểu thức chính quy `(?u)\b\w\w+\b`, tức **bỏ qua các từ chỉ có 1 ký tự**. Các tham số hay dùng: `stop_words` (loại từ dừng), `ngram_range=(1,2)` (lấy thêm cặp từ), `min_df`/`max_df` (loại từ quá hiếm/quá phổ biến).

Chuẩn hóa L2. Văn bản dài có số đếm lớn hơn chỉ vì dài hơn. Để so sánh công bằng, mỗi vector được chia cho chuẩn Euclid của nó:

$$\hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}, \qquad \|\mathbf{x}_d\|_2 = \sqrt{\textstyle\sum_j x_{dj}^2}. \tag{5.8}$$

Với d_1: $\|\mathbf{x}_{d_1}\|_2^2 = 1+1+1+1+2^2 = 8$, nên các số 1 trở thành $1/\sqrt{8} \approx 0{,}3536$ và số 2 trở thành $2/\sqrt{8} \approx 0{,}7071$. Tương tự $\|\mathbf{x}_{d_2}\|_2 = \sqrt{10}$ và $\|\mathbf{x}_{d_3}\|_2 = \sqrt{6}$:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3536	0.3536	0	0	0.3536	0.3536	0	0.7071
d_2	0.3162	0.3162	0	0.3162	0	0.3162	0.3162	0.3162	0.6325
d_3	0.4082	0.4082	0.4082	0.4082	0.4082	0	0	0	0.4082

Bảng 5.16: Ma trận đếm sau khi chuẩn hóa L2 theo từng hàng

5.6.2 TF-IDF

Count Vectorizer có một nhược điểm: những từ xuất hiện ở **mọi** văn bản (như "the", "are") có số đếm cao nhưng gần như không giúp phân biệt văn bản. Năm 1972, Karen Spärck Jones đề xuất trong bài báo *A statistical interpretation of term specificity and its application in retrieval* (*Journal of Documentation*) rằng trọng số của một từ nên tỉ lệ nghịch với số văn bản chứa nó — đó là **tần suất văn bản nghịch đảo** (Inverse Document Frequency – IDF). Salton và Buckley (1988) sau đó tổng kết các biến thể trọng số TF-IDF, trở thành chuẩn mực của ngành truy hồi thông tin suốt nhiều thập kỷ.

Định nghĩa 5.7 (TF-IDF)**.** *Với N là số văn bản và $\mathrm{df}(t)$ là số văn bản chứa từ t, trọng số TF-IDF của từ t trong văn bản d là*

$$\textit{tf-idf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t). \tag{5.9}$$

Dạng kinh điển của IDF là $\mathrm{idf}(t) = \ln \dfrac{N}{\mathrm{df}(t)}$*. Thư viện scikit-learn dùng hai biến thể có cộng thêm 1 để từ xuất hiện ở mọi văn bản không bị triệt tiêu hoàn toàn:*

$$\textit{Không làm trơn (}\texttt{smooth_idf=False}\textit{)}: \quad \mathrm{idf}(t) = 1 + \ln \frac{N}{\mathrm{df}(t)}, \tag{5.10}$$

$$\textit{Làm trơn (}\texttt{smooth_idf=True}\textit{, mặc định)}: \quad \mathrm{idf}(t) = 1 + \ln \frac{N+1}{\mathrm{df}(t)+1}. \tag{5.11}$$

Công thức làm trơn tương đương việc thêm một "văn bản ảo" chứa mọi từ đúng một lần, giúp tránh phép chia cho 0.

Ví dụ 5.9 (Tính tay IDF)**.** *Với $N = 3$ và* df *ở Bảng 5.15, chỉ có ba giá trị* $\mathrm{df} \in \{1,2,3\}$*:*

$\mathrm{df}(t)$	**Các từ**	**IDF không làm trơn (5.10)**	**IDF làm trơn (5.11)**
1	friends, outside	$1+\ln 3 = 2{,}0986$	$1+\ln 2 = 1{,}6931$
2	and, cats, dogs, house, in	$1+\ln 1{,}5 = 1{,}4055$	$1+\ln \frac{4}{3} = 1{,}2877$
3	are, the	$1+\ln 1 = 1$	$1+\ln 1 = 1$

Bảng 5.17: Giá trị IDF cho kho 3 văn bản

Nhân từng cột của ma trận đếm (Bảng 5.15) với IDF tương ứng, ta được ma trận TF-IDF (chưa chuẩn hóa). Chẳng hạn với từ "cats" trong d_1: tf $= 1$, df $= 2$ nên tf-idf $= 1 \times 1{,}2877 = 1{,}2877$ (làm trơn); với "the" trong d_1: $2 \times 1 = 2$.

	and	are	cats	dogs	friends	house	in	outside	the
IDF làm trơn (`smooth_idf=True`)									
d_1	0	1	1.2877	0	0	1.2877	1.2877	0	2
d_2	1.2877	1	0	1.2877	0	1.2877	1.2877	1.6931	2
d_3	1.2877	1	1.2877	1.2877	1.6931	0	0	0	1
IDF không làm trơn (`smooth_idf=False`)									
d_1	0	1	1.4055	0	0	1.4055	1.4055	0	2
d_2	1.4055	1	0	1.4055	0	1.4055	1.4055	2.0986	2
d_3	1.4055	1	1.4055	1.4055	2.0986	0	0	0	1

Bảng 5.18: Ma trận TF-IDF chưa chuẩn hóa (`norm=None`)

[Chỗ chèn hình]

`images/ch-preprocessing-tfidf-excel.png`

Hình 5.6: Tính TF-IDF trên bảng tính: ma trận đếm, df(t), idf(t, D) làm trơn, ma trận TF-IDF và ma trận chuẩn hóa L2.

[Chỗ chèn hình]

`images/ch-preprocessing-tfidf-nosmooth.png`

Hình 5.7: Ma trận TF-IDF với IDF không làm trơn (giá trị 1,40547 và 2,09861) — khớp với kết quả `smooth_idf=False` của scikit-learn.

Chuẩn hóa L2 và độ tương đồng cosine. Mặc định `TfidfVectorizer` còn chuẩn hóa L2 từng hàng. Áp dụng cho ma trận làm trơn ở Bảng 5.18:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3166	0.4077	0	0	0.4077	0.4077	0	0.6333
d_2	0.3382	0.2626	0	0.3382	0	0.3382	0.3382	0.4447	0.5252
d_3	0.4105	0.3188	0.4105	0.4105	0.5397	0	0	0	0.3188

Bảng 5.19: Ma trận TF-IDF mặc định của scikit-learn (`smooth_idf=True`, `norm="l2"`)

Vì mỗi hàng có độ dài 1, tích vô hướng giữa hai hàng chính là **độ tương đồng cosine**:

$$\cos(d_i, d_j) = \hat{\mathbf{x}}_{d_i} \cdot \hat{\mathbf{x}}_{d_j}. \tag{5.12}$$

Ta có $\cos(d_1, d_2) \approx 0{,}6915$, $\cos(d_1, d_3) \approx 0{,}4702$, $\cos(d_2, d_3) \approx 0{,}5288$: hai câu cùng nói về "in the house" (d_1, d_2) giống nhau nhất — phù hợp với trực giác.

Listing 5.12: TF-IDF Vectorizer với các cấu hình khác nhau

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]

# 1) Chi dem tu (tuong duong CountVectorizer)
v = TfidfVectorizer(use_idf=False, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray())

# 2) TF-IDF khong lam tron, chua chuan hoa -> 1.4055, 2.0986
v = TfidfVectorizer(use_idf=True, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray().round(4))

# 3) TF-IDF lam tron, chua chuan hoa -> 1.2877, 1.6931
v = TfidfVectorizer(use_idf=True, smooth_idf=True, norm=None)
print(v.fit_transform(corpus).toarray().round(4))
print(v.idf_.round(4))
# [1.2877 1.     1.2877 1.2877 1.6931 1.2877 1.2877 1.6931 1.    ]

# 4) Cau hinh mac dinh: lam tron + chuan hoa L2
v = TfidfVectorizer()
X = v.fit_transform(corpus)
print(X.toarray().round(4))
print(cosine_similarity(X).round(4))
# [[1.     0.6915 0.4702]
#  [0.6915 1.     0.5288]
#  [0.4702 0.5288 1.    ]]
```

Cấu hình	`use_idf`	`smooth_idf`	`norm`
Đếm từ thuần túy	False	–	None
Đếm từ chuẩn hóa L2	False	–	`"l2"`
TF-IDF thô	True	False	None
TF-IDF mặc định	True	True	`"l2"`

Bảng 5.20: Các cấu hình của `TfidfVectorizer` dùng trong chương

5.7 Chuẩn hóa thang đo đặc trưng (Feature Scaling)

5.7.1 Vì sao cần chuẩn hóa thang đo?

Xét hai đặc trưng của một căn nhà: diện tích lô đất (hàng nghìn đến hàng trăm nghìn feet vuông) và số phòng tắm (0 đến 4). Trong khoảng cách Euclid

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}, \tag{5.13}$$

chênh lệch diện tích sẽ áp đảo hoàn toàn chênh lệch số phòng tắm, dù trên thực tế số phòng tắm có thể quan trọng không kém. Chuẩn hóa thang đo đưa mọi đặc trưng về cùng một "đơn vị" để:

- Các thuật toán dựa trên **khoảng cách** (KNN, K-Means, SVM với nhân RBF) đối xử công bằng giữa các đặc trưng.
- **Gradient Descent** hội tụ nhanh hơn, vì mặt hàm mất mát bớt bị kéo dài theo một chiều.
- **PCA** không bị chi phối bởi đặc trưng có phương sai lớn chỉ do đơn vị đo.
- Các mô hình có **chính quy hóa** (Ridge, Lasso) phạt các hệ số một cách công bằng.

Ngược lại, các mô hình dựa trên **cây quyết định** (Decision Tree, Random Forest, XGBoost) chỉ so sánh ngưỡng trên từng đặc trưng nên **không** cần chuẩn hóa.

Ký hiệu một cột đặc trưng là $x = (x_1, \ldots, x_n)$. Bốn bộ chuẩn hóa phổ biến trong scikit-learn được định nghĩa như sau.

5.7.2 Min Max Scaler

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \in [0, 1]. \tag{5.14}$$

Tổng quát hơn, với `feature_range=(a, b)`: $x''_i = a + x'_i(b - a)$. Phép biến đổi giữ nguyên hình dạng phân phối, nhưng **rất nhạy với ngoại lai** vì min và max chính là các giá trị cực trị.

5.7.3 Max Abs Scaler

$$x'_i = \frac{x_i}{\max_j |x_j|} \in [-1, 1]. \tag{5.15}$$

Max Abs Scaler không dịch chuyển dữ liệu (không trừ đi giá trị nào), nên **giữ nguyên các số 0**. Đây là lựa chọn phù hợp cho **ma trận thưa** như ma trận TF-IDF ở mục trước: trừ đi trung bình sẽ biến mọi số 0 thành khác 0 và phá vỡ tính thưa.

5.7.4 Standard Scaler (chuẩn hóa z-score)

$$z_i = \frac{x_i - \mu}{\sigma}, \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}. \tag{5.16}$$

Sau biến đổi, cột có trung bình 0 và độ lệch chuẩn 1. Lưu ý scikit-learn dùng độ lệch chuẩn **tổng thể** (chia cho n, tức `ddof=0`), khác với `pandas.Series.std()` mặc định chia cho $n-1$. Standard Scaler là lựa chọn mặc định cho hồi quy tuyến tính, hồi quy logistic, SVM, PCA và mạng nơ-ron.

5.7.5 Robust Scaler

$$x_i' = \frac{x_i - \text{median}(x)}{\text{IQR}(x)}, \qquad \text{IQR} = Q_3 - Q_1, \tag{5.17}$$

trong đó Q_1, Q_3 là các phân vị 25% và 75%. Trung vị và khoảng tứ phân vị **gần như không bị ảnh hưởng bởi ngoại lai**, nên Robust Scaler phù hợp với dữ liệu lệch hoặc có giá trị bất thường.

5.7.6 Ví dụ so sánh bốn bộ chuẩn hóa

Ví dụ 5.10. *Cho cột $x = (2, 4, 6, 8, 30)$, trong đó 30 là một ngoại lai. Các đại lượng cần thiết:* $\min = 2$, $\max = 30$, $\max|x| = 30$, $\mu = 10$, $\sigma = \sqrt{104} \approx 10{,}198$, $\text{median} = 6$, $Q_1 = 4$, $Q_3 = 8$, $\text{IQR} = 4$.

x	MinMax	MaxAbs	Standard	Robust
2	0.0000	0.0667	-0.7845	-1.0
4	0.0714	0.1333	-0.5883	-0.5
6	0.1429	0.2000	-0.3922	0.0
8	0.2143	0.2667	-0.1961	0.5
30	1.0000	1.0000	1.9612	6.0

Bảng 5.21: Kết quả của bốn bộ chuẩn hóa trên cột có ngoại lai

Quan sát Bảng 5.21: với Min Max Scaler, bốn giá trị "bình thường" bị ép vào đoạn $[0;\ 0{,}21]$ — gần như mất khả năng phân biệt — chỉ vì một ngoại lai. Robust Scaler giữ các giá trị bình thường trải đều trong $[-1;\ 0{,}5]$ và đẩy ngoại lai ra xa (6,0), giúp ngoại lai **dễ nhận diện** thay vì làm méo phần còn lại.

Listing 5.13: So sánh bốn bộ chuẩn hóa trong scikit-learn

```
import numpy as np
from sklearn.preprocessing import (MinMaxScaler, MaxAbsScaler,
                                   StandardScaler, RobustScaler)

x = np.array([2, 4, 6, 8, 30], dtype=float).reshape(-1, 1)

for scaler in [MinMaxScaler(), MaxAbsScaler(),
               StandardScaler(), RobustScaler()]:
    print(type(scaler).__name__, scaler.fit_transform(x).ravel().
        round(4))
```

```
# MinMaxScaler   [0.      0.0714  0.1429  0.2143  1.    ]
# MaxAbsScaler   [0.0667  0.1333  0.2     0.2667  1.    ]
# StandardScaler [-0.7845 -0.5883 -0.3922 -0.1961  1.9612]
# RobustScaler   [-1.   -0.5   0.    0.5   6. ]
```

[Chỗ chèn hình]

`images/ch-preprocessing-scalers.png`

Hình 5.8: Phân phối của một đặc trưng trước và sau khi áp dụng bốn bộ chuẩn hóa (gợi ý: vẽ histogram hoặc boxplot bằng matplotlib).

Bộ chuẩn hóa	**Miền giá trị**	**Bền với ngoại lai**	**Khi nào dùng**
MinMaxScaler	$[0, 1]$	Không	Cần miền cố định (ảnh, mạng nơ-ron), dữ liệu ít ngoại lai
MaxAbsScaler	$[-1, 1]$	Không	Ma trận thưa (TF-IDF), dữ liệu đã tâm ở 0
StandardScaler	Không giới hạn	Trung bình	Mặc định cho mô hình tuyến tính, SVM, PCA
RobustScaler	Không giới hạn	Có	Dữ liệu lệch, nhiều ngoại lai

Bảng 5.22: Tóm tắt lựa chọn bộ chuẩn hóa

5.8 Dữ liệu mất cân bằng và SMOTE

Trong nhiều bài toán phân lớp, lớp quan trọng nhất lại là lớp **hiếm**: giao dịch gian lận (thường dưới 1%), bệnh hiếm, thiết bị sắp hỏng. Mô hình huấn luyện trên dữ liệu như vậy có xu hướng "bỏ qua" lớp thiểu số mà vẫn đạt accuracy cao — *nghịch lý accuracy* sẽ được phân tích ở Chương 11. Có ba nhóm giải pháp:

- **Ở mức thuật toán**: gán trọng số lớn hơn cho lỗi trên lớp thiểu số, ví dụ đặt tham số `class_weight` bằng `"balanced"`.
- **Giảm mẫu lớp đa số** (undersampling): bỏ bớt mẫu của lớp đông — đơn giản nhưng mất thông tin.
- **Tăng mẫu lớp thiểu số** (oversampling): nhân bản ngẫu nhiên các mẫu hiếm dễ gây quá khớp, vì mô hình gặp đi gặp lại đúng những điểm đó.

SMOTE (Synthetic Minority Over-sampling Technique), do Chawla, Bowyer, Hall và Kegelmeyer công bố năm 2002, khắc phục nhược điểm của việc nhân bản: thay vì sao chép, SMOTE **tổng hợp mẫu mới** nằm trên đoạn thẳng nối một mẫu thiểu số với một láng giềng cùng lớp của nó.

Định nghĩa 5.8 (Sinh mẫu SMOTE)**.** *Với mỗi mẫu thiểu số $\mathbf{x}_i$: tìm k láng giềng gần nhất **trong lớp thiểu số**; chọn ngẫu nhiên một láng giềng $\mathbf{x}_{nn}$ và một số $\lambda \sim U(0,1)$; mẫu mới là*

$$\mathbf{x}_{\text{mới}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i). \tag{5.18}$$

Ví dụ 5.11 (Tạo một mẫu SMOTE)**.** *Lớp thiểu số gồm $P_1(2,3)$, $P_2(4,4)$, $P_3(3,6)$; chọn $k=2$. Hai láng giềng của P_1 là P_2 ($d=\sqrt{5}\approx 2{,}236$) và P_3 ($d=\sqrt{10}\approx 3{,}162$). Giả sử lần rút ngẫu nhiên chọn được P_2 và $\lambda = 0{,}4$:*

$$\mathbf{x}_{\text{mới}} = (2,3) + 0{,}4 \times \big[(4,4)-(2,3)\big] = (2,3) + (0{,}8;\ 0{,}4) = (2{,}8;\ 3{,}4).$$

Điểm mới nằm trên đoạn P_1P_2, bên trong vùng của lớp thiểu số, và không trùng với mẫu nào đã có.

[Chỗ chèn hình]

images/ch-preprocessing-smote.png

Hình 5.9: SMOTE sinh mẫu tổng hợp trên đoạn thẳng nối các mẫu thiểu số với láng giềng cùng lớp.

SMOTE được cài đặt trong thư viện `imbalanced-learn` (Lemaître, Nogueira & Aridas, 2017), cài bằng lệnh `pip install imbalanced-learn`. Các biến thể đáng chú ý: **Borderline-SMOTE** (Han và cộng sự, 2005) chỉ sinh mẫu gần biên quyết định; **ADASYN** (He và cộng sự, 2008) sinh nhiều mẫu hơn ở vùng khó học; **SMOTE-NC** xử lý dữ liệu có cột định danh.

Listing 5.14: SMOTE với imbalanced-learn

```
from collections import Counter
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE

X, y = make_classification(n_samples=1000, n_features=4,
    n_informative=2,
                           weights=[0.9, 0.1], random_state=42)
print(Counter(y))                       # lop 0: 895, lop 1: 105
X_res, y_res = SMOTE(k_neighbors=5, random_state=42).fit_resample(X,
    y)
print(Counter(y_res))                   # lop 0: 895, lop 1: 895
```

Nguyên tắc vàng: chỉ áp dụng SMOTE trên **tập huấn luyện**. Nếu sinh mẫu trước khi chia dữ liệu, các mẫu tổng hợp được nội suy từ chính những điểm sẽ rơi vào tập kiểm tra, khiến kết quả đánh giá lạc quan giả tạo (rò rỉ dữ liệu). Thư viện `imbalanced-learn` cung cấp `Pipeline` riêng, trong đó bước lấy mẫu **chỉ chạy khi `fit`**, tức chỉ trên phần huấn luyện của mỗi fold kiểm định chéo:

Listing 5.15: So sánh các cách xử lý mất cân bằng trong Pipeline của imbalanced-learn

```
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate

models = {
    "khong xu ly": ImbPipeline([("scale", StandardScaler()),
                                ("clf", LogisticRegression())]),
    "class_weight": ImbPipeline([("scale", StandardScaler()),
                                 ("clf", LogisticRegression(
                                    class_weight="balanced"))]),
    "smote": ImbPipeline([("scale", StandardScaler()),
                          ("smote", SMOTE(random_state=42)),   # chi
                             chay khi fit
                          ("clf", LogisticRegression())]),
}
for name, m in models.items():
    r = cross_validate(m, X, y, cv=5, scoring=["recall", "precision",
        "f1"])
    print(name, {k[5:]: round(v.mean(), 3) for k, v in r.items() if k
       .startswith("test")})
```

Cách xử lý	Recall	Precision	F1
Không xử lý	0,486	0,807	0,589
`class_weight="balanced"`	0,838	0,437	0,572
SMOTE trong pipeline	0,838	0,441	0,575

Bảng 5.23: Hồi quy logistic trên dữ liệu tỉ lệ 90/10, kiểm định chéo 5-fold; lớp dương là lớp thiểu số

Kết quả cho một bài học quan trọng: SMOTE giúp mô hình tìm ra nhiều mẫu thiểu số hơn hẳn (recall tăng từ 0,486 lên 0,838), nhưng phải trả giá bằng nhiều báo động nhầm hơn (precision giảm từ 0,807 xuống 0,441); F1 gần như không đổi, và hiệu quả tương đương cách đơn giản `class_weight="balanced"`. SMOTE vì vậy **không phải phép màu**: nó dịch chuyển điểm cân bằng giữa precision và recall, và nên được chọn khi **bỏ sót lớp thiểu số đắt hơn báo động nhầm** (sàng lọc bệnh, phát hiện gian lận). Việc so sánh phải dựa trên độ đo phù hợp với bài toán (Chương 11), không phải accuracy.

5.9 Kết hợp các bước: Pipeline và ColumnTransformer

Dữ liệu thực tế thường có **nhiều kiểu cột cùng lúc**. scikit-learn cung cấp:

- `ColumnTransformer`: áp dụng các bộ biến đổi khác nhau cho các nhóm cột khác nhau rồi ghép kết quả.
- `Pipeline`: xâu chuỗi các bước tiền xử lý và mô hình thành một đối tượng duy nhất. Khi gọi `pipeline.fit(X_train, y_train)`, mọi bước `fit` chỉ nhìn thấy tập huấn luyện — đây là cách tự động **tránh rò rỉ dữ liệu**.

[Chỗ chèn hình]
`images/ch-preprocessing-pipeline.png`

Hình 5.10: Sơ đồ ColumnTransformer: cột số đi qua SimpleImputer + Scaler, cột định danh qua One Hot Encoder, cột thứ bậc qua Ordinal Encoder, sau đó ghép lại thành ma trận đặc trưng.

5.10 Nghiên cứu tình huống: bộ dữ liệu Ames Housing

5.10.1 Giới thiệu bộ dữ liệu

Bộ dữ liệu **Ames Housing** do Dean De Cock (Đại học Truman State) công bố năm 2011 trên *Journal of Statistics Education*, như một lựa chọn thay thế hiện đại cho bộ dữ liệu giá nhà Boston kinh điển. Dữ liệu ghi nhận các giao dịch bán nhà ở thành phố Ames, bang Iowa (Hoa Kỳ) trong giai đoạn 2006–2010, lấy từ Văn phòng Định giá Tài sản của thành phố.

File `data/AmesHousing.txt` (phân tách bằng ký tự tab) gồm **2930 dòng** và **82 cột**: hai cột định danh bản ghi (`Order`, `PID`), biến mục tiêu `SalePrice` (giá bán, USD) và 79 biến mô tả căn nhà. Đây là bộ dữ liệu lý tưởng cho chương này vì nó chứa **đủ mọi kiểu dữ liệu** đã học:

Kiểu	**Ví dụ cột**	**Giá trị mẫu**
Định danh	`MS Zoning`, `Neighborhood`, `Street`, `Bldg Type`	RL, RM, FV; NAmes, Gilbert; Pave, Grvl
Định danh mã hóa bằng số	`MS SubClass`	020, 060, 120 (mã loại nhà, **không phải** đại lượng)
Thứ bậc	`Exter Qual`, `Kitchen Qual`, `Bsmt Qual`	Po $<$ Fa $<$ TA $<$ Gd $<$ Ex
Số rời rạc	`Full Bath`, `Garage Cars`, `Yr Sold`	0–4; 0–5; 2006–2010
Số liên tục	`Lot Area`, `Gr Liv Area`, `SalePrice`	`Lot Area` tối đa 215245 ft^2

Bảng 5.24: Các kiểu dữ liệu trong Ames Housing

5.10.2 Những "cái bẫy" khi đọc dữ liệu

Quan sát kỹ file dữ liệu cho thấy nhiều vấn đề rất điển hình mà người làm ML phải xử lý:

1. **"NA" không phải lúc nào cũng là thiếu dữ liệu.** Ở các cột `Alley`, `Fence`, `Pool QC`, `Fireplace Qu`, `Garage Type`, `Bsmt Qual`…, chuỗi `NA` mang nghĩa *"không có"* (không có hẻm, không có hàng rào, không có garage). Nếu đọc bằng `pd.read_csv` mặc định, pandas sẽ biến tất cả `NA` thành `NaN` và ta **đánh mất thông tin**. Trong khi đó, ô **rỗng** (ví dụ `Lot Frontage` ở nhiều dòng, `Mas Vnr Type` ở một số dòng) mới thực sự là dữ liệu thiếu.

2. **Khoảng trắng thừa:** cột `Sale Type` có giá trị `"WD "` (có dấu cách cuối). Không làm sạch thì `"WD"` và `"WD "` bị coi là hai nhãn khác nhau khi One Hot.

3. **Số nhưng là phân loại:** `MS SubClass` (020, 060…) là mã loại nhà; dùng trực tiếp như số sẽ gây thứ tự giả giống hệt vấn đề của Label Encoder.

4. **Lỗi nhập liệu:** có bản ghi với `Garage Yr Blt` = 2207 — một năm không thể xảy ra (nhiều khả năng là 2007).

5. **Ngoại lai:** một vài căn có `Gr Liv Area` trên 4000 ft^2 nhưng giá bán thấp bất thường (giao dịch bán từng phần). Chính De Cock khuyến nghị loại các căn nhà trên 4000 ft^2 khi dùng dữ liệu cho giảng dạy. Cột `Lot Area` cũng lệch phải rất mạnh (tối đa 215245 ft^2).

5.10.3 Đọc và khảo sát dữ liệu

Listing 5.16: Đọc Ames Housing và giữ nguyên ý nghĩa của "NA"

```
import pandas as pd

# keep_default_na=False: KHONG bien "NA" thanh NaN
# na_values=[""]       : chi o rong moi la du lieu thieu
df = pd.read_csv("data/AmesHousing.txt", sep="\t",
                 keep_default_na=False, na_values=[""])
print(df.shape)                          # (2930, 82)

# Lam sach khoang trang thua o cac cot chu
obj_cols = df.select_dtypes("object").columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip())

# MS SubClass la ma phan loai -> doi sang kieu chuoi
df["MS SubClass"] = df["MS SubClass"].astype(str)

# Thong ke du lieu thieu that su
missing = df.isna().sum()
print(missing[missing > 0].sort_values(ascending=False))

# Sua loi nhap lieu va loai ngoai lai theo khuyen nghi cua De Cock
df.loc[df["Garage Yr Blt"] > 2010, "Garage Yr Blt"] = None
df = df[df["Gr Liv Area"] <= 4000]
print(df.shape)

```

```
print(df["Exter Qual"].value_counts())
print(df[["Lot Area", "Gr Liv Area", "SalePrice"]].describe())
```

[Chỗ chèn hình]

images/ch-preprocessing-ames-outliers.png

Hình 5.11: Biểu đồ phân tán `Gr Liv Area` – `SalePrice` của Ames Housing; các điểm bên phải ngưỡng 4000 ft^2 là ngoại lai.

5.10.4 Xây dựng quy trình tiền xử lý hoàn chỉnh

Ta chọn một tập con đặc trưng đại diện cho từng kiểu dữ liệu và áp dụng đúng kỹ thuật đã học cho từng nhóm:

Listing 5.17: Pipeline tiền xử lý cho Ames Housing

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (OneHotEncoder, OrdinalEncoder,
                                   StandardScaler, RobustScaler)

nominal = ["MS Zoning", "Neighborhood", "Bldg Type", "MS SubClass",
           "Alley", "Garage Type"]
quality = ["Exter Qual", "Kitchen Qual", "Bsmt Qual", "Fireplace Qu"]
numeric = ["Gr Liv Area", "Total Bsmt SF", "Garage Cars",
           "Full Bath", "Year Built", "Overall Qual"]
skewed  = ["Lot Area", "Lot Frontage"]

# "NA" o cac cot chat luong = khong co -> muc thap nhat
levels = ["NA", "Po", "Fa", "TA", "Gd", "Ex"]

X = df[nominal + quality + numeric + skewed]
y = np.log(df["SalePrice"])      # log giup giam do lech cua gia

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

preprocess = ColumnTransformer([
    ("nom", Pipeline([
```

```
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), nominal),
    ("ord", Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="NA"
           )),
        ("ordinal", OrdinalEncoder(categories=[levels] * len(quality)
           )),
    ]), quality),
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric),
    ("skew", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", RobustScaler()),
    ]), skewed),
])

# CHI fit tren tap huan luyen, transform cho ca hai tap
X_train_t = preprocess.fit_transform(X_train)
X_test_t  = preprocess.transform(X_test)
print(X_train_t.shape, X_test_t.shape)
print(preprocess.get_feature_names_out()[:10])
```

Ma trận `X_train_t` thu được chỉ gồm các số thực, đã đồng nhất thang đo và không còn giá trị thiếu — sẵn sàng cho các mô hình hồi quy ở chương tiếp theo. Khi ghép `preprocess` với một mô hình trong cùng một `Pipeline`, toàn bộ quy trình có thể được đánh giá bằng kiểm định chéo mà không lo rò rỉ dữ liệu:

Listing 5.18: Ghép tiền xử lý với mô hình trong một Pipeline

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

model = Pipeline([("prep", preprocess), ("reg", LinearRegression())])
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")
print(scores.mean().round(3))
```

5.11 Tóm tắt chương

- Tiền xử lý dữ liệu là bước nền tảng trong quy trình KDD (Fayyad và cộng sự, 1996) và pha Chuẩn bị dữ liệu của CRISP-DM; nguyên lý "rác vào, rác ra" nhắc rằng chất lượng mô hình bị giới hạn bởi chất lượng dữ liệu.
- Dữ liệu thiếu có ba cơ chế MCAR, MAR, MNAR (Rubin, 1976). **Simple Imputer** điền bằng trung bình/trung vị/mode; **KNN Imputer** lấy trung bình k láng giềng theo khoảng cách nan-Euclid $\sqrt{\frac{d}{|P|}\sum_{j\in P}(x_j - y_j)^2}$; **Iterative Imputer** hồi quy lặp từng cột theo các cột còn lại — mạnh nhưng có thể ngoại suy phi thực tế khi dữ liệu ít hoặc đa cộng tuyến, nên luôn phải kiểm tra giá trị điền.

- Ngoại lai: z-score dễ bị che lấp; quy tắc IQR của Tukey bền vững hơn; **LOF** so sánh mật độ cục bộ của một điểm với láng giềng, LOF $\gg 1$ là ngoại lai.
- Mô hình ML chỉ làm việc với số: dữ liệu chữ phải được mã hóa sao cho **không tạo ra quan hệ giả**.
- **Label Encoder** ánh xạ nhãn $\rightarrow$ số nguyên, gọn nhẹ nhưng tạo thứ tự giả; dùng cho biến mục tiêu. **Ordinal Encoder** dùng cho đặc trưng thứ bậc.
- **One Hot Encoder** biến mỗi nhãn thành một vector đơn vị, mọi nhãn cách đều $\sqrt{2}$; phù hợp với KNN, K-Means, mô hình tuyến tính.
- **Count Vectorizer** biểu diễn văn bản bằng số lần xuất hiện của từ; **TF-IDF** (Spärck Jones, 1972) giảm trọng số của các từ phổ biến; chuẩn hóa L2 cho phép đo độ tương đồng cosine bằng tích vô hướng.
- Bốn bộ chuẩn hóa **MinMax, MaxAbs, Standard, Robust** khác nhau về miền giá trị, khả năng giữ tính thưa và độ bền với ngoại lai.
- **SMOTE** sinh mẫu thiểu số bằng nội suy $\mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i)$; chỉ áp dụng trên tập huấn luyện (dùng Pipeline của imbalanced-learn) và đánh giá bằng recall, precision, F1 thay vì accuracy.
- Luôn `fit` trên tập huấn luyện; dùng `Pipeline` và `ColumnTransformer` để quy trình gọn gàng và tránh rò rỉ dữ liệu.

5.12 Bài tập

1. Cho dãy gồm các phần tử `cat`, `dog`, `fish`, `dog`, `cat`, `bird`. Hãy (a) tự cài đặt Label Encoder theo thứ tự xuất hiện; (b) so sánh với kết quả của `LabelEncoder` trong scikit-learn và giải thích vì sao khác nhau.
2. Với dãy ở bài 1, tự cài đặt One Hot Encoder bằng list, for và dictionary. Chứng minh rằng khoảng cách Euclid giữa hai vector one-hot khác nhau bất kỳ luôn bằng $\sqrt{2}$.
3. Thêm câu d_4 = "the birds are outside" vào kho văn bản ở mục TF-IDF. Tính tay $\mathrm{df}(t)$ và $\mathrm{idf}(t)$ (làm trơn) cho các từ *are, outside, the, birds*, sau đó kiểm tra bằng `TfidfVectorizer`.
4. Dùng bảng tính (Excel/Google Sheets) dựng lại toàn bộ Bảng 5.15–5.19. Gợi ý: hàm `LN`, `SQRT`, `SUMSQ`.
5. Cho cột $x = (10, 12, 11, 13, 12, 100)$. Tính tay kết quả của bốn bộ chuẩn hóa và nhận xét bộ nào phù hợp nhất. Vì sao?
6. Giải thích vì sao gọi `StandardScaler().fit(X)` trên toàn bộ dữ liệu trước khi chia train/test là rò rỉ dữ liệu. Thiết kế thí nghiệm trên Ames Housing để minh họa.
7. Với Ames Housing: đếm số lượng giá trị `NA` ("không có") và ô rỗng (thiếu thật sự) trong từng cột; bổ sung các cột `Heating QC`, `Garage Finish`, `Foundation` vào pipeline với kỹ thuật mã hóa phù hợp.

8. Cho cột $(5, 8, ?, 6, 100, 7)$. Điền giá trị thiếu bằng trung bình và bằng trung vị. Cách nào hợp lý hơn? Vì sao? (Đáp số: 25,2 và 7.)

9. Tính khoảng cách nan-Euclid giữa $(1, \text{nan}, 3, 4)$ và $(2, 5, \text{nan}, 6)$, rồi kiểm tra bằng `nan_euclidean_di`

10. Với Bảng 5.5, chạy tay lại KNN Imputer với $k = 2$, sau đó với $k = 3$ và trọng số $1/d$. So sánh với `KNNImputer(n_neighbors=2)` và `KNNImputer(n_neighbors=3, weights="distance"`

11. Tiếp tục ví dụ Iterative Imputer thêm một vòng nữa (bắt đầu từ các giá trị điền của vòng 1), rồi so sánh với `IterativeImputer(estimator=LinearRegression(), imputation_order="ro` `max_iter=2)`. Giải thích vì sao giá trị của x_3 không hội tụ về miền $[1;\ 8]$.

12. Thêm điểm $F(2, 2)$ vào ví dụ LOF và tính lại bảng LOF với $k = 3$. Hệ số LOF của E thay đổi thế nào?

13. Trên `load_breast_cancer`, giữ lại toàn bộ lớp lành tính và chỉ 10% lớp ác tính để tạo dữ liệu mất cân bằng. So sánh recall, precision, F1 của hồi quy logistic khi: không xử lý, `class_weight="balanced"`, SMOTE trong pipeline, và SMOTE áp dụng **sai** lên toàn bộ dữ liệu trước khi kiểm định chéo.

Tài liệu tham khảo của chương

1. Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. *Proceedings of ACM SIGMOD 2000*, 93–104.

2. Buitinck, L. et al. (2013). API design for machine learning software: experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*.

3. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

4. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

5. Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From Data Mining to Knowledge Discovery in Databases. *AI Magazine*, 17(3), 37–54.

6. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.), Chương 3: Data Preprocessing. Morgan Kaufmann.

7. Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning. *Advances in Intelligent Computing (ICIC 2005)*, LNCS 3644, 878–887.

8. Harris, Z. S. (1954). Distributional Structure. *Word*, 10(2–3), 146–162.

9. Hawkins, D. M. (1980). *Identification of Outliers*. Chapman and Hall.

10. He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. *Proceedings of IJCNN 2008*, 1322–1328.

11. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1–5.

12. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413–422.

13. Luhn, H. P. (1957). A Statistical Approach to Mechanized Encoding and Searching of Literary Information. *IBM Journal of Research and Development*, 1(4), 309–317.

14. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

15. Pyle, D. (1999). *Data Preparation for Data Mining*. Morgan Kaufmann.

16. Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, 63(3), 581–592.

17. Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley.

18. Salton, G., Wong, A., & Yang, C. S. (1975). A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 18(11), 613–620.

19. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

20. Shiffler, R. E. (1988). Maximum Z Scores and Outliers. *The American Statistician*, 42(1), 79–80.

21. Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.

22. Stevens, S. S. (1946). On the Theory of Scales of Measurement. *Science*, 103(2684), 677–680.

23. Suits, D. B. (1957). Use of Dummy Variables in Regression Equations. *Journal of the American Statistical Association*, 52(280), 548–551.

24. Troyanskaya, O., Cantor, M., Sherlock, G., Brown, P., Hastie, T., Tibshirani, R., Botstein, D., & Altman, R. B. (2001). Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6), 520–525.

25. Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

26. van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, 45(3), 1–67.

27. Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*.

Chương 6: Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một trong những thuật toán học có giám sát (Supervised Learning) cơ bản và phổ biến nhất trong Machine Learning, dùng để dự đoán giá trị đầu ra liên tục.

6.1 Mô hình Hồi quy tuyến tính đơn biến

Định nghĩa 6.1 (Hồi quy tuyến tính đơn biến)**.** *Cho tập dữ liệu* $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. *Mô hình biểu diễn mối quan hệ giữa biến độc lập* x *và biến phụ thuộc* y *theo dạng tuyến tính:*

$$y = w_0 + w_1 x + \epsilon \tag{6.1}$$

trong đó w_0 *là hệ số chặn (intercept),* w_1 *là hệ số góc (slope), và* ϵ *là sai số ngẫu nhiên.*

6.2 Hàm mất mát (Loss Function)

Để tìm các hệ số w_0, w_1 tối ưu, người ta sử dụng sai số bình phương trung bình (Mean Squared Error - MSE):

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - (w_0 + w_1 x_i))^2 \tag{6.2}$$

Mục tiêu của bài toán là tìm (w_0, w_1) sao cho $J(w_0, w_1)$ đạt giá trị nhỏ nhất.

6.3 Cài đặt Hồi quy tuyến tính với Scikit-Learn

Dưới đây là đoạn code Python minh họa việc xây dựng mô hình Linear Regression trên tập dữ liệu thực tế:

Listing 6.1: Thực thi Linear Regression bằng Scikit-Learn

```
import numpy as np
from sklearn.linear_model import LinearRegression

# 1. Tao du lieu gia dinh
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])

```

```
# 2. Khoi tao va huan luyen mo hinh
model = LinearRegression()
model.fit(X, y)

# 3. In ket qua trong so w0 va w1
print("He so w1 (slope):", model.coef_[0])
print("He so w0 (intercept):", model.intercept_)

# 4. Du doan gia tri moi
y_pred = model.predict([[6]])
print("Du doan cho X=6:", y_pred[0])
```

6.4 Tóm tắt chương

- Bản chất mô hình Hồi quy tuyến tính và bài toán tối ưu hàm MSE.
- Phương pháp hạ độ dốc (Gradient Descent) để cập nhật trọng số.
- Thao tác huấn luyện mô hình bằng thư viện `scikit-learn`.

Chương 7: Phân lớp (Classification)

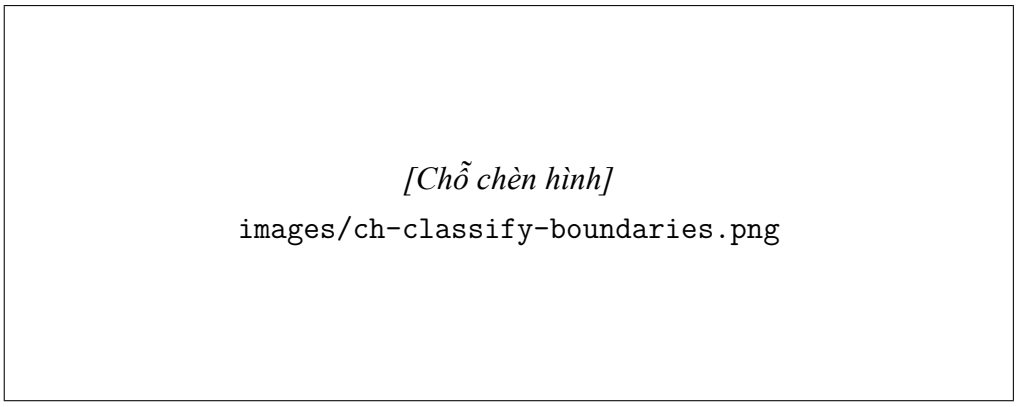
Phân lớp (classification) là bài toán học có giám sát trong đó đầu ra là một **nhãn rời rạc**: thư rác hay không, khối u lành hay ác tính, khách hàng có rời bỏ dịch vụ hay không, chữ số viết tay là 0 hay 9… Nếu chỉ có hai nhãn ta có **phân lớp nhị phân**; nhiều hơn hai nhãn là **phân lớp đa lớp**. Theo kế hoạch giảng dạy, chương này trình bày các thuật toán nền tảng: **KNN, Naive Bayes, Logistic Regression, Decision Tree, SVM**; các phương pháp kết hợp (Random Forest, AdaBoost, XGBoost) được trình bày ở chương kế tiếp.
Sau khi học xong chương này, người học có khả năng:

- Trình bày ý tưởng, lịch sử và giả định của năm họ thuật toán phân lớp kinh điển.
- Tự tính tay một bước dự đoán của KNN (bỏ phiếu đều, có trọng số, và KNN cho hồi quy), Naive Bayes, Logistic Regression và một lần chia nút của cây quyết định.
- Cài đặt và so sánh các thuật toán bằng scikit-learn trên dữ liệu thực.
- Lựa chọn thuật toán và bước tiền xử lý phù hợp cho từng loại dữ liệu.

7.1 Bài toán phân lớp

Định nghĩa 7.1 (Bài toán phân lớp)**.** *Cho tập huấn luyện* $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ *với* $\mathbf{x}_i \in \mathbb{R}^d$ *và* $y_i \in \{1, \ldots, K\}$*. Cần học hàm* $f : \mathbb{R}^d \to \{1, \ldots, K\}$ *sao cho* $f(\mathbf{x})$ *dự đoán đúng nhãn của các điểm* ***chưa từng thấy****. Nhiều thuật toán còn ước lượng xác suất* $P(y = k \mid \mathbf{x})$ *và chọn nhãn có xác suất lớn nhất.*

Về mặt hình học, bộ phân lớp chia không gian đặc trưng thành các vùng; ranh giới giữa các vùng gọi là **biên quyết định** (decision boundary). Các thuật toán khác nhau tạo ra biên có hình dạng khác nhau: tuyến tính (Logistic Regression, SVM tuyến tính), các đoạn song song trục (Decision Tree), hay biên phức tạp tùy dữ liệu (KNN, SVM với nhân).



Hình 7.1: Biên quyết định của KNN, Logistic Regression, Decision Tree và SVM (nhân RBF) trên cùng một tập dữ liệu hai chiều.

Xuyên suốt chương, ta minh họa bằng bộ dữ liệu **Iris** do Ronald Fisher công bố năm 1936: 150 bông hoa thuộc 3 loài (*setosa*, *versicolor*, *virginica*), mỗi bông có 4 đặc trưng: chiều dài, chiều rộng của đài hoa và cánh hoa (cm).

Listing 7.1: Chuẩn bị dữ liệu Iris

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X, y = iris.data, iris.target
print(X.shape, iris.target_names)   # (150, 4) ['setosa' 'versicolor'
    'virginica']

# stratify=y: giu nguyen ti le cac lop trong tap train va test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

7.2 K láng giềng gần nhất (KNN)

KNN (K-Nearest Neighbors) được Evelyn Fix và Joseph Hodges đề xuất năm 1951 trong một báo cáo cho Không quân Hoa Kỳ; năm 1967, Cover và Hart chứng minh rằng khi số mẫu tiến ra vô cùng, sai số của luật 1-NN không vượt quá **hai lần** sai số Bayes tối ưu. KNN là thuật toán **lười** (lazy learning): không học tham số nào, chỉ ghi nhớ dữ liệu và trì hoãn mọi tính toán đến lúc dự đoán.

Định nghĩa 7.2 (Thuật toán KNN)**.** *Để phân lớp điểm mới* $\mathbf{q}$*:*

1. *Tính khoảng cách từ* $\mathbf{q}$ *đến mọi điểm huấn luyện, thường dùng khoảng cách Euclid* $d(\mathbf{q}, \mathbf{x}) = \sqrt{\sum_j (q_j - x_j)^2}$.
2. *Chọn* K *điểm gần nhất.*
3. *Gán nhãn chiếm đa số trong* K *điểm đó (có thể bỏ phiếu có trọng số* $1/d$*).*

Ví dụ 7.1 (Tính tay KNN phân lớp)**.** *Cho 5 mẫu huấn luyện có hai đặc trưng x_1, x_2 và nhãn $y \in \{0, 1\}$ (Bảng 7.1). Dự đoán nhãn của mẫu mới* $\mathbf{q} = (2, 4)$ *với khoảng cách Euclid* $d = \sqrt{(x_1 - 2)^2 + (x_2 - 4)^2}$.

STT	x_1	x_2	y	Khoảng cách đến $\mathbf{q} = (2,4)$	Thứ hạng
1	7	6	0	$\sqrt{5^2 + 2^2} = \sqrt{29} \approx 5{,}385$	4
2	2	4	1	$\sqrt{0^2 + 0^2} = 0$	1
3	1	5	1	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1{,}414$	2
4	3	9	1	$\sqrt{1^2 + 5^2} = \sqrt{26} \approx 5{,}099$	3
5	8	7	0	$\sqrt{6^2 + 3^2} = \sqrt{45} \approx 6{,}708$	5

Bảng 7.1: Dữ liệu huấn luyện và khoảng cách đến điểm cần dự đoán

Với $K = 3$: ba mẫu gần nhất là 2, 3, 4, đều có nhãn $1 \Rightarrow \hat{y} = 1$. Với $K = 4$: thêm mẫu 1 (nhãn 0), kết quả bỏ phiếu $3 : 1$, vẫn dự đoán $\hat{y} = 1$.

7.2.1 KNN có trọng số

Bỏ phiếu đều coi láng giềng gần nhất và láng giềng thứ K như nhau, dù một điểm có thể rất gần còn điểm kia khá xa. Dudani (1976) đề xuất **bỏ phiếu có trọng số**: láng giềng càng gần, lá phiếu càng nặng. Trọng số phổ biến nhất là nghịch đảo khoảng cách:

$$w_i = \frac{1}{d(\mathbf{q}, \mathbf{x}_i) + \varepsilon}, \qquad \hat{y} = \arg\max_{c} \sum_{i \in N_K(\mathbf{q}),\ y_i = c} w_i, \tag{7.1}$$

trong đó ε là một số rất nhỏ (ví dụ 10^{-5}) để tránh chia cho 0 khi $\mathbf{q}$ trùng với một điểm huấn luyện.

Ví dụ 7.2 (Trọng số khi điểm cần dự đoán trùng một mẫu)**.** *Tiếp ví dụ trên với $K = 4$ và $\varepsilon = 10^{-5}$:*

$$w_2 = \frac{1}{0 + 10^{-5}} = 100\,000, \qquad w_3 = \frac{1}{1{,}414} \approx 0{,}707,$$
$$w_4 = \frac{1}{5{,}099} \approx 0{,}196, \qquad w_1 = \frac{1}{5{,}385} \approx 0{,}186.$$

Tổng trọng số của lớp 1 là $100\,000 + 0{,}707 + 0{,}196 \approx 100\,000{,}9$, của lớp 0 là $0{,}186$. Chuẩn hóa về tỉ lệ: lớp 0 chiếm $\frac{0{,}186}{100\,001{,}09} \approx 1{,}86 \times 10^{-6} \approx 0$, lớp 1 chiếm ≈ 1. Mẫu trùng khớp gần như quyết định toàn bộ kết quả. Chú ý $\frac{1}{0+10^{-5}} = 10^5$ chứ không phải 1 — một nhầm lẫn hay gặp khi tính tay. Trong scikit-learn, nếu có láng giềng cách đúng 0 thì `weights="distance"` chỉ tính các láng giềng đó.

Ví dụ 7.3 (Trọng số phá thế hòa phiếu)**.** *Cũng với dữ liệu ở Bảng 7.1, xét* $\mathbf{q} = (5, 6)$ *và $K = 4$. Khoảng cách đến các mẫu 1, 5, 2, 4 lần lượt là 2; $\sqrt{10} \approx 3{,}162$; $\sqrt{13} \approx 3{,}606$; $\sqrt{13} \approx 3{,}606$ (mẫu 3 xa nhất, $\sqrt{17} \approx 4{,}123$). Bỏ phiếu đều cho kết quả **hòa** $2 : 2$: mẫu 1, 5 nhãn 0; mẫu 2, 4 nhãn 1. Với trọng số $1/d$:*

$$W_0 = \frac{1}{2} + \frac{1}{3{,}162} \approx 0{,}816, \qquad W_1 = \frac{1}{3{,}606} + \frac{1}{3{,}606} \approx 0{,}555 \ \Rightarrow\ \hat{y} = 0.$$

Khi hòa phiếu, scikit-learn với `weights="uniform"` chọn lớp có nhãn nhỏ hơn — một quy ước tùy ý; trọng số khoảng cách cho lời giải có cơ sở hơn. Đây là lý do khi K chẵn (dễ hòa phiếu) nên dùng KNN có trọng số.

7.2.2 KNN cho bài toán hồi quy

KNN dùng được cho cả hồi quy: thay vì bỏ phiếu, ta lấy **trung bình** (hoặc trung bình có trọng số) **giá trị mục tiêu** của K láng giềng:

$$\hat{y} = \frac{1}{K} \sum_{i \in N_K(\mathbf{q})} y_i \qquad \text{hoặc} \qquad \hat{y} = \frac{\sum_{i \in N_K(\mathbf{q})} w_i \, y_i}{\sum_{i \in N_K(\mathbf{q})} w_i}, \quad w_i = \frac{1}{d(\mathbf{q}, \mathbf{x}_i)}. \tag{7.2}$$

Ví dụ 7.4 (Tính tay KNN hồi quy với $K = 4$)**.** *Cùng 5 mẫu như Bảng 7.1 nhưng y là số thực. Dự đoán cho* $\mathbf{q} = (3, 4)$.

STT	x_1	x_2	y	$d(\mathbf{q}, \mathbf{x}_i)$	$w_i = 1/d$
1	7	6	7,3	$\sqrt{4^2 + 2^2} \approx 4{,}472$	0,224
2	2	4	5,4	$\sqrt{1^2 + 0^2} = 1$	1
3	1	5	4,2	$\sqrt{2^2 + 1^2} \approx 2{,}236$	0,447
4	3	9	5,1	$\sqrt{0^2 + 5^2} = 5$	0,2
5	8	7	9,6	$\sqrt{5^2 + 3^2} \approx 5{,}831$	(không thuộc $K = 4$)

Bảng 7.2: KNN hồi quy: khoảng cách và trọng số đến $\mathbf{q} = (3, 4)$

Bốn láng giềng gần nhất là mẫu 2, 3, 1, 4.

- Trung bình đều: $\hat{y} = \dfrac{5{,}4 + 4{,}2 + 7{,}3 + 5{,}1}{4} = \dfrac{22}{4} = 5{,}5.$
- Trung bình có trọng số: $\hat{y} = \dfrac{5{,}4 \times 1 + 4{,}2 \times 0{,}447 + 7{,}3 \times 0{,}224 + 5{,}1 \times 0{,}2}{1 + 0{,}447 + 0{,}224 + 0{,}2} \approx 5{,}31.$

Lưu ý: phép trung bình lấy trên **giá trị mục tiêu** y_i của các láng giềng, không phải trên khoảng cách. Đây cũng chính là cơ chế của KNN Imputer ở mục 5.3: cột cần điền đóng vai trò y.

Lưu ý thực hành.

- KNN dựa hoàn toàn vào khoảng cách nên **bắt buộc chuẩn hóa dữ liệu** và **mã hóa One Hot** cho biến định danh (Chương 5).
- K nhỏ: biên quyết định gồ ghề, nhạy với nhiễu; K lớn: biên mượt nhưng có thể thiếu khớp. Với bài toán hai lớp nên chọn K lẻ để tránh hòa phiếu.
- Chi phí dự đoán lớn với dữ liệu nhiều mẫu; chất lượng giảm mạnh khi số chiều cao (*lời nguyền số chiều*, xem chương Giảm chiều dữ liệu).

Listing 7.2: KNN với scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn = make_pipeline(StandardScaler(), KNeighborsClassifier(
   n_neighbors=5))
knn.fit(X_train, y_train)
```

```
print("Accuracy:", knn.score(X_test, y_test))

# Cac vi du tinh tay o tren
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

X5 = np.array([[7, 6], [2, 4], [1, 5], [3, 9], [8, 7]])
y5 = np.array([0, 1, 1, 1, 0])
print(KNeighborsClassifier(3).fit(X5, y5).predict([[2, 4]]))
            # [1]
knn_w = KNeighborsClassifier(4, weights="distance").fit(X5, y5)
print(knn_w.predict_proba([[5, 6]]).round(4))
                           # [[0.5954 0.4046]]

y_reg = np.array([7.3, 5.4, 4.2, 5.1, 9.6])
for w in ["uniform", "distance"]:
    reg = KNeighborsRegressor(4, weights=w).fit(X5, y_reg)
    print(w, reg.predict([[3, 4]]).round(4))
                                  # 5.5 ; 5.3082
```

7.3 Naive Bayes

Định lý Bayes do mục sư Thomas Bayes đề xuất và được công bố sau khi ông mất (1763), sau đó Pierre-Simon Laplace phát triển độc lập và tổng quát hóa. Năm 1961, Maron dùng mô hình Bayes "ngây thơ" để phân loại tài liệu tự động; đến cuối thập niên 1990, Naive Bayes trở thành phương pháp lọc thư rác kinh điển.

Định nghĩa 7.3 (Bộ phân lớp Naive Bayes)**.** *Theo định lý Bayes:*

$$P(y \mid \mathbf{x}) = \frac{P(y)\,P(\mathbf{x} \mid y)}{P(\mathbf{x})} \propto P(y)\,P(\mathbf{x} \mid y). \tag{7.3}$$

Giả định "ngây thơ" (naive): các đặc trưng ***độc lập có điều kiện*** *khi biết lớp, nên* $P(\mathbf{x} \mid y) = \prod_{j=1}^{d} P(x_j \mid y)$*. Nhãn dự đoán là*

$$\hat{y} = \arg\max_{k} \; P(y = k) \prod_{j=1}^{d} P(x_j \mid y = k). \tag{7.4}$$

Ví dụ 7.5 (Lọc thư rác)**.** *Một hộp thư có 6 email: 3 spam, 3 không spam (ham). Từ "free" xuất hiện trong 2/3 email spam và 1/3 email ham; từ "meeting" xuất hiện trong 1/3 email spam và 2/3 email ham. Phân lớp một email* ***có*** *"free" và* ***không có*** *"meeting":*

$$\begin{aligned} \textit{Spam}: &\quad P(\textit{spam}) \cdot P(\textit{free} \mid \textit{spam}) \cdot P(\neg\textit{meeting} \mid \textit{spam}) = \tfrac{1}{2} \cdot \tfrac{2}{3} \cdot \tfrac{2}{3} = 0{,}2222, \\ \textit{Ham}: &\quad P(\textit{ham}) \cdot P(\textit{free} \mid \textit{ham}) \cdot P(\neg\textit{meeting} \mid \textit{ham}) = \tfrac{1}{2} \cdot \tfrac{1}{3} \cdot \tfrac{1}{3} = 0{,}0556. \end{aligned}$$

Chuẩn hóa: $P(\textit{spam} \mid \mathbf{x}) = 0{,}2222/(0{,}2222 + 0{,}0556) = 0{,}8$*. Email được dự đoán là* ***spam*** *với xác suất 80%.*

Các biến thể. `GaussianNB` (đặc trưng liên tục, giả định phân phối chuẩn trong mỗi lớp), `MultinomialNB` (đặc trưng đếm — rất hợp với ma trận Count/TF-IDF ở Chương 5), `BernoulliNB` (đặc trưng nhị phân có/không). Để tránh xác suất bằng 0 khi một từ chưa từng xuất hiện trong một lớp, dùng **làm trơn Laplace**: $P(x_j \mid y) = \frac{\text{đếm}+\alpha}{\text{tổng}+\alpha\cdot|V|}$ (tham số `alpha`).

Listing 7.3: Naive Bayes cho phân loại văn bản kết hợp TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.pipeline import make_pipeline

texts = ["free money now", "win a free prize", "free offer today",
         "meeting at noon", "project meeting notes", "lunch meeting
            free"]
labels = ["spam", "spam", "spam", "ham", "ham", "ham"]

spam_filter = make_pipeline(TfidfVectorizer(), MultinomialNB(alpha
   =1.0))
spam_filter.fit(texts, labels)
print(spam_filter.predict(["free prize money", "notes from the
   meeting"]))
print(spam_filter.predict_proba(["free prize money"]).round(3))

# Du lieu so lien tuc: GaussianNB
print(GaussianNB().fit(X_train, y_train).score(X_test, y_test))
```

Dù giả định độc lập hiếm khi đúng, Naive Bayes thường cho kết quả tốt bất ngờ với văn bản, huấn luyện cực nhanh và cần ít dữ liệu. Nhược điểm: xác suất ước lượng thường bị "quá tự tin" (gần 0 hoặc 1).

7.4 Hồi quy logistic (Logistic Regression)

Hàm logistic được nhà toán học Bỉ Pierre-François Verhulst dùng từ thập niên 1830–1840 để mô tả tăng trưởng dân số. Năm 1958, David Cox đề xuất mô hình hồi quy logistic cho biến đầu ra nhị phân. Dù tên gọi có chữ "hồi quy", đây là **thuật toán phân lớp**.

Định nghĩa 7.4 (Hồi quy logistic nhị phân)**.** *Mô hình ước lượng xác suất thuộc lớp dương:*

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{7.5}$$

và dự đoán $\hat{y} = 1$ khi xác suất $\geq 0{,}5$, tức khi $\mathbf{w}^\top \mathbf{x} + b \geq 0$. Do đó biên quyết định là một ***siêu phẳng****.*

z	-2	0	1	2
$\sigma(z)$	0.1192	0.5	0.7311	0.8808

Bảng 7.3: Một số giá trị của hàm sigmoid

Hàm mất mát. Tham số được tìm bằng cách cực tiểu **entropy chéo** (log loss), tương đương cực đại hợp lý:

$$J(\mathbf{w}, b) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\Big], \qquad \hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b). \tag{7.6}$$

Hàm này lồi, không có nghiệm đóng, nên được giải bằng Gradient Descent hoặc các phương pháp tối ưu bậc hai (thuật toán `lbfgs` mặc định của scikit-learn). Gradient có dạng rất gọn: $\nabla_{\mathbf{w}} J = \frac{1}{n}\sum_i (\hat{p}_i - y_i)\,\mathbf{x}_i$ — giống hệt hồi quy tuyến tính, chỉ thay $\hat{y}$ bằng $\hat{p}$.

Diễn giải. $\ln \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b$: mỗi khi x_j tăng 1 đơn vị, *odds* (tỉ số $p/(1-p)$) được nhân với e^{w_j}. Đây là lý do hồi quy logistic được ưa chuộng trong y khoa và tài chính, nơi cần giải thích mô hình.

Đa lớp. Với K lớp, dùng hàm **softmax**: $P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ với $z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$. Tham số `C` trong scikit-learn là nghịch đảo của mức chính quy hóa L2 (giống Ridge): C nhỏ $\Rightarrow$ chính quy hóa mạnh.

Listing 7.4: Logistic Regression với scikit-learn

```
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(StandardScaler(), LogisticRegression(C=1.0,
    max_iter=1000))
logreg.fit(X_train, y_train)
print("Accuracy:", logreg.score(X_test, y_test))
print(logreg.predict_proba(X_test[:3]).round(3))   # xac suat 3 lop
print(logreg[-1].coef_.round(2))                   # he so cho tung
    lop
```

[Chỗ chèn hình]
`images/ch-classify-sigmoid.png`

Hình 7.2: Đồ thị hàm sigmoid $\sigma(z)$ và ngưỡng quyết định 0,5.

7.5 Cây quyết định (Decision Tree)

Cây quyết định mô phỏng cách con người ra quyết định bằng chuỗi câu hỏi "nếu – thì". Hai dòng thuật toán kinh điển: **CART** (Classification and Regression Trees) của Breiman, Friedman, Olshen và Stone (1984), và **ID3** (1986) cùng hậu duệ C4.5 (1993) của Ross Quinlan. scikit-learn cài đặt phiên bản tối ưu của CART.

7.5.1 Độ đo độ hỗn tạp

Tại mỗi nút có tỉ lệ các lớp $p_1, \ldots, p_K$, độ "hỗn tạp" (impurity) được đo bằng:

$$\text{Gini (CART)}: \quad G = 1 - \sum_{k=1}^{K} p_k^2, \tag{7.7}$$

$$\text{Entropy (ID3/C4.5)}: \quad H = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{7.8}$$

Nút "thuần" (chỉ một lớp) có $G = H = 0$; với hai lớp cân bằng, $G = 0{,}5$ và $H = 1$. Thuật toán chọn câu hỏi chia nút làm **giảm độ hỗn tạp nhiều nhất**, tính theo trung bình có trọng số của các nút con. Với entropy, mức giảm này gọi là **độ lợi thông tin** (Information Gain):

$$\text{IG} = H(\text{cha}) - \sum_{c \in \text{con}} \frac{n_c}{n} H(c). \tag{7.9}$$

Ví dụ 7.6 (Tính một lần chia nút)**.** *Nút cha có 10 mẫu: 6 "Có", 4 "Không". Một câu hỏi chia thành nút trái 4 mẫu (4 Có, 0 Không) và nút phải 6 mẫu (2 Có, 4 Không).*

- *Gini cha:* $1 - (0{,}6^2 + 0{,}4^2) = 0{,}48$.
- *Gini trái:* 0*; Gini phải:* $1 - \big((2/6)^2 + (4/6)^2\big) = 0{,}4444$.
- *Gini sau chia:* $0{,}4 \times 0 + 0{,}6 \times 0{,}4444 = 0{,}2667 \Rightarrow$ *giảm* $0{,}2133$.
- *Entropy cha:* $0{,}9710$*; entropy phải:* $0{,}9183$*;* $\text{IG} = 0{,}9710 - 0{,}6 \times 0{,}9183 = 0{,}4200$.

Thuật toán thử mọi đặc trưng và mọi ngưỡng, rồi chọn phép chia có mức giảm lớn nhất.

Kiểm soát quá khớp. Cây phát triển hết cỡ sẽ có lá thuần, khớp hoàn hảo dữ liệu huấn luyện nhưng tổng quát kém. Các tham số giới hạn: `max_depth`, `min_samples_split`, `min_samples_leaf`, hoặc cắt tỉa sau bằng `ccp_alpha`.

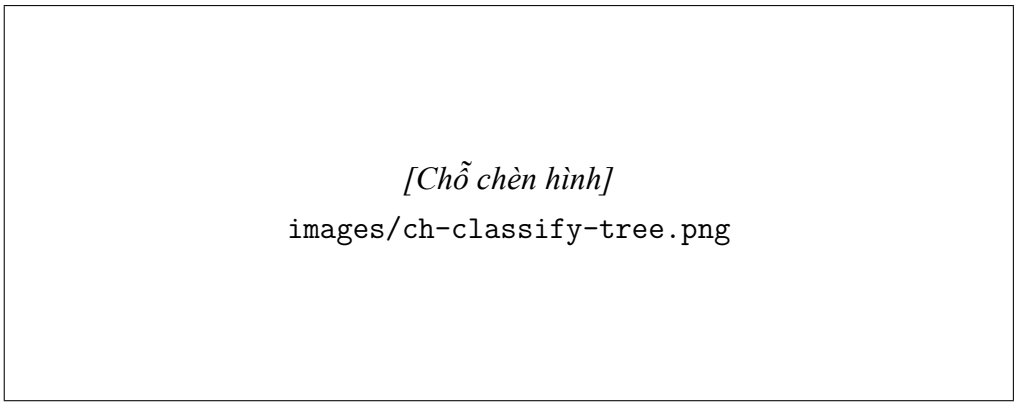
Ưu điểm. Dễ diễn giải, vẽ được; không cần chuẩn hóa dữ liệu; xử lý được quan hệ phi tuyến và tương tác. **Nhược điểm:** không ổn định (đổi dữ liệu một chút, cây thay đổi nhiều), dễ quá khớp — được khắc phục bởi Random Forest ở chương sau.

Listing 7.5: Cây quyết định và trực quan hóa

```
from sklearn.tree import DecisionTreeClassifier, export_text,
    plot_tree
import matplotlib.pyplot as plt

tree = DecisionTreeClassifier(criterion="gini", max_depth=3,
    random_state=42)
tree.fit(X_train, y_train)
print("Accuracy:", tree.score(X_test, y_test))
print(export_text(tree, feature_names=iris.feature_names))

plt.figure(figsize=(12, 6))
plot_tree(tree, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```



Hình 7.3: Cây quyết định độ sâu 3 trên dữ liệu Iris (kết quả của `plot_tree`).

7.6 Máy vector hỗ trợ (SVM)

Ý tưởng siêu phẳng lề cực đại được Vladimir Vapnik và Alexey Chervonenkis phát triển tại Liên Xô từ thập niên 1960. Năm 1992, Boser, Guyon và Vapnik kết hợp với **thủ thuật nhân** (kernel trick) để tạo biên phi tuyến; năm 1995, Cortes và Vapnik đề xuất phiên bản **lề mềm** (soft margin) dùng phổ biến đến ngày nay.

7.6.1 Lề cực đại

Với dữ liệu hai lớp $y_i \in \{-1, +1\}$ tách được tuyến tính, có vô số siêu phẳng $\mathbf{w}^\top \mathbf{x} + b = 0$ phân tách hai lớp. SVM chọn siêu phẳng có **lề** (khoảng cách đến điểm gần nhất của mỗi lớp) lớn nhất. Độ rộng lề bằng $2/\|\mathbf{w}\|$, nên bài toán là:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1,\ \forall i. \tag{7.10}$$

Các điểm nằm đúng trên biên của lề gọi là **vector hỗ trợ** (support vectors); chỉ chúng quyết định siêu phẳng — bỏ các điểm khác đi, kết quả không đổi.

7.6.2 Lề mềm và tham số C

Dữ liệu thực thường không tách được hoàn toàn. Lề mềm cho phép vi phạm với biến bù $\xi_i \geq 0$:

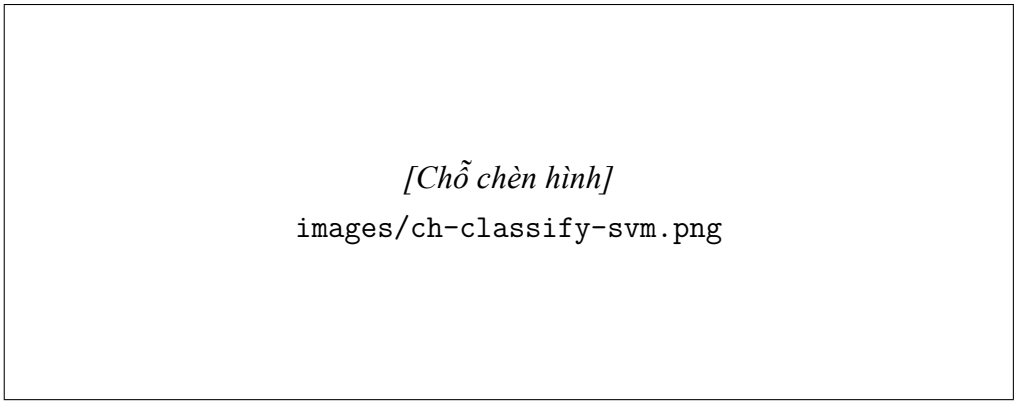
$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i. \tag{7.11}$$

C lớn: phạt vi phạm nặng, lề hẹp, dễ quá khớp; C nhỏ: lề rộng, chấp nhận sai một số điểm, tổng quát tốt hơn.

7.6.3 Thủ thuật nhân

Ý tưởng: ánh xạ dữ liệu sang không gian nhiều chiều hơn, nơi chúng tách được tuyến tính. Vì bài toán đối ngẫu của SVM chỉ cần tích vô hướng giữa các điểm, ta thay tích vô hướng bằng một **hàm nhân** $k(\mathbf{x}, \mathbf{z})$ mà không cần tính ánh xạ tường minh:

- Tuyến tính: $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- Đa thức: $k(\mathbf{x}, \mathbf{z}) = (\gamma\, \mathbf{x}^\top \mathbf{z} + r)^p$.
- RBF (Gauss): $k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$ — mặc định trong scikit-learn; γ lớn cho biên càng "uốn lượn".



Hình 7.4: SVM tuyến tính: siêu phẳng phân tách, lề cực đại và các vector hỗ trợ (khoanh tròn).

Listing 7.6: SVM với tìm kiếm tham số C và gamma

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

svm = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = GridSearchCV(svm, {"svc__C": [0.1, 1, 10, 100],
                          "svc__gamma": [0.01, 0.1, 1]}, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.score(X_test, y_test))
print("So vector ho tro:", grid.best_estimator_[-1].n_support_)
```

SVM cần **chuẩn hóa dữ liệu** (vì dựa trên khoảng cách và tích vô hướng), hoạt động tốt trên dữ liệu số chiều cao như văn bản, nhưng huấn luyện chậm khi số mẫu lớn (độ phức tạp cỡ $O(n^2)$ đến $O(n^3)$) và không trực tiếp cho xác suất.

7.7 So sánh các thuật toán

Listing 7.7: So sánh năm thuật toán bằng kiểm định chéo

```
from sklearn.model_selection import cross_val_score

models = {
    "KNN":          make_pipeline(StandardScaler(),
       KNeighborsClassifier(5)),
    "NaiveBayes":   GaussianNB(),
    "Logistic":     make_pipeline(StandardScaler(), LogisticRegression
       (max_iter=1000)),
    "DecisionTree": DecisionTreeClassifier(max_depth=3, random_state
       =42),
```

```
    "SVM":          make_pipeline(StandardScaler(), SVC()),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=5)
    print(f"{name:12s} {s.mean():.3f} +/- {s.std():.3f}")
```

Thuật toán	Biên quyết định	Cần chuẩn hóa	Ưu điểm	Nhược điểm
KNN	Linh hoạt	Có	Đơn giản, không cần huấn luyện	Chậm khi dự đoán, kém ở số chiều cao
Naive Bayes	Tuyến tính/bậc 2	Không	Rất nhanh, tốt với văn bản	Giả định độc lập, xác suất quá tự tin
Logistic Regression	Tuyến tính	Có	Cho xác suất, dễ diễn giải	Không nắm bắt phi tuyến nếu không tạo đặc trưng
Decision Tree	Song song trục	Không	Dễ giải thích, xử lý dữ liệu hỗn hợp	Dễ quá khớp, không ổn định
SVM	Tuyến tính hoặc theo nhân	Có	Mạnh ở số chiều cao, lề tối ưu	Chậm với dữ liệu lớn, khó chọn tham số

Bảng 7.4: So sánh các thuật toán phân lớp cơ bản

Các chỉ số đánh giá như ma trận nhầm lẫn, precision, recall, F1 và ROC-AUC được trình bày ở chương Đánh giá mô hình.

7.8 Tóm tắt chương

- KNN gán nhãn theo đa số của K láng giềng gần nhất; bỏ phiếu có trọng số $1/(d+\varepsilon)$ giúp phá thế hòa; với hồi quy, KNN lấy trung bình (có trọng số) giá trị mục tiêu của láng giềng. Cần chuẩn hóa dữ liệu và chọn K phù hợp.
- Naive Bayes áp dụng định lý Bayes với giả định độc lập có điều kiện; rất hiệu quả cho văn bản.
- Logistic Regression ước lượng xác suất qua hàm sigmoid/softmax, tối ưu entropy chéo; biên tuyến tính, dễ diễn giải.
- Cây quyết định chia nút theo Gini hoặc Entropy; dễ giải thích nhưng dễ quá khớp.
- SVM tìm siêu phẳng lề cực đại; thủ thuật nhân cho phép biên phi tuyến.

7.9 Bài tập

1. Cho 6 điểm $P_1(1,1)$, $P_2(2,1)$, $P_3(1,2)$ thuộc lớp A và $P_4(5,5)$, $P_5(6,5)$, $P_6(5,6)$ thuộc lớp B. (a) Phân lớp $\mathbf{q} = (2{,}5;\ 2{,}5)$ với $K = 3$ (đáp số: A). (b) Phân lớp $\mathbf{q} = (4,4)$ với $K = 1, 3, 5$. Khi $K = 5$, kết quả có thay đổi nếu dùng bỏ phiếu có trọng số $1/d$ không?

2. Với dữ liệu Bảng 7.2, dự đoán y tại $\mathbf{q} = (6, 6)$ bằng KNN hồi quy với $K = 2$ và $K = 3$, theo cả trung bình đều và trung bình có trọng số. Kiểm tra bằng `KNeighborsRegressor`.

3. Trong ví dụ lọc thư rác, tính lại xác suất spam khi email có cả "free" và "meeting".

4. Một nút có 8 mẫu (5 A, 3 B) được chia thành hai nút (4 A, 0 B) và (1 A, 3 B). Tính Gini, Entropy trước và sau chia và độ lợi thông tin.

5. Chứng minh $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ và suy ra công thức gradient của hàm log loss.

6. Trên bộ dữ liệu `load_breast_cancer`, so sánh năm thuật toán có và không có `StandardScaler`. Thuật toán nào nhạy với chuẩn hóa nhất? Giải thích.

7. Dùng `TfidfVectorizer` kết hợp `MultinomialNB` và `LinearSVC` trên bộ dữ liệu 20 Newsgroups (hàm `fetch_20newsgroups`, chọn 4 chủ đề). So sánh độ chính xác và thời gian huấn luyện.

Tài liệu tham khảo của chương

1. Bayes, T. (1763). An Essay towards Solving a Problem in the Doctrine of Chances. *Philosophical Transactions of the Royal Society of London*, 53, 370–418.

2. Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of COLT '92*, 144–152.

3. Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

5. Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

6. Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society, Series B*, 20(2), 215–242.

7. Dudani, S. A. (1976). The Distance-Weighted k-Nearest-Neighbor Rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4), 325–327.

8. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.

9. Fix, E., & Hodges, J. L. (1951). *Discriminatory Analysis. Nonparametric Discrimination: Consistency Properties*. USAF School of Aviation Medicine, Randolph Field, Texas.

10. Maron, M. E. (1961). Automatic Indexing: An Experimental Inquiry. *Journal of the ACM*, 8(3), 404–417.

11. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.

Chương 8: Học kết hợp (Ensemble Learning)

Một cây quyết định đơn lẻ dễ quá khớp và không ổn định; một mô hình tuyến tính có thể quá đơn giản. **Học kết hợp** (ensemble learning) xây dựng **nhiều mô hình** rồi gộp dự đoán của chúng, thường cho kết quả tốt hơn bất kỳ mô hình thành phần nào. Trong kế hoạch giảng dạy, các thuật toán **Random Forest, AdaBoost, XGBoost** thuộc nhóm phân lớp; chúng cũng dùng được cho hồi quy. Trên dữ liệu dạng bảng (tabular), các phương pháp boosting hiện vẫn là lựa chọn hàng đầu trong nhiều cuộc thi khoa học dữ liệu.
Sau khi học xong chương này, người học có khả năng:

- Giải thích vì sao kết hợp nhiều mô hình yếu lại cho mô hình mạnh (định lý bồi thẩm đoàn Condorcet, phân rã độ chệch – phương sai).
- Phân biệt hai chiến lược chính: **bagging** (song song, giảm phương sai) và **boosting** (tuần tự, giảm độ chệch).
- Tự tính một vòng cập nhật trọng số của AdaBoost.
- Cài đặt và tinh chỉnh Random Forest, AdaBoost, Gradient Boosting, XGBoost; đọc mức độ quan trọng của đặc trưng.

8.1 Trí tuệ đám đông

Năm 1785, Hầu tước de Condorcet phát biểu **định lý bồi thẩm đoàn**: nếu mỗi thành viên độc lập đưa ra quyết định đúng với xác suất $p > 0{,}5$, thì xác suất quyết định theo đa số là đúng sẽ tăng dần về 1 khi số thành viên tăng. Năm 1907, Francis Galton quan sát 787 người dự đoán cân nặng một con bò tại hội chợ: giá trị *trung vị* các dự đoán chỉ lệch khoảng 1% so với cân nặng thật — chính xác hơn hầu hết từng cá nhân.

Ví dụ 8.1. *Ba bộ phân lớp **độc lập**, mỗi bộ đúng với xác suất* $0{,}7$. *Bỏ phiếu đa số đúng khi có ít nhất 2 trong 3 bộ đúng:*

$$P(\text{đa số đúng}) = 0{,}7^3 + 3 \times 0{,}7^2 \times 0{,}3 = 0{,}343 + 0{,}441 = 0{,}784 > 0{,}7.$$

Điều kiện then chốt là các mô hình phải **đủ tốt** (tốt hơn đoán ngẫu nhiên) và **đa dạng** (sai ở những chỗ khác nhau). Nếu cả ba mô hình giống hệt nhau, bỏ phiếu chẳng cải thiện gì. Mọi phương pháp ensemble đều là các cách khác nhau để tạo ra sự đa dạng.

8.1.1 Độ chệch và phương sai

Sai số kỳ vọng của một mô hình trên dữ liệu mới có thể phân rã thành

$$\text{Sai số} = \text{Độ chệch}^2 + \text{Phương sai} + \text{Nhiễu không thể giảm}. \tag{8.1}$$

Độ chệch (bias) cao: mô hình quá đơn giản, thiếu khớp. **Phương sai** (variance) cao: mô hình thay đổi mạnh theo dữ liệu huấn luyện, quá khớp. Lấy trung bình B mô hình có phương sai σ^2, tương quan đôi một ρ, phương sai của trung bình là

$$\rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2, \tag{8.2}$$

nên **giảm tương quan** ρ giữa các mô hình là chìa khóa — đây chính là ý tưởng của Random Forest.

[Chỗ chèn hình]
`images/ch-ensemble-bagging-boosting.png`

Hình 8.1: So sánh hai chiến lược: bagging huấn luyện song song trên các mẫu bootstrap rồi bỏ phiếu; boosting huấn luyện tuần tự, mỗi mô hình tập trung vào lỗi của mô hình trước.

8.2 Bagging và Random Forest

8.2.1 Bagging

Bagging (Bootstrap AGGregatING) do Leo Breiman đề xuất năm 1996:

1. Tạo B **mẫu bootstrap**: mỗi mẫu gồm n điểm được rút **có hoàn lại** từ n điểm huấn luyện (kỹ thuật bootstrap của Efron, 1979).
2. Huấn luyện một mô hình (thường là cây quyết định sâu) trên mỗi mẫu.
3. Dự đoán bằng bỏ phiếu đa số (phân lớp) hoặc lấy trung bình (hồi quy).

Xác suất một điểm **không** được chọn vào một mẫu bootstrap là $(1-1/n)^n \approx e^{-1} \approx 0{,}368$. Như vậy mỗi cây chỉ thấy khoảng 63,2% dữ liệu; 36,8% còn lại gọi là **dữ liệu ngoài túi** (out-of-bag, OOB) và được dùng để ước lượng sai số miễn phí, không cần tập kiểm tra riêng.

8.2.2 Random Forest

Năm 2001, Breiman đề xuất **Random Forest**: bagging cây quyết định, và thêm một nguồn ngẫu nhiên — **tại mỗi nút**, chỉ xét một tập con ngẫu nhiên gồm m đặc trưng để chọn phép chia (thường $m = \sqrt{d}$ cho phân lớp). Nhờ vậy, dù có một đặc trưng rất mạnh, không phải cây nào cũng dùng nó ở gốc; các cây ít tương quan hơn và phương sai của rừng giảm mạnh.

Tham số	**Ý nghĩa**
`n_estimators`	Số cây; càng nhiều càng ổn định (không gây quá khớp), chỉ tốn thời gian.
`max_features`	Số đặc trưng xét ở mỗi nút (`"sqrt"`, `"log2"`, số thực); nhỏ ⇒ cây đa dạng hơn.
`max_depth`, `min_samples_leaf`	Giới hạn độ sâu và kích thước lá của từng cây.
`oob_score`	Tính độ chính xác trên dữ liệu ngoài túi.
`n_jobs=-1`	Huấn luyện song song trên mọi nhân CPU.

Bảng 8.1: Các tham số chính của `RandomForestClassifier`

Mức độ quan trọng của đặc trưng. Random Forest cung cấp `feature_importances_`: tổng mức giảm Gini do mỗi đặc trưng đóng góp, lấy trung bình qua các cây. Chỉ số này thiên vị đặc trưng có nhiều giá trị; **permutation importance** (xáo trộn một cột và đo mức giảm độ chính xác trên tập kiểm tra) đáng tin cậy hơn.

Listing 8.1: Random Forest trên dữ liệu ung thư vú Wisconsin

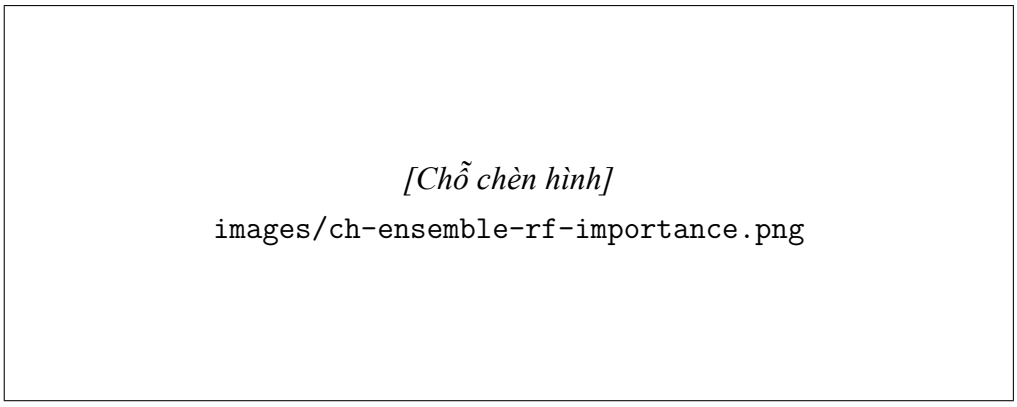
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
import pandas as pd

data = load_breast_cancer()
X, y = data.data, data.target              # 569 mau, 30 dac trung
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=300, max_features="sqrt",
                            oob_score=True, n_jobs=-1, random_state
                               =42)
rf.fit(X_train, y_train)
print("OOB score :", rf.oob_score_)
print("Test score:", rf.score(X_test, y_test))

imp = pd.Series(rf.feature_importances_, index=data.feature_names)
print(imp.sort_values(ascending=False).head(5))

perm = permutation_importance(rf, X_test, y_test, n_repeats=10,
   random_state=0)
print(pd.Series(perm.importances_mean, index=data.feature_names)
        .sort_values(ascending=False).head(5))
```



Hình 8.2: Mức độ quan trọng của 10 đặc trưng hàng đầu theo Random Forest trên dữ liệu ung thư vú.

8.3 AdaBoost

Năm 1988, Michael Kearns và Leslie Valiant đặt câu hỏi: liệu có thể "khuếch đại" một tập các bộ học **yếu** (chỉ tốt hơn đoán ngẫu nhiên một chút) thành một bộ học **mạnh**? Robert Schapire trả lời khẳng định năm 1990, và năm 1997 Yoav Freund cùng Schapire công bố **AdaBoost** (Adaptive Boosting) — công trình được trao giải Gödel năm 2003.

Định nghĩa 8.1 (Thuật toán AdaBoost nhị phân, $y_i \in \{-1, +1\}$)**.** *1. Khởi tạo trọng số các mẫu* $w_i = 1/n$.

2. *Với* $t = 1, \dots, T$:
   (a) *Huấn luyện bộ học yếu* h_t *(thường là* ***gốc cây*** *– decision stump, cây độ sâu 1) trên dữ liệu có trọng số* w.
   (b) *Tính sai số có trọng số* $\varepsilon_t = \sum_{i:\, h_t(\mathbf{x}_i) \neq y_i} w_i$.
   (c) *Tính trọng số của bộ học:* $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$.
   (d) *Cập nhật:* $w_i \leftarrow w_i \cdot e^{-\alpha_t y_i h_t(\mathbf{x}_i)}$*, rồi chuẩn hóa để* $\sum_i w_i = 1$.
3. *Dự đoán:* $H(\mathbf{x}) = \operatorname{sign}\Big(\sum_{t=1}^{T} \alpha_t h_t(\mathbf{x}) \Big)$.

Mẫu bị đoán sai được nhân với $e^{\alpha_t} > 1$ (tăng trọng số), mẫu đoán đúng nhân với $e^{-\alpha_t} < 1$ (giảm trọng số). Bộ học kế tiếp vì thế phải "chú ý" nhiều hơn đến các mẫu khó. Bộ học có sai số nhỏ có α_t lớn, tức tiếng nói lớn hơn trong lần bỏ phiếu cuối.

Ví dụ 8.2 (Một vòng AdaBoost)**.** *Có 10 mẫu, trọng số ban đầu* $0{,}1$. *Bộ học yếu đầu tiên đoán sai 3 mẫu:*

- $\varepsilon_1 = 3 \times 0{,}1 = 0{,}3$; $\quad \alpha_1 = \frac{1}{2} \ln \frac{0{,}7}{0{,}3} = 0{,}4236$.
- *Mẫu sai:* $0{,}1 \times e^{0{,}4236} = 0{,}1 \times 1{,}5275 = 0{,}15275$.
- *Mẫu đúng:* $0{,}1 \times e^{-0{,}4236} = 0{,}1 \times 0{,}6547 = 0{,}06547$.
- *Hằng số chuẩn hóa:* $Z = 3 \times 0{,}15275 + 7 \times 0{,}06547 = 0{,}9165$.

- *Trọng số mới: mẫu sai* $0{,}15275/0{,}9165 = 0{,}1667$*; mẫu đúng* $0{,}06547/0{,}9165 = 0{,}0714$.

Tổng trọng số của các mẫu sai bây giờ là $3 \times 0{,}1667 = 0{,}5$*: đối với trọng số mới, bộ học vừa rồi chỉ còn ngang đoán ngẫu nhiên, buộc bộ học tiếp theo phải khác nó.*

Listing 8.2: AdaBoost với gốc cây

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),   # decision stump
    n_estimators=200, learning_rate=0.5, random_state=42)
ada.fit(X_train, y_train)
print("Test score:", ada.score(X_test, y_test))
print(ada.estimator_weights_[:5].round(3))    # cac alpha_t dau tien
```

AdaBoost nhạy với **nhiễu nhãn và ngoại lai**: các mẫu bị gán nhãn sai liên tục được tăng trọng số và kéo mô hình lệch đi.

8.4 Gradient Boosting và XGBoost

8.4.1 Gradient Boosting

Năm 2001, Jerome Friedman tổng quát hóa boosting thành **Gradient Boosting**: xây dựng mô hình cộng $F_M(\mathbf{x}) = F_0 + \sum_{m=1}^{M} \eta\, h_m(\mathbf{x})$, trong đó mỗi cây mới h_m được huấn luyện để khớp với **gradient âm của hàm mất mát** (gọi là *giả phần dư*) tại mô hình hiện tại:

$$r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}. \tag{8.3}$$

Với mất mát bình phương $L = \frac{1}{2}(y-F)^2$, giả phần dư chính là **phần dư** $y_i - F_{m-1}(\mathbf{x}_i)$: mỗi cây mới học cách sửa sai số còn lại của các cây trước. Đây là Gradient Descent trong *không gian hàm*. Tốc độ học η (`learning_rate`, thường 0,01–0,1) càng nhỏ càng cần nhiều cây nhưng tổng quát càng tốt.

Ví dụ 8.3 (Ý tưởng phần dư)**.** *Cần dự đoán giá ba căn nhà (nghìn USD): 150, 200, 250. Mô hình khởi đầu* F_0 *= trung bình = 200, phần dư là* $(-50, 0, +50)$*. Cây thứ nhất học phần dư này, giả sử dự đoán được* $(-40, 0, +40)$*; với* $\eta = 0{,}5$*, mô hình mới là* $200 + 0{,}5 \times (-40, 0, 40) = (180, 200, 220)$*, phần dư còn* $(-30, 0, 30)$*. Cây thứ hai tiếp tục học phần dư mới, cứ thế sai số giảm dần.*

8.4.2 XGBoost

XGBoost (eXtreme Gradient Boosting) do Tianqi Chen và Carlos Guestrin công bố năm 2016, trở nên nổi tiếng nhờ thống trị các cuộc thi Kaggle. So với Gradient Boosting gốc, XGBoost:

- Thêm **chính quy hóa** vào hàm mục tiêu: $\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_m \left(\gamma T_m + \frac{1}{2}\lambda \|\mathbf{w}_m\|^2\right)$, với T_m là số lá và $\mathbf{w}_m$ là giá trị các lá của cây m.

- Dùng **khai triển Taylor bậc hai** (gradient g_i và hessian h_i), cho công thức tối ưu tường minh của giá trị lá $w_j^* = -\frac{\sum_{i\in j} g_i}{\sum_{i\in j} h_i + \lambda}$ và độ lợi khi chia nút:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma. \tag{8.4}$$

- Tự xử lý **giá trị thiếu** (học hướng đi mặc định ở mỗi nút), lấy mẫu con theo hàng và theo cột, tính toán song song và tối ưu bộ nhớ đệm.

Các thư viện cùng họ: **LightGBM** (Microsoft, 2017) và **CatBoost** (Yandex, 2017, xử lý biến phân loại trực tiếp). scikit-learn có `HistGradientBoostingClassifier` với ý tưởng tương tự LightGBM.

Listing 8.3: Gradient Boosting và XGBoost

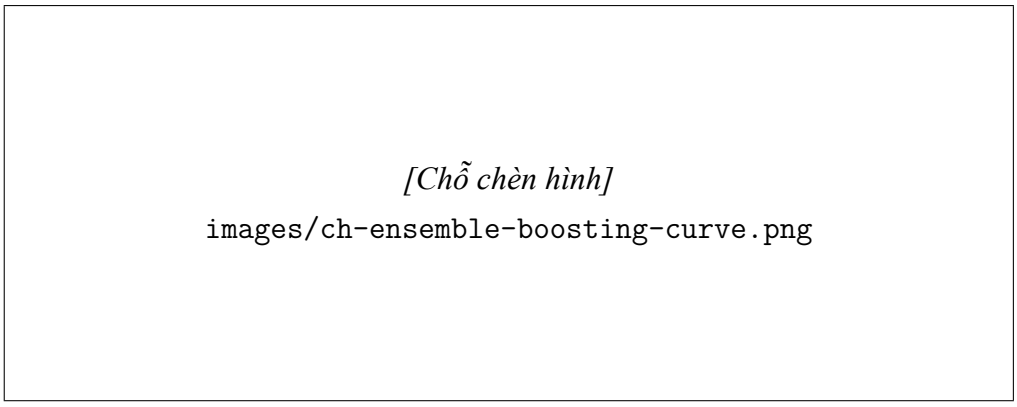
```
from sklearn.ensemble import GradientBoostingClassifier,
   HistGradientBoostingClassifier

gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.05,
                                max_depth=3, subsample=0.8,
                                   random_state=42)
gb.fit(X_train, y_train)
print("GradientBoosting:", gb.score(X_test, y_test))

hgb = HistGradientBoostingClassifier(max_iter=300, learning_rate
   =0.05)
print("HistGB:", hgb.fit(X_train, y_train).score(X_test, y_test))

# pip install xgboost
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=500, learning_rate=0.05, max_depth
   =4,
                    subsample=0.8, colsample_bytree=0.8,
                    reg_lambda=1.0, eval_metric="logloss",
                    early_stopping_rounds=30)
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
print("XGBoost:", xgb.score(X_test, y_test), "so cay:", xgb.
   best_iteration)
```

Lưu ý: Ví dụ trên dùng tập kiểm tra để dừng sớm cho gọn. Trong thực tế cần tách thêm một **tập kiểm định** (validation) riêng; dùng tập kiểm tra để chọn số cây là một dạng rò rỉ dữ liệu (xem chương Đánh giá mô hình).



Hình 8.3: Sai số trên tập huấn luyện và tập kiểm định theo số cây của Gradient Boosting; điểm dừng sớm (early stopping) là nơi sai số kiểm định nhỏ nhất.

8.5 Voting và Stacking

Ngoài việc dùng nhiều phiên bản của cùng một thuật toán, ta có thể kết hợp các thuật toán **khác loại**:

- **Voting**: bỏ phiếu cứng (đa số nhãn) hoặc mềm (trung bình xác suất) giữa các mô hình như Logistic Regression, SVM, Random Forest.
- **Stacking** (Wolpert, 1992): dùng dự đoán của các mô hình tầng 1 làm đầu vào cho một **mô hình tầng 2** (meta-learner); các dự đoán này phải lấy từ kiểm định chéo để tránh rò rỉ.

Listing 8.4: Voting và Stacking

```
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

base = [
    ("lr",  make_pipeline(StandardScaler(), LogisticRegression(
       max_iter=1000))),
    ("svm", make_pipeline(StandardScaler(), SVC(probability=True))),
    ("rf",  RandomForestClassifier(n_estimators=200, random_state=42)
       ),
]
vote = VotingClassifier(base, voting="soft").fit(X_train, y_train)
stack = StackingClassifier(base, final_estimator=LogisticRegression()
   , cv=5)
stack.fit(X_train, y_train)
print("Voting:", vote.score(X_test, y_test), "Stacking:", stack.score
   (X_test, y_test))
```

8.6 Tổng kết so sánh

Phương pháp	Chiến lược	Giảm chủ yếu	Ưu điểm	Nhược điểm
Bagging / Random Forest	Song song	Phương sai	Ổn định, ít cần tinh chỉnh, có OOB	Mô hình lớn, khó diễn giải
AdaBoost	Tuần tự	Độ chệch	Đơn giản, lý thuyết đẹp	Nhạy nhiễu và ngoại lai
Gradient Boosting / XGBoost	Tuần tự	Độ chệch (và phương sai nhờ chính quy hóa)	Độ chính xác cao nhất trên dữ liệu bảng	Nhiều tham số, dễ quá khớp nếu không dừng sớm
Voting / Stacking	Kết hợp khác loại	Cả hai	Tận dụng thế mạnh từng mô hình	Tốn tài nguyên

Bảng 8.2: So sánh các phương pháp học kết hợp

8.7 Tóm tắt chương

- Ensemble dựa trên "trí tuệ đám đông": các mô hình tốt hơn ngẫu nhiên và đa dạng, khi kết hợp sẽ cho kết quả tốt hơn.
- Bagging huấn luyện trên các mẫu bootstrap, giảm phương sai; Random Forest thêm chọn đặc trưng ngẫu nhiên ở mỗi nút để giảm tương quan giữa các cây.
- AdaBoost tăng trọng số các mẫu bị đoán sai; $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$.
- Gradient Boosting khớp từng cây với gradient âm của hàm mất mát; XGBoost bổ sung chính quy hóa, đạo hàm bậc hai và nhiều tối ưu kỹ thuật.

8.8 Bài tập

1. Tính xác suất bỏ phiếu đa số đúng với 5 bộ phân lớp độc lập, mỗi bộ đúng với xác suất 0,6. So sánh với trường hợp 1 bộ phân lớp.
2. Mô phỏng bằng Python: rút 1000 mẫu bootstrap từ $n = 100$ điểm và kiểm chứng tỉ lệ điểm ngoài túi xấp xỉ 36,8%.
3. Làm tiếp ví dụ AdaBoost: ở vòng 2, bộ học yếu đoán sai 2 mẫu vốn được đoán đúng ở vòng 1. Tính ε_2, α_2 và trọng số mới.
4. Trên dữ liệu ung thư vú, vẽ độ chính xác của Random Forest theo `n_estimators` $\in \{1, 5, 10, 50, 100, 300\}$ và theo `max_features`. Nhận xét.
5. So sánh Decision Tree, Random Forest, AdaBoost, Gradient Boosting và XGBoost trên Ames Housing (bài toán hồi quy, dùng các lớp `...Regressor`). Mô hình nào có RMSE thấp nhất?

Tài liệu tham khảo của chương

1. Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24(2), 123–140.
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD '16*, 785–794.
4. Condorcet, M. de (1785). *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. Paris.
5. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26.
6. Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
7. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), 1189–1232.
8. Galton, F. (1907). Vox Populi. *Nature*, 75, 450–451.
9. Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems 30*.
10. Schapire, R. E. (1990). The Strength of Weak Learnability. *Machine Learning*, 5(2), 197–227.
11. Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241–259.

Chương 9: Phân cụm (Clustering)

Các chương trước đều là **học có giám sát**: dữ liệu có sẵn nhãn y. Trong thực tế, phần lớn dữ liệu **không có nhãn** — gán nhãn tốn kém, hoặc ta chưa biết dữ liệu có những nhóm nào. **Phân cụm** (clustering) là bài toán **học không giám sát** (unsupervised learning): tự động chia dữ liệu thành các nhóm (cụm) sao cho các điểm trong cùng cụm *giống nhau* và các điểm ở cụm khác nhau thì *khác nhau*. Ứng dụng tiêu biểu: phân khúc khách hàng, gom nhóm tài liệu, nén ảnh (lượng tử hóa màu), phát hiện bất thường.

Theo kế hoạch giảng dạy, chương này trình bày bốn thuật toán: **K-Means, Hierarchical clustering, DBSCAN, Mean-shift**.
Sau khi học xong chương này, người học có khả năng:

- Phân biệt học có giám sát và không giám sát; nêu các họ thuật toán phân cụm chính.
- Tự chạy tay thuật toán K-Means và DBSCAN trên dữ liệu nhỏ.
- Đọc và cắt cây phân cấp (dendrogram).
- Chọn số cụm và đánh giá chất lượng phân cụm bằng phương pháp khuỷu tay và hệ số Silhouette.

9.1 Bài toán phân cụm

Định nghĩa 9.1 (Phân cụm)**.** *Cho tập dữ liệu $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$* ***không có nhãn****. Phân cụm là tìm một phép gán $c : \{1, \ldots, n\} \to \{1, \ldots, K\}$ (hoặc thêm nhãn "nhiễu") sao cho độ tương tự trong cụm cao và độ tương tự giữa các cụm thấp.*

"Giống nhau" được đo bằng **khoảng cách**, nên mọi lưu ý ở Chương 5 đều áp dụng: phải **chuẩn hóa thang đo** và **mã hóa One Hot** các biến định danh; nếu không, đặc trưng có thang đo lớn sẽ quyết định toàn bộ kết quả phân cụm.

Họ thuật toán	Ý tưởng	Đại diện
Phân hoạch	Chia thành K cụm, tối ưu một hàm mục tiêu	K-Means, K-Medoids
Phân cấp	Xây dựng cây gộp/tách cụm lồng nhau	Agglomerative (single, complete, average, Ward)
Dựa trên mật độ	Cụm là vùng dày đặc, ngăn cách bởi vùng thưa	DBSCAN, OPTICS, Mean-shift
Dựa trên mô hình	Dữ liệu sinh ra từ hỗn hợp phân phối	Gaussian Mixture Model

Bảng 9.1: Các họ thuật toán phân cụm

9.2 K-Means

Thuật toán K-Means được Stuart Lloyd tại Bell Labs đề xuất năm 1957 cho bài toán lượng tử hóa tín hiệu (công bố chính thức năm 1982); tên gọi "k-means" do James MacQueen đặt năm 1967. Đến nay đây vẫn là thuật toán phân cụm được dùng nhiều nhất nhờ đơn giản và nhanh.

Định nghĩa 9.2 (Hàm mục tiêu K-Means)**.** *Cho số cụm K, tìm các tâm cụm $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ và phép gán c để cực tiểu* ***tổng bình phương khoảng cách trong cụm*** *(Within-Cluster Sum of Squares – WCSS, thuộc tính `inertia_` trong scikit-learn):*

$$J = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{c(i)} \right\|^2. \tag{9.1}$$

Thuật toán Lloyd:

1. **Khởi tạo** K tâm cụm (ngẫu nhiên, chọn điểm dữ liệu, hoặc K-Means++).
2. **Bước gán**: gán mỗi điểm vào cụm có tâm gần nhất.
3. **Bước cập nhật**: tâm cụm mới = trung bình cộng các điểm thuộc cụm.
4. Lặp lại bước 2–3 đến khi phép gán không đổi.

Mỗi bước đều không làm tăng J, nên thuật toán luôn hội tụ — nhưng chỉ đến **cực tiểu địa phương**, phụ thuộc vào khởi tạo. Arthur và Vassilvitskii (2007) đề xuất **K-Means++**: chọn tâm đầu tiên ngẫu nhiên, các tâm tiếp theo được chọn với xác suất tỉ lệ với bình phương khoảng cách đến tâm gần nhất đã chọn, giúp các tâm tản ra; đây là khởi tạo mặc định của scikit-learn. Tham số `n_init` chạy nhiều lần khởi tạo và giữ kết quả có J nhỏ nhất.

Ví dụ 9.1 (Chạy tay K-Means)**.** *Cho 5 điểm $P_1(22, 10)$, $P_2(25, 15)$, $P_3(45, 50)$, $P_4(50, 55)$, $P_5(23, 12)$, $K = 2$, khởi tạo tâm $\boldsymbol{\mu}_1 = P_1 = (22, 10)$ và $\boldsymbol{\mu}_2 = P_4 = (50, 55)$.*

Vòng 1 – bước gán:

Điểm	**Tọa độ**	$d(\cdot, \boldsymbol{\mu}_1)$	$d(\cdot, \boldsymbol{\mu}_2)$	**Cụm**
P_1	(22, 10)	0.00	53.00	1
P_2	(25, 15)	5.83	47.17	1
P_3	(45, 50)	46.14	7.07	2
P_4	(50, 55)	53.00	0.00	2
P_5	(23, 12)	2.24	50.77	1

Bảng 9.2: Khoảng cách Euclid đến hai tâm khởi tạo

Vòng 1 – bước cập nhật:

$$\boldsymbol{\mu}_1 = \tfrac{1}{3}\big[(22,10)+(25,15)+(23,12)\big] = (23{,}33;\ 12{,}33),$$
$$\boldsymbol{\mu}_2 = \tfrac{1}{2}\big[(45,50)+(50,55)\big] = (47{,}5;\ 52{,}5).$$

Vòng 2: khoảng cách mới từ $P_1, \ldots, P_5$ đến $\boldsymbol{\mu}_1$ là $(2{,}69;\ 3{,}14;\ 43{,}45;\ 50{,}31;\ 0{,}47)$ và đến $\boldsymbol{\mu}_2$ là $(49{,}56;\ 43{,}73;\ 3{,}54;\ 3{,}54;\ 47{,}33)$. Phép gán không đổi $\{P_1, P_2, P_5\}, \{P_3, P_4\} \Rightarrow$ **hội tụ**. Giá trị hàm mục tiêu $J = 42{,}33$.

Listing 9.1: K-Means với scikit-learn (ví dụ trên lớp)

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[22, 10], [25, 15], [45, 50], [50, 55], [23, 12]])

kmeans = KMeans(n_clusters=2, init=X[[0, 3]], n_init=1)
kmeans.fit(X)
print(kmeans.cluster_centers_)   # [[23.33 12.33] [47.5 52.5]]
print(kmeans.labels_)            # [0 0 1 1 0]
print(kmeans.inertia_)           # 42.33
print(kmeans.predict([[30, 20]]))  # gan diem moi vao cum gan nhat
```

[Chỗ chèn hình]

images/ch-clustering-kmeans-steps.png

Hình 9.1: Các vòng lặp của K-Means trên 5 điểm: khởi tạo tâm, bước gán, bước cập nhật và hội tụ.

9.2.1 Chọn số cụm K

Phương pháp khuỷu tay (elbow). Vẽ J (inertia) theo K. J luôn giảm khi K tăng (bằng 0 khi $K = n$), nhưng đến một điểm thì giảm chậm lại rõ rệt — "khuỷu tay" của đường cong là ứng viên tốt cho K.

Hệ số Silhouette (Rousseeuw, 1987). Với mỗi điểm i: $a(i)$ là khoảng cách trung bình đến các điểm **cùng cụm**, $b(i)$ là khoảng cách trung bình nhỏ nhất đến các điểm của **một cụm khác**:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]. \tag{9.2}$$

$s(i)$ gần 1: điểm nằm đúng cụm và xa cụm khác; gần 0: nằm ở ranh giới; âm: có lẽ bị gán sai cụm. Silhouette trung bình càng cao, phân cụm càng tốt.

Listing 9.2: Chọn K bằng elbow và silhouette

```
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0,
   random_state=42)
ks, inertias, sils = range(2, 10), [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertias.append(km.inertia_)
    sils.append(silhouette_score(X, km.labels_))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot(ks, inertias, "o-"); ax[0].set_title("Elbow (inertia)")
ax[1].plot(ks, sils, "o-");     ax[1].set_title("Silhouette")
plt.show()
```

[Chỗ chèn hình]

images/ch-clustering-elbow.png

Hình 9.2: Đường cong inertia (khuỷu tay tại $K = 4$) và hệ số Silhouette theo số cụm.

Hạn chế của K-Means. Phải chọn trước K; giả định cụm có dạng "cầu" và kích thước tương đương; nhạy với ngoại lai (trung bình bị kéo lệch); chỉ dùng được với khoảng cách Euclid (biến thể K-Medoids dùng được khoảng cách bất kỳ).

9.3 Phân cụm phân cấp (Hierarchical Clustering)

Phân cụm phân cấp bắt nguồn từ ngành phân loại học số (numerical taxonomy) của Sokal và Sneath (1963) và được Johnson (1967) hệ thống hóa. Kết quả không phải một phân hoạch duy nhất mà là một **cây phân cấp** (dendrogram): cắt cây ở độ cao nào sẽ được số cụm tương ứng.

Thuật toán gộp dần (agglomerative, từ dưới lên):

1. Ban đầu mỗi điểm là một cụm.
2. Tìm hai cụm **gần nhau nhất** và gộp chúng lại.
3. Lặp lại đến khi chỉ còn một cụm.

Khoảng cách giữa hai cụm A, B được định nghĩa bởi **tiêu chí liên kết** (linkage):

Liên kết	**Công thức**	**Đặc điểm**
Single	$\min_{a\in A, b\in B} d(a,b)$	Tạo cụm dài, hình dạng tùy ý; dễ bị "hiệu ứng chuỗi"
Complete	$\max_{a\in A, b\in B} d(a,b)$	Cụm gọn, đường kính nhỏ
Average	trung bình mọi cặp $d(a,b)$	Cân bằng giữa hai loại trên
Ward (1963)	mức tăng WCSS khi gộp	Cụm kích thước đều; mặc định của scikit-learn

Bảng 9.3: Các tiêu chí liên kết

Ví dụ 9.2 (Single linkage trên dữ liệu một chiều)**.** *Cho 5 điểm trên trục số:* $1, 2, 4, 7, 11$.

1. *Khoảng cách nhỏ nhất là* $d(1,2) = 1 \Rightarrow$ *gộp* $\{1,2\}$ *ở độ cao 1.*
2. *Khoảng cách từ* $\{1,2\}$ *đến 4 là* $\min(3,2) = 2$ *(nhỏ nhất)* $\Rightarrow$ *gộp* $\{1,2,4\}$ *ở độ cao 2.*
3. *Khoảng cách từ* $\{1,2,4\}$ *đến 7 là 3* $\Rightarrow$ *gộp* $\{1,2,4,7\}$ *ở độ cao 3.*
4. *Cuối cùng gộp với 11 ở độ cao 4.*

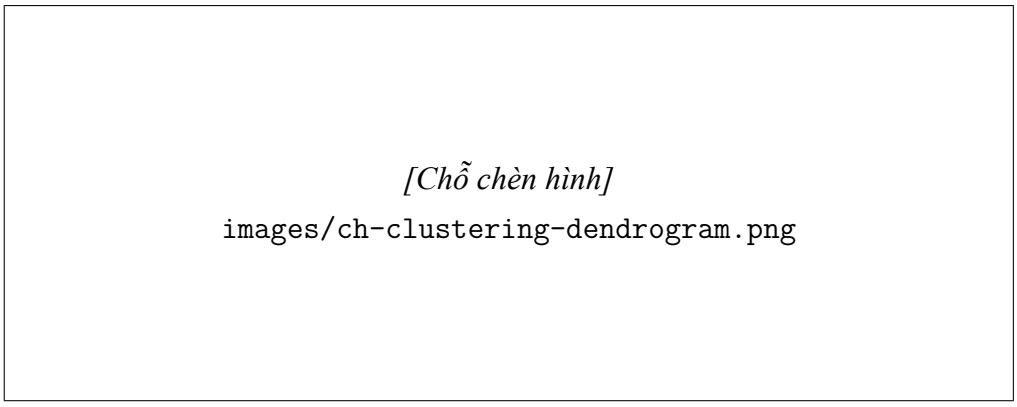
Cắt dendrogram ở độ cao 3,5 được 2 cụm: $\{1,2,4,7\}$ *và* $\{11\}$.

Listing 9.3: Phân cụm phân cấp và vẽ dendrogram

```
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

p = np.array([[1], [2], [4], [7], [11]])
print(AgglomerativeClustering(n_clusters=2, linkage="single")
      .fit_predict(p))                          # [0 0 0 0 1]

Z = linkage(p, method="single")
print(Z)        # moi dong: [cum_a, cum_b, do_cao, so_diem]
dendrogram(Z, labels=["1", "2", "4", "7", "11"])
plt.axhline(3.5, linestyle="--")                # duong cat -> 2 cum
plt.show()
```



Hình 9.3: Dendrogram của 5 điểm với single linkage; đường nét đứt ở độ cao 3,5 cắt cây thành 2 cụm.

Ưu điểm: không cần chọn trước K, kết quả trực quan. Nhược điểm: độ phức tạp tối thiểu $O(n^2)$ bộ nhớ, không phù hợp với dữ liệu rất lớn; quyết định gộp không thể hoàn tác.

9.4 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) do Ester, Kriegel, Sander và Xu công bố năm 1996, được trao giải "Test of Time" của hội nghị KDD năm 2014. DBSCAN định nghĩa cụm là **vùng có mật độ điểm cao**, nhờ đó tìm được cụm có **hình dạng bất kỳ** và tự nhận diện **nhiễu**, không cần biết trước số cụm.

Định nghĩa 9.3 (Các khái niệm của DBSCAN)**.** *Với hai tham số ε (`eps`) và MinPts (`min_samples`):*

- ***ε-lân cận** của p: $N_\varepsilon(p) = \{q : d(p,q) \le \varepsilon\}$ (bao gồm cả p).*
- ***Điểm lõi** (core): $|N_\varepsilon(p)| \ge$ MinPts.*
- ***Điểm biên** (border): không phải điểm lõi nhưng nằm trong ε-lân cận của một điểm lõi.*
- ***Điểm nhiễu** (noise): không phải lõi, cũng không phải biên.*

*Một cụm là tập lớn nhất các điểm **liên thông mật độ**: bắt đầu từ một điểm lõi, lan rộng qua các điểm lõi trong lân cận của nhau, kèm theo các điểm biên của chúng.*

Ví dụ 9.3 (Chạy tay DBSCAN)**.** *Cho 9 điểm (đánh số từ 0 theo Python), $\varepsilon = 2$, MinPts $= 3$:*

Điểm	Tọa độ	ε-lân cận	$\lvert N_\varepsilon\rvert$	Loại
0	(2, 10)	{0}	1	Nhiễu
1	(2, 5)	{1}	1	Nhiễu
2	(8, 4)	{2, 4, 5}	3	**Lõi**
3	(5, 8)	{3, 7}	2	Biên (gần lõi 7)
4	(7, 5)	{2, 4, 5}	3	**Lõi**
5	(6, 4)	{2, 4, 5}	3	**Lõi**
6	(1, 2)	{6}	1	Nhiễu
7	(4, 9)	{3, 7, 8}	3	**Lõi**
8	(3, 7.5)	{7, 8}	2	Biên (gần lõi 7)

Bảng 9.4: Phân loại điểm của DBSCAN ($\varepsilon = 2$, *MinPts* $= 3$)

Ví dụ $d(2,4) = \sqrt{1^2+1^2} = 1{,}41$, $d(2,5) = 2{,}00$, $d(7,3) = 1{,}41$, $d(7,8) = 1{,}80$, còn $d(3,8) = 2{,}06 > 2$. Kết quả: cụm 0 = {2, 4, 5}, cụm 1 = {3, 7, 8} (lõi 7 với hai điểm biên 3 và 8), nhiễu = {0, 1, 6}.

Listing 9.4: DBSCAN với scikit-learn (ví dụ trên lớp)

```
from sklearn.cluster import DBSCAN

X = [[2, 10], [2, 5], [8, 4], [5, 8], [7, 5],
     [6, 4], [1, 2], [4, 9], [3, 7.5]]
dbscan = DBSCAN(eps=2, min_samples=3).fit(X)
print(dbscan.labels_)               # [-1 -1  0  1  0  0 -1  1  1]
   (-1 = nhieu)
print(dbscan.core_sample_indices_)  # [2 4 5 7]
print(dbscan.components_)           # toa do cac diem loi
```

[Chỗ chèn hình]

`images/ch-clustering-dbscan.png`

Hình 9.4: Kết quả DBSCAN trên 9 điểm: điểm lõi, điểm biên, điểm nhiễu và vòng tròn bán kính $\varepsilon = 2$ quanh các điểm lõi.

Chọn tham số. Quy tắc kinh nghiệm: *MinPts* $\geq d + 1$ (thường $2d$); để chọn ε, vẽ **đồ thị k-khoảng cách**: khoảng cách từ mỗi điểm đến láng giềng thứ *MinPts*, sắp xếp tăng dần; điểm "gãy" của đồ thị là ứng viên cho ε. DBSCAN gặp khó khi các cụm có **mật độ rất khác nhau** — khi đó dùng OPTICS (Ankerst và cộng sự, 1999) hoặc HDBSCAN.

9.5 Mean-shift

Mean-shift được Fukunaga và Hostetler đề xuất năm 1975 để ước lượng gradient của hàm mật độ, và được Comaniciu và Meer (2002) phổ biến trong thị giác máy tính (phân đoạn ảnh, theo vết đối tượng). Ý tưởng: coi dữ liệu là mẫu từ một hàm mật độ; mỗi **đỉnh** (mode) của hàm mật độ là một cụm.

Thuật toán: với mỗi điểm khởi đầu $\mathbf{x}$, lặp:

$$\mathbf{x} \leftarrow \frac{\sum_i K(\mathbf{x}_i - \mathbf{x})\, \mathbf{x}_i}{\sum_i K(\mathbf{x}_i - \mathbf{x})}, \tag{9.3}$$

tức dịch $\mathbf{x}$ về **trung bình có trọng số** của các điểm trong cửa sổ (nhân K có độ rộng `bandwidth`). Điểm "leo dốc" dần lên đỉnh mật độ gần nhất; các điểm hội tụ về cùng một đỉnh thuộc cùng một cụm.

Listing 9.5: Mean-shift với ước lượng bandwidth

```
from sklearn.cluster import MeanShift, estimate_bandwidth

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0,
    random_state=42)
bw = estimate_bandwidth(X, quantile=0.2)
ms = MeanShift(bandwidth=bw).fit(X)
print("So cum tim duoc:", len(ms.cluster_centers_))
```

Ưu điểm: không cần chọn số cụm, tìm được cụm hình dạng tùy ý. Nhược điểm: chi phí tính toán lớn và kết quả phụ thuộc nhiều vào `bandwidth`.

9.6 So sánh và ứng dụng

Thuật toán	**Cần biết K**	**Hình dạng cụm**	**Phát hiện nhiễu**	**Tham số chính**	**Quy mô dữ liệu**
K-Means	Có	Cầu, đều nhau	Không	K	Rất lớn
Phân cấp	Không (cắt cây)	Tùy liên kết	Không	linkage, ngưỡng cắt	Nhỏ – vừa
DBSCAN	Không	Tùy ý	Có	`eps`, `min_samples`	Vừa – lớn
Mean-shift	Không	Tùy ý	Không	`bandwidth`	Nhỏ – vừa

Bảng 9.5: So sánh các thuật toán phân cụm

[Chỗ chèn hình]
`images/ch-clustering-compare.png`

Hình 9.5: Kết quả của K-Means, Agglomerative, DBSCAN và Mean-shift trên các tập dữ liệu dạng “mặt trăng”, “vòng tròn” và “khối cầu”.

Listing 9.6: Phân khúc khách hàng: tiền xử lý rồi phân cụm

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# customers: DataFrame cac cot so (tuoi, thu nhap, so lan mua, tong
    chi tieu)
```

```
seg = make_pipeline(StandardScaler(), KMeans(n_clusters=4, n_init=10,
    random_state=0))
labels = seg.fit_predict(customers)
customers["segment"] = labels
print(customers.groupby("segment").mean())   # chan dung tung phan
    khuc

# Truc quan hoa tren 2 chieu bang PCA (xem chuong Giam chieu du lieu)
X2 = PCA(2).fit_transform(StandardScaler().fit_transform(customers.
    drop(columns="segment")))
```

9.7 Tóm tắt chương

- Phân cụm là học không giám sát, dựa trên khoảng cách nên bắt buộc tiền xử lý đúng.
- K-Means lặp hai bước gán – cập nhật để cực tiểu WCSS; dùng K-Means++ và nhiều lần khởi tạo; chọn K bằng elbow và Silhouette.
- Phân cụm phân cấp xây dendrogram bằng cách gộp dần theo tiêu chí liên kết; cắt cây để lấy số cụm.
- DBSCAN tìm cụm theo mật độ, phân loại điểm lõi – biên – nhiễu, tìm được cụm hình dạng tùy ý.
- Mean-shift dịch chuyển điểm về đỉnh mật độ, không cần biết số cụm.

9.8 Bài tập

1. Chạy tay K-Means cho 5 điểm ở ví dụ nhưng khởi tạo tâm là P_1 và P_2. Kết quả cuối có giống ví dụ không? Rút ra nhận xét về vai trò của khởi tạo.
2. Cho 6 điểm 1 chiều: $2, 3, 8, 9, 10, 25$. Vẽ dendrogram với single linkage và complete linkage.
3. Với 9 điểm của ví dụ DBSCAN, chạy lại với $\varepsilon = 3$, *MinPts* $= 3$. Lập bảng phân loại điểm như Bảng 9.4.
4. Sinh dữ liệu `make_moons(n_samples=300, noise=0.05)`, so sánh K-Means ($K = 2$) và DBSCAN. Giải thích sự khác biệt.
5. Lượng tử hóa màu: dùng K-Means với $K \in \{4, 16, 64\}$ để nén màu một bức ảnh (mỗi pixel là một điểm trong không gian RGB). So sánh ảnh kết quả.
6. Phân cụm các căn nhà trong Ames Housing theo `Gr Liv Area`, `Overall Qual`, `Year Built` (đã chuẩn hóa). Mô tả đặc điểm và giá trung bình của từng cụm.

Tài liệu tham khảo của chương

1. Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering Points To Identify the Clustering Structure. *Proceedings of ACM SIGMOD '99*, 49–60.

2. Arthur, D., & Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding. *Proceedings of SODA '07*, 1027–1035.

3. Comaniciu, D., & Meer, P. (2002). Mean Shift: A Robust Approach Toward Feature Space Analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5), 603–619.

4. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. *Proceedings of KDD '96*, 226–231.

5. Fukunaga, K., & Hostetler, L. (1975). The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on Information Theory*, 21(1), 32–40.

6. Johnson, S. C. (1967). Hierarchical clustering schemes. *Psychometrika*, 32(3), 241–254.

7. Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

8. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.

9. Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

10. Ward, J. H. (1963). Hierarchical Grouping to Optimize an Objective Function. *Journal of the American Statistical Association*, 58(301), 236–244.

Chương 10: Giảm chiều dữ liệu (Dimensionality Reduction)

Dữ liệu thực tế thường có rất nhiều đặc trưng: một ảnh 28×28 có 784 chiều, một văn bản sau khi qua TF-IDF (Chương 5) có hàng chục nghìn chiều, dữ liệu gen có hàng triệu chiều. Số chiều lớn gây ra nhiều khó khăn mà Richard Bellman (1957) gọi chung là **lời nguyền số chiều** (curse of dimensionality). **Giảm chiều dữ liệu** là quá trình biểu diễn dữ liệu bằng ít chiều hơn nhưng giữ lại phần lớn thông tin.

Theo kế hoạch giảng dạy, chương này tập trung vào hai kỹ thuật nền tảng: **SVD** (phân tích giá trị suy biến) và **PCA** (phân tích thành phần chính).
Sau khi học xong chương này, người học có khả năng:

- Giải thích lời nguyền số chiều và phân biệt lựa chọn đặc trưng với trích xuất đặc trưng.
- Tính tay phương sai, hiệp phương sai, hệ số tương quan và cặp trị riêng – vector riêng của ma trận 2×2.
- Tính tay SVD của ma trận nhỏ và xấp xỉ hạng thấp.
- Tính tay PCA (chuẩn hóa tâm, ma trận hiệp phương sai, trị riêng, chiếu dữ liệu).
- Dùng `PCA`, `TruncatedSVD` của scikit-learn và chọn số thành phần theo phương sai giải thích.

10.1 Lời nguyền số chiều

Khi số chiều d tăng:

- **Dữ liệu trở nên thưa**: để phủ đều khối lập phương đơn vị với mật độ 10 điểm mỗi trục cần 10^d điểm — 100 điểm cho $d = 2$, nhưng 10^{10} điểm cho $d = 10$.
- **Khoảng cách mất ý nghĩa**: trong không gian nhiều chiều, khoảng cách từ một điểm đến láng giềng gần nhất và xa nhất gần bằng nhau (Beyer và cộng sự, 1999), khiến KNN và K-Means kém hiệu quả.
- **Quá khớp**: số tham số tăng theo d, mô hình dễ học thuộc nhiễu khi số mẫu không đủ.
- **Chi phí** lưu trữ và tính toán tăng; không thể trực quan hóa trên 3 chiều.

Hướng tiếp cận	Ý tưởng	Ví dụ
Lựa chọn đặc trưng (feature selection)	Giữ lại một tập con các cột gốc	Lọc theo phương sai, tương quan, Lasso, `SelectKBest`
Trích xuất đặc trưng tuyến tính	Tạo cột mới là tổ hợp tuyến tính các cột gốc	**PCA**, **SVD**, LDA
Trích xuất phi tuyến (manifold learning)	Bảo toàn cấu trúc lân cận cục bộ	Kernel PCA, t-SNE, UMAP, Autoencoder

Bảng 10.1: Các hướng giảm chiều dữ liệu

10.2 Kiến thức nền: phương sai, hiệp phương sai và trị riêng

Cả SVD và PCA đều được xây dựng trên ba khái niệm: phương sai, hiệp phương sai và cặp trị riêng – vector riêng. Mục này ôn lại chúng qua các ví dụ tính tay.

10.2.1 Phương sai

Định nghĩa 10.1 (Phương sai)**.** *Phương sai (variance) đo mức độ phân tán của dữ liệu quanh giá trị trung bình:*

$$\mathrm{Var}(A) = \frac{1}{n}\sum_{i=1}^{n}(a_i - \bar{a})^2. \tag{10.1}$$

Khi ước lượng phương sai của tổng thể từ một mẫu, người ta thường chia cho $n-1$ thay vì n (hiệu chỉnh Bessel) để được ước lượng không chệch. Độ lệch chuẩn σ là căn bậc hai của phương sai.

Ví dụ 10.1 (Tính phương sai)**.** $A = (3, 7, 5)$ *có* $\bar{a} = \frac{3+7+5}{3} = 5$ *và*

$$\mathrm{Var}(A) = \frac{(3-5)^2 + (7-5)^2 + (5-5)^2}{3} = \frac{8}{3} \approx 2{,}67.$$

Nếu chia cho $n-1$ *thì được* $\frac{8}{2} = 4$.

Lưu ý khi lập trình: `np.var` mặc định chia cho n (`ddof=0`), còn `pandas.Series.var` và `np.cov` mặc định chia cho $n-1$ (`ddof=1`). Với mẫu nhỏ, hai quy ước cho kết quả khác nhau đáng kể, nên cần ghi rõ quy ước đang dùng.

10.2.2 Hiệp phương sai và hệ số tương quan

Định nghĩa 10.2 (Hiệp phương sai và hệ số tương quan)**.** *Hiệp phương sai (covariance) đo mức độ* ***cùng biến thiên*** *của hai biến:*

$$\mathrm{Cov}(A, B) = \frac{1}{n}\sum_{i=1}^{n}(a_i - \bar{a})(b_i - \bar{b}). \tag{10.2}$$

$\mathrm{Cov} > 0$*: hai biến có xu hướng cùng tăng;* $\mathrm{Cov} < 0$*: biến này tăng thì biến kia giảm;* $\mathrm{Cov} \approx 0$*: không có quan hệ tuyến tính. Vì độ lớn của hiệp phương sai phụ thuộc đơn vị đo, Karl Pearson (1896) chuẩn hóa nó thành* ***hệ số tương quan***

$$r = \frac{\mathrm{Cov}(A, B)}{\sigma_A\,\sigma_B} \in [-1, 1]. \tag{10.3}$$

Ví dụ 10.2 (Tính hiệp phương sai và hệ số tương quan)**.** *$A = (2, 8, 4)$ và $B = (6, 1, 5)$ có $\bar{a} = \frac{14}{3} \approx 4{,}67$, $\bar{b} = \frac{12}{3} = 4$:*

$$\begin{aligned}\mathrm{Cov}(A,B) &= \tfrac{1}{3}\big[(2-4{,}67)(6-4) + (8-4{,}67)(1-4) + (4-4{,}67)(5-4)\big] \\ &= \tfrac{1}{3}\,(-5{,}33 - 10 - 0{,}67) = -\tfrac{16}{3} \approx -5{,}33.\end{aligned}$$

*Dấu **âm** cho biết khi A tăng thì B giảm: quan sát thứ hai có A lớn nhất nhưng B nhỏ nhất. Với $\sigma_A^2 = \frac{56}{9} \approx 6{,}22$ và $\sigma_B^2 = \frac{14}{3} \approx 4{,}67$:*

$$r = \frac{-5{,}33}{\sqrt{6{,}22 \times 4{,}67}} \approx -0{,}99,$$

*tức hai biến có quan hệ tuyến tính nghịch rất mạnh. Khi tính tay cần giữ đúng **dấu** của từng tích $(a_i - \bar{a})(b_i - \bar{b})$: bỏ sót dấu âm sẽ đảo ngược hoàn toàn ý nghĩa của kết quả.*

Ma trận hiệp phương sai. Với dữ liệu d chiều, các phương sai và hiệp phương sai được gom thành ma trận đối xứng $\mathbf{C} \in \mathbb{R}^{d\times d}$ với $C_{jk} = \mathrm{Cov}(x_j, x_k)$; đường chéo chính là phương sai của từng biến. Nếu $\mathbf{X}_c$ là dữ liệu đã trừ trung bình từng cột thì $\mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c$ — quy ước chia $n-1$ mà PCA của scikit-learn sử dụng.

10.2.3 Trị riêng và vector riêng

Định nghĩa 10.3 (Trị riêng, vector riêng)**.** *Cho ma trận vuông $\mathbf{A}$. Số λ và vector $\mathbf{v} \neq \mathbf{0}$ thỏa $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ được gọi là **trị riêng** và **vector riêng** tương ứng: nhân với $\mathbf{A}$ chỉ co giãn $\mathbf{v}$ theo hệ số λ mà không làm đổi phương của nó. Các trị riêng là nghiệm của **phương trình đặc trưng** $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.*

Ví dụ 10.3 (Tính tay trị riêng và vector riêng)**.** *Cho* $\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}$.

Bước 1 – phương trình đặc trưng:

$$\det\begin{pmatrix} 2-\lambda & 3 \\ 4 & 1-\lambda \end{pmatrix} = (2-\lambda)(1-\lambda) - 3\cdot 4 = \lambda^2 - 3\lambda - 10 = 0 \;\Leftrightarrow\; \lambda_1 = 5,\ \lambda_2 = -2.$$

Bước 2 – vector riêng ứng với $\lambda_1 = 5$: giải $(\mathbf{A} - 5\mathbf{I})\mathbf{v} = \mathbf{0}$,

$$\begin{pmatrix} -3 & 3 \\ 4 & -4 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; -v_1 + v_2 = 0 \;\Leftrightarrow\; v_1 = v_2.$$

Chọn $v_2 = 1$ thì $v_1 = 1$; chuẩn hóa về độ dài 1: $\mathbf{v}_1 = \frac{1}{\sqrt{1^2+1^2}}(1,1) \approx (0{,}7071;\ 0{,}7071)$.
Bước 3 – vector riêng ứng với $\lambda_2 = -2$:

$$\begin{pmatrix} 4 & 3 \\ 4 & 3 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; 4v_1 + 3v_2 = 0 \;\Leftrightarrow\; v_1 = -\tfrac{3}{4}v_2.$$

Chọn $v_2 = 4$ thì $v_1 = -3$; chuẩn hóa: $\mathbf{v}_2 = \frac{1}{\sqrt{(-3)^2+4^2}}(-3,4) = (-0{,}6;\ 0{,}8)$.
Kiểm tra nhanh: tổng các trị riêng bằng **vết** (tổng đường chéo) của ma trận, $5 + (-2) = 2 + 1$; tích các trị riêng bằng **định thức**, $5 \times (-2) = 2\cdot 1 - 3\cdot 4 = -10$.

Ma trận $\mathbf{A}$ ở ví dụ này **không đối xứng** nên hai vector riêng không vuông góc ($\mathbf{v}_1 \cdot \mathbf{v}_2 \approx 0{,}14 \neq 0$). Ngược lại, với ma trận **đối xứng** như $\mathbf{X}^\top\mathbf{X}$ hay ma trận hiệp phương sai, **định lý phổ** bảo đảm mọi trị riêng là số thực và có thể chọn các vector riêng đôi một vuông góc — đây chính là tính chất mà SVD và PCA khai thác.

Listing 10.1: Phương sai, hiệp phương sai và trị riêng với NumPy

```
import numpy as np

A = np.array([3, 7, 5])
print(np.var(A), np.var(A, ddof=1))        # 2.6667 4.0
a, b = np.array([2, 8, 4]), np.array([6, 1, 5])
print(np.cov(a, b, ddof=0)[0, 1])          # -5.3333
print(np.corrcoef(a, b)[0, 1])             # -0.9897

M = np.array([[2, 3], [4, 1]])
eigval, eigvec = np.linalg.eig(M)   # NumPy moi co the tra ve kieu so
    phuc (5.+0.j)
print(eigval.real)     # [ 5. -2.]
print(eigvec.real)     # moi cot la mot vector rieng: (0.7071,
   0.7071) va (-0.6, 0.8)
```

10.3 Phân tích giá trị suy biến (SVD)

10.3.1 Lịch sử

SVD được Eugenio Beltrami (1873) và Camille Jordan (1874) phát hiện độc lập; Eckart và Young (1936) chứng minh tính chất xấp xỉ hạng thấp tối ưu — nền tảng của mọi ứng dụng nén dữ liệu. Thuật toán tính SVD ổn định số do Golub và Kahan (1965) đề xuất là cái mà NumPy dùng ngày nay. SVD còn nổi tiếng nhờ cuộc thi Netflix Prize (2006–2009), nơi các biến thể phân rã ma trận chiếm ưu thế trong hệ gợi ý.

Định lý 10.1 (Phân tích giá trị suy biến)**.** *Mọi ma trận thực* $\mathbf{A} \in \mathbb{R}^{m\times n}$ *đều phân tích được thành*

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\top}, \tag{10.4}$$

trong đó $\mathbf{U} \in \mathbb{R}^{m\times m}$ *và* $\mathbf{V} \in \mathbb{R}^{n\times n}$ *là các ma trận trực giao (cột là vector suy biến trái/phải),* $\boldsymbol{\Sigma} \in \mathbb{R}^{m\times n}$ *là ma trận đường chéo với các* ***giá trị suy biến*** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$*. Ngoài ra* σ_i^2 *là trị riêng của* $\mathbf{A}\mathbf{A}^{\top}$ *(và của* $\mathbf{A}^{\top}\mathbf{A}$*), cột của* $\mathbf{U}$ *là vector riêng của* $\mathbf{A}\mathbf{A}^{\top}$*, cột của* $\mathbf{V}$ *là vector riêng của* $\mathbf{A}^{\top}\mathbf{A}$*.*

Định lý 10.2 (Eckart–Young)**.** *Giữ lại* k *giá trị suy biến lớn nhất,* $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\top}$ *là ma trận hạng* k ***gần*** $\mathbf{A}$ ***nhất*** *theo chuẩn Frobenius, với sai số* $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}$*.*

Ví dụ 10.4 (Tính tay SVD)**.** *Cho* $\mathbf{M} = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}$.

Bước 1. $\mathbf{M}\mathbf{M}^{\top} = \begin{pmatrix} 17 & 8 \\ 8 & 17 \end{pmatrix}$, phương trình đặc trưng $(17-\lambda)^2 - 64 = 0$ cho $\lambda_1 = 25$, $\lambda_2 = 9$. Vậy $\sigma_1 = 5$, $\sigma_2 = 3$.
Bước 2. Vector riêng của $\mathbf{M}\mathbf{M}^{\top}$: $\mathbf{u}_1 = \frac{1}{\sqrt{2}}(1,1)^{\top}$, $\mathbf{u}_2 = \frac{1}{\sqrt{2}}(1,-1)^{\top}$.
Bước 3. $\mathbf{v}_i = \mathbf{M}^{\top}\mathbf{u}_i/\sigma_i$:

$$\mathbf{v}_1 = \tfrac{1}{5\sqrt{2}}(5,5,0)^{\top} = \tfrac{1}{\sqrt{2}}(1,1,0)^{\top}, \qquad \mathbf{v}_2 = \tfrac{1}{3\sqrt{2}}(1,-1,4)^{\top} = \tfrac{1}{\sqrt{18}}(1,-1,4)^{\top}.$$

Vector thứ ba $\mathbf{v}_3 = \frac{1}{3}(2,-2,-1)^{\top}$ vuông góc với hai vector trên (ứng với giá trị suy biến 0).

Bước 4 – xấp xỉ hạng 1. $\mathbf{M}_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 5 \cdot \frac{1}{2}\begin{pmatrix}1 & 1 & 0\\ 1 & 1 & 0\end{pmatrix} = \begin{pmatrix}2{,}5 & 2{,}5 & 0\\ 2{,}5 & 2{,}5 & 0\end{pmatrix}$, sai số $\|\mathbf{M} - \mathbf{M}_1\|_F = \sigma_2 = 3$. Thành phần thứ nhất giữ $\sigma_1^2/(\sigma_1^2+\sigma_2^2) = 25/34 \approx 73{,}5\%$ "năng lượng" của ma trận.

Listing 10.2: SVD với NumPy (ví dụ trên lớp)

```
import numpy as np

M = np.array([[3, 2, 2], [2, 3, -2]])
U, s, Vt = np.linalg.svd(M)
print(s)          # [5. 3.]
print(U)          # cot = vector suy bien trai (co the doi dau)
print(Vt)         # dong = vector suy bien phai

# Tai tao lai M tu 3 thanh phan
Sigma = np.zeros(M.shape); Sigma[:2, :2] = np.diag(s)
print(np.allclose(U @ Sigma @ Vt, M))   # True

# Xap xi hang 1
M1 = s[0] * np.outer(U[:, 0], Vt[0])
print(M1.round(2))                       # [[2.5 2.5 0.] [2.5 2.5
    0.]]
print(np.linalg.norm(M - M1))            # 3.0 = sigma_2
```

Lưu ý: vector suy biến chỉ xác định duy nhất đến **dấu**: NumPy có thể trả về $-\mathbf{u}_1, -\mathbf{v}_1$, kết quả vẫn đúng.

10.3.2 Tính SVD thông qua $\mathbf{X}^\top\mathbf{X}$

Định lý trên gợi ý cách tính SVD bằng tay cho ma trận có nhiều dòng: (1) lập $\mathbf{X}^\top\mathbf{X}$ — ma trận vuông, đối xứng, kích thước bằng số cột; (2) tìm các trị riêng λ_i và vector riêng đơn vị $\mathbf{v}_i$ của nó; (3) giá trị suy biến $\sigma_i = \sqrt{\lambda_i}$; (4) vector suy biến trái $\mathbf{u}_i = \mathbf{X}\mathbf{v}_i/\sigma_i$.

Ví dụ 10.5 (SVD của ma trận 5×2)**.** *Cho* $\mathbf{X} = \begin{pmatrix}3 & 7\\ 1 & 5\\ 6 & 4\\ 8 & 1\\ 2 & 3\end{pmatrix}$.

Bước 1 – lập $\mathbf{X}^\top\mathbf{X}$:

$$\mathbf{X}^\top\mathbf{X} = \begin{pmatrix}3 & 1 & 6 & 8 & 2\\ 7 & 5 & 4 & 1 & 3\end{pmatrix}\begin{pmatrix}3 & 7\\ 1 & 5\\ 6 & 4\\ 8 & 1\\ 2 & 3\end{pmatrix} = \begin{pmatrix}114 & 64\\ 64 & 100\end{pmatrix},$$

trong đó $114 = 3^2+1^2+6^2+8^2+2^2$ và $64 = 3\cdot 7 + 1\cdot 5 + 6\cdot 4 + 8\cdot 1 + 2\cdot 3$.
Bước 2 – trị riêng:

$$(114-\lambda)(100-\lambda) - 64^2 = \lambda^2 - 214\lambda + 7304 = 0 \;\Leftrightarrow\; \lambda_1 \approx 171{,}38,\ \lambda_2 \approx 42{,}62.$$

Bước 3 – vector riêng. Với λ_1: $(114 - 171{,}38)\,v_1 + 64\,v_2 = 0 \Leftrightarrow v_1 = 1{,}1153\,v_2$. Chọn $v_2 = 1$ được $(1{,}1153;\ 1)$, chuẩn hóa cho $\mathbf{v}_1 = (1{,}1153;\ 1)/1{,}4980 \approx (0{,}7446;\ 0{,}6676)$. Với λ_2: $(114 - 42{,}62)\,v_1 + 64\,v_2 = 0 \Leftrightarrow v_1 = -0{,}8966\,v_2$. Chọn $v_2 = 1$ được $(-0{,}8966;\ 1)$, chuẩn hóa cho $\mathbf{v}_2 \approx (-0{,}6676;\ 0{,}7446)$. Hai vector vuông góc: $0{,}7446 \times (-0{,}6676) + 0{,}6676 \times 0{,}7446 = 0$. **Bước 4 – giá trị và vector suy biến:** $\sigma_1 = \sqrt{171{,}38} \approx 13{,}09$, $\sigma_2 = \sqrt{42{,}62} \approx 6{,}53$, và $\mathbf{u}_1 = \mathbf{X}\mathbf{v}_1/\sigma_1 \approx (0{,}528;\ 0{,}312;\ 0{,}545;\ 0{,}506;\ 0{,}267)$.

Sai lầm thường gặp khi chuẩn hóa vector riêng. Từ $v_1 = -0{,}8966\,v_2$, nếu chọn $v_2 = 1$ thì vector là $(v_1, v_2) = (-0{,}8966;\ 1)$ — **không phải** $(1;\ -0{,}8966)$. Đảo chỗ hai thành phần cho ra một vector khác hẳn, không còn là vector riêng. Cách tự kiểm tra đơn giản nhất là nhân lại: $\mathbf{X}^\top\mathbf{X}\,\mathbf{v}$ phải bằng $\lambda\mathbf{v}$, và vector sau khi chuẩn hóa phải có độ dài đúng bằng 1.

Listing 10.3: Kiểm tra cách tính SVD qua trị riêng

```
X = np.array([[3, 7], [1, 5], [6, 4], [8, 1], [2, 3]])
lam, V = np.linalg.eigh(X.T @ X)      # tri rieng tang dan: [ 42.6183
    171.3817]
print(np.sqrt(lam[::-1]))             # [13.0913  6.5283]
U, s, Vt = np.linalg.svd(X, full_matrices=False)
print(s)                              # [13.0913  6.5283] -- trung
    khop
print(Vt)                             # moi dong la mot v_i (co the
    doi dau)
```

10.3.3 Ứng dụng: nén ảnh và TruncatedSVD

Một ảnh xám là ma trận $m \times n$. Lưu $\mathbf{A}_k$ chỉ cần $k(m+n+1)$ số thay vì mn. Với ảnh 512×512 và $k = 50$: $50 \times 1025 = 51\,250$ số so với $262\,144$ — tiết kiệm khoảng 80%.

Listing 10.4: Nén ảnh bằng SVD

```
import matplotlib.pyplot as plt
from skimage import data, color

img = color.rgb2gray(data.astronaut())       # anh xam 512 x 512
U, s, Vt = np.linalg.svd(img, full_matrices=False)
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, k in zip(axes, [5, 20, 50, 512]):
    ax.imshow(U[:, :k] @ np.diag(s[:k]) @ Vt[:k], cmap="gray")
    ax.set_title(f"k = {k}"); ax.axis("off")
plt.show()
```

[Chỗ chèn hình]
images/ch-reduce-svd-image.png

Hình 10.1: Ảnh tái tạo từ $k = 5, 20, 50$ giá trị suy biến lớn nhất so với ảnh gốc.

Với ma trận thưa như kết quả TF-IDF, scikit-learn cung cấp `TruncatedSVD`: tính trực tiếp k thành phần đầu mà không cần trừ trung bình (trừ trung bình sẽ phá vỡ tính thưa). Áp dụng lên ma trận văn bản – từ, kỹ thuật này được gọi là **Phân tích ngữ nghĩa tiềm ẩn** (Latent Semantic Analysis – LSA, Deerwester và cộng sự, 1990): mỗi thành phần tương ứng một "chủ đề".

Listing 10.5: LSA: TF-IDF + TruncatedSVD

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

docs = ["python machine learning", "python data analysis",
        "deep learning neural network", "neural network training",
        "data analysis with pandas"]
X = TfidfVectorizer().fit_transform(docs)          # ma tran thua 5 x V
lsa = TruncatedSVD(n_components=2, random_state=0)
Z = lsa.fit_transform(X)                           # 5 x 2
print(lsa.explained_variance_ratio_)
print(Z.round(3))   # van ban cung chu de nam gan nhau
```

10.4 Phân tích thành phần chính (PCA)

10.4.1 Lịch sử và ý tưởng

PCA do Karl Pearson phát minh năm 1901 dưới dạng bài toán "tìm đường thẳng và mặt phẳng khớp nhất với hệ điểm", và được Harold Hotelling (1933) phát triển độc lập, đặt tên "thành phần chính". Trong xử lý tín hiệu, PCA còn được gọi là biến đổi Karhunen–Loève.

Định nghĩa 10.4 (Thành phần chính)**.** *Cho dữ liệu đã chuẩn hóa tâm* $\mathbf{X}_c$ *(*$n \times d$*).* ***Thành phần chính thứ nhất*** *là vector đơn vị* $\mathbf{w}_1$ *sao cho phương sai của dữ liệu chiếu lên* $\mathbf{w}_1$ *là* ***lớn nhất****:*

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \mathbf{w}^\top \mathbf{C}\mathbf{w}, \qquad \mathbf{C} = \tfrac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c. \tag{10.5}$$

Các thành phần tiếp theo cũng cực đại phương sai nhưng phải ***vuông góc*** *với các thành phần trước.*

Giải bài toán tối ưu bằng nhân tử Lagrange cho $\mathbf{Cw} = \lambda\mathbf{w}$: các thành phần chính là **vector riêng** của ma trận hiệp phương sai, và phương sai theo thành phần thứ j chính là **trị riêng** λ_j. Tương đương, cực đại phương sai cũng là cực tiểu sai số tái tạo (khoảng cách bình phương từ điểm đến không gian con) — đúng cách nhìn ban đầu của Pearson.

Quan hệ PCA – SVD. Nếu $\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ thì $\mathbf{C} = \mathbf{V}\frac{\mathbf{\Sigma}^2}{n-1}\mathbf{V}^\top$: thành phần chính là cột của $\mathbf{V}$, trị riêng $\lambda_j = \sigma_j^2/(n-1)$. scikit-learn tính PCA bằng SVD vì ổn định số hơn việc lập ma trận hiệp phương sai.

Các bước PCA:

1. Chuẩn hóa tâm (trừ trung bình mỗi cột); nếu các cột khác đơn vị đo thì chuẩn hóa thêm bằng `StandardScaler`.
2. Tính ma trận hiệp phương sai $\mathbf{C}$.
3. Tìm trị riêng, vector riêng; sắp xếp trị riêng giảm dần.
4. Chọn k vector riêng đầu làm ma trận $\mathbf{W}_k$ ($d \times k$).
5. Chiếu dữ liệu: $\mathbf{Z} = \mathbf{X}_c\mathbf{W}_k$ ($n \times k$).

Ví dụ 10.6 (Tính tay PCA)**.** *Giảm 5 điểm hai chiều trong Bảng 10.2 xuống còn một chiều.*

x_1	3	5	0	1	2
x_2	6	4	2	0	3

Bảng 10.2: Dữ liệu ví dụ PCA

Bước 1 – chuẩn hóa tâm. $\bar{x}_1 = \frac{3+5+0+1+2}{5} = 2{,}2$ và $\bar{x}_2 = \frac{6+4+2+0+3}{5} = 3$. Trừ trung bình từng cột, $x'_{ij} = x_{ij} - \bar{x}_j$:

$$\mathbf{X}_c = \begin{pmatrix} 0{,}8 & 3 \\ 2{,}8 & 1 \\ -2{,}2 & -1 \\ -1{,}2 & -3 \\ -0{,}2 & 0 \end{pmatrix}.$$

Bước 2 – ma trận hiệp phương sai:

$$\mathbf{X}_c^\top\mathbf{X}_c = \begin{pmatrix} 14{,}8 & 11 \\ 11 & 20 \end{pmatrix}, \qquad \mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^\top\mathbf{X}_c = \frac{1}{4}\begin{pmatrix} 14{,}8 & 11 \\ 11 & 20 \end{pmatrix} = \begin{pmatrix} 3{,}7 & 2{,}75 \\ 2{,}75 & 5 \end{pmatrix}.$$

Hiệp phương sai dương ($2{,}75$) cho thấy hai biến có xu hướng tăng cùng nhau.

Bước 3 – trị riêng:

$$(3{,}7-\lambda)(5-\lambda) - 2{,}75^2 = \lambda^2 - 8{,}7\lambda + 10{,}9375 = 0 \;\Leftrightarrow\; \lambda_1 \approx 7{,}1758,\ \lambda_2 \approx 1{,}5242.$$

Bước 4 – vector riêng. Với $\lambda_1 = 7{,}1758$:

$$\begin{pmatrix} -3{,}4758 & 2{,}75 \\ 2{,}75 & -2{,}1758 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; v_1 = 0{,}7912\, v_2.$$

Chọn $v_2 = 1$ được $(v_1, v_2) = (0{,}7912;\ 1)$; chuẩn hóa: $\mathbf{w}_1 = \dfrac{(0{,}7912;\ 1)}{\sqrt{0{,}7912^2 + 1^2}} = \dfrac{(0{,}7912;\ 1)}{1{,}2751} \approx (0{,}6205;\ 0{,}7842)$.

Với $\lambda_2 = 1{,}5242$: $2{,}1758\, v_1 + 2{,}75\, v_2 = 0 \Leftrightarrow v_1 = -1{,}2639\, v_2$; chọn $v_2 = 1$ được $(-1{,}2639;\ 1)$, chuẩn hóa cho $\mathbf{w}_2 \approx (-0{,}7842;\ 0{,}6205)$.

Kiểm tra: $\mathbf{C}\mathbf{w}_1 = (3{,}7 \times 0{,}6205 + 2{,}75 \times 0{,}7842;\ \ 2{,}75 \times 0{,}6205 + 5 \times 0{,}7842) \approx (4{,}452;\ 5{,}627) \approx 7{,}1758 \times (0{,}6205;\ 0{,}7842)$.

Bước 5 – tỉ lệ phương sai giải thích: $\frac{7{,}1758}{7{,}1758 + 1{,}5242} = \frac{7{,}1758}{8{,}7} \approx 82{,}5\%$; thành phần thứ nhất giữ khoảng 82,5% thông tin.

Bước 6 – chiếu dữ liệu $\mathbf{z} = \mathbf{X}_c\, \mathbf{w}_1$. Chẳng hạn với điểm đầu tiên: $z_1 = 0{,}8 \times 0{,}6205 + 3 \times 0{,}7842 \approx 2{,}849$. Toàn bộ:

$$\mathbf{z} \approx (2{,}849;\ 2{,}522;\ -2{,}149;\ -3{,}097;\ -0{,}124).$$

Mỗi điểm hai chiều giờ chỉ còn một con số; có thể tái tạo gần đúng điểm gốc bằng $\hat{\mathbf{x}} = \bar{\mathbf{x}} + z\, \mathbf{w}_1$.

Sai lầm thường gặp. (1) Từ $v_1 = 0{,}7912\, v_2$ mà viết vector thành $(1;\ 0{,}7912)$ là **đảo nhầm thành phần**: vector chuẩn hóa thu được, $(0{,}7842;\ 0{,}6205)$, không còn là vector riêng của $\mathbf{C}$. Hãy luôn kiểm tra lại $\mathbf{C}\mathbf{w} = \lambda\mathbf{w}$ như ở bước 4. (2) Quên chuẩn hóa tâm trước khi lập ma trận hiệp phương sai: khi đó ta đang tính SVD của dữ liệu gốc (như ví dụ 5×2 ở mục SVD) chứ không phải PCA, và thành phần chính đầu tiên bị kéo lệch về phía vector trung bình của dữ liệu.

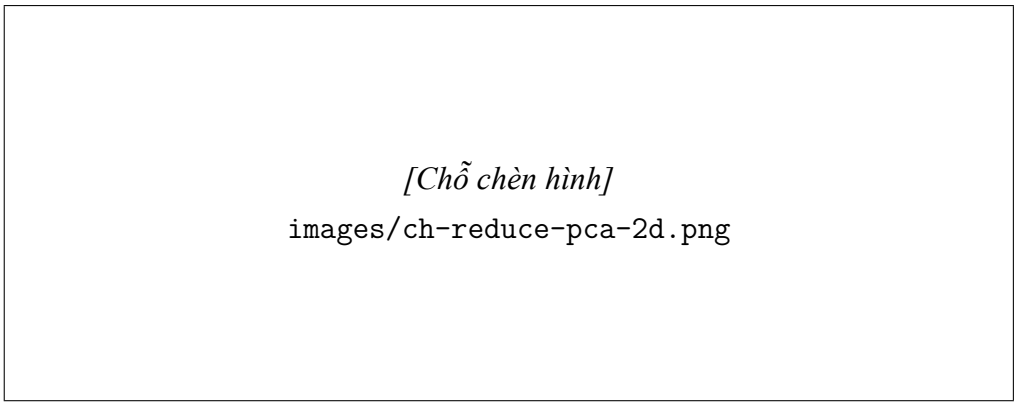
Listing 10.6: Kiểm tra ví dụ bằng NumPy và scikit-learn

```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[3, 6], [5, 4], [0, 2], [1, 0], [2, 3]])

# Cach 1: tu cai dat
Xc = X - X.mean(axis=0)
C = np.cov(Xc, rowvar=False)                # chia n - 1: [[3.7 2.75]
    [2.75 5.]]
eigval, eigvec = np.linalg.eigh(C)          # tri rieng tang dan
order = np.argsort(eigval)[::-1]
eigval, eigvec = eigval[order], eigvec[:, order]
print(eigval)                               # [7.1758 1.5242]
print(Xc @ eigvec[:, 0])                    # toa do tren PC1 (co the
    doi dau)

# Cach 2: scikit-learn
pca = PCA(n_components=1).fit(X)
print(pca.components_)                      # [[0.6205 0.7842]]
print(pca.explained_variance_ratio_)        # [0.8248]
Z = pca.transform(X)                        # [2.8491 2.5216 -2.1493
   -3.0973 -0.1241]
X_rec = pca.inverse_transform(Z)            # tai tao (nam tren duong
    PC1)
```



Hình 10.2: Dữ liệu ví dụ với hai trục thành phần chính $\mathbf{w}_1$, $\mathbf{w}_2$ và hình chiếu của các điểm lên $\mathbf{w}_1$.

10.4.2 Chọn số thành phần

- **Ngưỡng phương sai tích lũy**: chọn k nhỏ nhất để $\sum_{j\le k}\lambda_j / \sum_j \lambda_j \ge 90\%$ (hoặc 95%). Trong scikit-learn chỉ cần `PCA(n_components=0.95)`.
- **Biểu đồ scree** (Cattell, 1966): vẽ trị riêng theo thứ tự, chọn tại điểm "gãy".
- **Tiêu chí Kaiser**: với dữ liệu đã chuẩn hóa, giữ các thành phần có $\lambda_j > 1$.
- **Theo mục tiêu**: $k = 2$ hoặc 3 để trực quan hóa; với bài toán dự đoán, coi k là siêu tham số và chọn bằng kiểm định chéo (Chương 11).

Listing 10.7: PCA trên tập chữ số viết tay

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

digits = load_digits()                  # 1797 anh 8x8 = 64 chieu
pca = PCA().fit(digits.data)
cum = np.cumsum(pca.explained_variance_ratio_)
print("So thanh phan giu 95% phuong sai:", np.argmax(cum >= 0.95) +
    1)   # 29

plt.plot(range(1, 65), cum, marker=".")
plt.axhline(0.95, linestyle="--"); plt.xlabel("So thanh phan")
plt.ylabel("Phuong sai tich luy"); plt.show()

Z = PCA(n_components=2).fit_transform(digits.data)
plt.scatter(Z[:, 0], Z[:, 1], c=digits.target, cmap="tab10", s=8)
plt.colorbar(); plt.show()
```

[Chỗ chèn hình]
images/ch-reduce-pca-variance.png

Hình 10.3: Phương sai tích lũy theo số thành phần chính trên tập `digits`: khoảng 29 thành phần giữ 95% phương sai.

[Chỗ chèn hình]
images/ch-reduce-pca-digits.png

Hình 10.4: Tập `digits` chiếu xuống 2 thành phần chính đầu tiên, tô màu theo chữ số.

10.4.3 PCA trong pipeline học máy

PCA là một bộ biến đổi (transformer) như mọi bộ tiền xử lý khác: phải `fit` trên tập huấn luyện rồi mới `transform` tập kiểm tra, để tránh rò rỉ dữ liệu.

Listing 10.8: StandardScaler + PCA + Logistic Regression

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_tr, X_te, y_tr, y_te = train_test_split(digits.data, digits.target,
                                          test_size=0.25,
                                             random_state=0)
for k in [5, 10, 20, 40]:
    model = make_pipeline(StandardScaler(), PCA(n_components=k),
                          LogisticRegression(max_iter=2000))
    model.fit(X_tr, y_tr)
    print(k, round(model.score(X_te, y_te), 4))
```

Hạn chế của PCA.

- Chỉ bắt được quan hệ **tuyến tính**; với dữ liệu nằm trên đa tạp cong (ví dụ "cuộn Thụy Sĩ") cần Kernel PCA, t-SNE (van der Maaten & Hinton, 2008) hoặc UMAP.
- **Nhạy với thang đo** và ngoại lai (phương sai bị chi phối bởi cột có giá trị lớn).
- Thành phần mới là tổ hợp của mọi cột gốc nên **khó diễn giải**.
- PCA **không dùng nhãn**: hướng có phương sai lớn chưa chắc là hướng phân biệt lớp tốt nhất — khi có nhãn, có thể dùng LDA (Fisher, 1936).

10.5 Tóm tắt chương

- Số chiều lớn làm dữ liệu thưa, khoảng cách mất ý nghĩa và tăng nguy cơ quá khớp.
- Phương sai, hiệp phương sai (chú ý dấu và quy ước chia n hay $n-1$) và cặp trị riêng – vector riêng (kiểm tra bằng vết, định thức và phép nhân $\mathbf{Av} = \lambda\mathbf{v}$) là nền tảng toán học của SVD và PCA.
- SVD: $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$; giữ k giá trị suy biến lớn nhất cho xấp xỉ hạng k tối ưu (Eckart–Young). Dùng cho nén ảnh, LSA, hệ gợi ý.
- PCA tìm các hướng trực giao có phương sai lớn nhất: vector riêng của ma trận hiệp phương sai, tính thực tế bằng SVD của dữ liệu đã chuẩn hóa tâm.
- Chọn số thành phần theo phương sai tích lũy, biểu đồ scree hoặc kiểm định chéo; luôn đặt PCA trong pipeline.

10.6 Bài tập

1. Tính phương sai (chia n và chia $n-1$) của $(4, 8, 6, 2)$. Tính hiệp phương sai và hệ số tương quan của $A = (1, 2, 3, 4)$ và $B = (8, 6, 5, 1)$; giải thích ý nghĩa của dấu. Kiểm tra bằng `np.var`, `np.cov`, `np.corrcoef`.
2. Tìm trị riêng và vector riêng đơn vị của $\begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$ (đáp số: $\lambda = 5$ và $\lambda = 2$). Kiểm tra kết quả bằng vết, định thức và `np.linalg.eig`.
3. Tiếp ví dụ SVD của ma trận 5×2: tính $\mathbf{u}_2 = \mathbf{X}\mathbf{v}_2/\sigma_2$, kiểm tra $\mathbf{u}_1 \perp \mathbf{u}_2$ và tái tạo $\mathbf{X} = \sigma_1\mathbf{u}_1\mathbf{v}_1^\top + \sigma_2\mathbf{u}_2\mathbf{v}_2^\top$.
4. Làm lại toàn bộ các bước PCA cho 10 điểm của Smith (2002):

$$x_1 = (2{,}5;\ 0{,}5;\ 2{,}2;\ 1{,}9;\ 3{,}1;\ 2{,}3;\ 2{,}0;\ 1{,}0;\ 1{,}5;\ 1{,}1),$$
$$x_2 = (2{,}4;\ 0{,}7;\ 2{,}9;\ 2{,}2;\ 3{,}0;\ 2{,}7;\ 1{,}6;\ 1{,}1;\ 1{,}6;\ 0{,}9).$$

(Đáp số: $\lambda_1 \approx 1{,}2840$, $\mathbf{w}_1 \approx (0{,}6779;\ 0{,}7352)$, thành phần thứ nhất giữ khoảng 96,3% phương sai.)

5. Tính tay SVD của $\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$ và $\mathbf{B} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Ma trận nào có hạng 1?

6. Với dữ liệu ở Bảng 10.2, tái tạo các điểm từ 1 thành phần chính và tính tổng bình phương sai số tái tạo. So sánh với $(n-1)\lambda_2$.

7. Chứng minh rằng các tọa độ chiếu $\mathbf{z}$ trên thành phần chính thứ nhất có trung bình 0 và phương sai λ_1.

8. Trên tập `load_breast_cancer`, so sánh tỉ lệ phương sai giải thích của PCA khi có và không có `StandardScaler`. Giải thích.

9. Nén một bức ảnh bằng SVD với $k = 10, 30, 100$; tính tỉ lệ nén và sai số Frobenius tương đối.

10. Với tập Ames Housing đã tiền xử lý ở Chương 5, đặt PCA vào pipeline Ridge. Dùng `GridSearchCV` để chọn `n_components`.

Tài liệu tham khảo của chương

1. Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

2. Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When Is "Nearest Neighbor" Meaningful? *Proceedings of ICDT '99*, 217–235.

3. Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.

4. Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

5. Golub, G., & Kahan, W. (1965). Calculating the Singular Values and Pseudo-Inverse of a Matrix. *SIAM Journal on Numerical Analysis*, 2(2), 205–224.

6. Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

7. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

8. Pearson, K. (1896). Mathematical Contributions to the Theory of Evolution. III. Regression, Heredity, and Panmixia. *Philosophical Transactions of the Royal Society of London, Series A*, 187, 253–318.

9. Pearson, K. (1901). On Lines and Planes of Closest Fit to Systems of Points in Space. *Philosophical Magazine*, 2(11), 559–572.

10. Smith, L. I. (2002). *A tutorial on Principal Components Analysis*. University of Otago.

11. Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.

12. van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.

Chương 11: Đánh giá và lựa chọn mô hình (Validation)

Các chương trước đã giới thiệu hàng chục mô hình, mỗi mô hình lại có nhiều siêu tham số. Câu hỏi trung tâm của chương này là: **làm sao biết một mô hình tốt đến đâu trên dữ liệu nó chưa từng thấy**, và **làm sao chọn được mô hình và siêu tham số tốt nhất** mà không tự lừa mình. Đánh giá sai là nguyên nhân phổ biến nhất khiến mô hình đạt kết quả rất cao trong phòng thí nghiệm nhưng thất bại khi triển khai.
Sau khi học xong chương này, người học có khả năng:

- Giải thích lỗi tổng quát hóa, quá khớp, dưới khớp và đánh đổi bias – variance.
- Thiết kế đúng việc chia dữ liệu: train / validation / test, k-fold, stratified, time series split.
- Tính tay và diễn giải các độ đo phân lớp (ma trận nhầm lẫn, precision, recall, F1, ROC – AUC) và hồi quy (MAE, MSE, RMSE, R^2).
- Tìm siêu tham số bằng `GridSearchCV`, `RandomizedSearchCV` và tránh rò rỉ dữ liệu.

11.1 Lỗi tổng quát hóa

Mục tiêu của học máy không phải là khớp dữ liệu huấn luyện mà là **tổng quát hóa** (generalization): dự đoán tốt trên dữ liệu mới lấy từ cùng phân phối. Gọi $\mathcal{D}$ là phân phối dữ liệu và L là hàm mất mát; **lỗi tổng quát hóa** (rủi ro thực) của mô hình f là

$$R(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\big[L(y, f(\mathbf{x}))\big]. \tag{11.1}$$

Ta không biết $\mathcal{D}$, nên chỉ có thể **ước lượng** $R(f)$ bằng trung bình mất mát trên một tập dữ liệu **độc lập** với quá trình huấn luyện. Lỗi trên tập huấn luyện luôn là ước lượng **quá lạc quan**.

Tình trạng	Lỗi train	Lỗi test	Cách khắc phục
Dưới khớp (underfitting)	Cao	Cao	Mô hình phức tạp hơn, thêm đặc trưng, giảm điều chuẩn
Vừa khớp	Thấp	Thấp	—
Quá khớp (overfitting)	Rất thấp	Cao	Thêm dữ liệu, điều chuẩn, giảm độ phức tạp, ensemble

Bảng 11.1: Chẩn đoán qua lỗi huấn luyện và lỗi kiểm tra

Đối với mất mát bình phương, sai số kỳ vọng tại một điểm phân tích được thành (Geman và cộng sự, 1992):

$$\mathbb{E}\big[(y - \hat{f}(\mathbf{x}))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(\mathbf{x})] - f(\mathbf{x})\big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(\mathbf{x})]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{nhiễu}}. \tag{11.2}$$

Mô hình đơn giản có bias cao, mô hình phức tạp có variance cao; điểm tối ưu nằm ở giữa.

[Chỗ chèn hình]

`images/ch-validation-bias-variance.png`

Hình 11.1: Lỗi huấn luyện và lỗi kiểm tra theo độ phức tạp của mô hình: vùng dưới khớp, vùng tối ưu và vùng quá khớp.

11.2 Chiến lược chia dữ liệu

11.2.1 Hold-out: train / validation / test

- **Tập huấn luyện** (train): học tham số mô hình.
- **Tập kiểm định** (validation): so sánh mô hình, chọn siêu tham số.
- **Tập kiểm tra** (test): đánh giá **một lần duy nhất** ở cuối. Nếu dùng tập test để chọn mô hình, nó trở thành tập validation và ước lượng không còn khách quan.

Tỉ lệ thường dùng là 60/20/20 hoặc 70/15/15; với dữ liệu rất lớn, tập test chỉ cần 1%. Với bài toán phân lớp, dùng `stratify=y` để giữ tỉ lệ các lớp.

Listing 11.1: Chia dữ liệu ba phần

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)  # 0.25*0.8 = 0.2
print(len(X_train), len(X_val), len(X_test))   # 341 114 114
```

11.2.2 Kiểm định chéo k-fold

Hold-out lãng phí dữ liệu và kết quả phụ thuộc vào cách chia ngẫu nhiên. **Kiểm định chéo** (cross-validation), được Stone (1974) và Geisser (1975) hình thức hóa, khắc phục điều này:

1. Chia dữ liệu thành k phần (fold) gần bằng nhau.
2. Lần lượt dùng mỗi phần làm tập validation, $k-1$ phần còn lại làm tập huấn luyện.
3. Điểm cuối cùng là trung bình (và độ lệch chuẩn) của k lần đánh giá.

Mỗi mẫu được dùng để kiểm định đúng một lần. $k = 5$ hoặc 10 là lựa chọn phổ biến (Kohavi, 1995). Trường hợp $k = n$ gọi là **Leave-One-Out** (LOOCV): ít bias nhưng tốn kém và có variance cao.

[Chỗ chèn hình]

`images/ch-validation-kfold.png`

Hình 11.2: Sơ đồ kiểm định chéo 5-fold: mỗi lượt một fold (tô đậm) làm tập kiểm định.

Bộ chia	**Khi nào dùng**
`KFold`	Hồi quy, dữ liệu độc lập cùng phân phối
`StratifiedKFold`	Phân lớp (mặc định của `cross_val_score` khi mô hình là bộ phân lớp), đặc biệt khi lớp mất cân bằng
`GroupKFold`	Nhiều mẫu từ cùng một đối tượng (bệnh nhân, khách hàng): mọi mẫu của một nhóm phải nằm cùng phía
`TimeSeriesSplit`	Chuỗi thời gian: luôn huấn luyện trên quá khứ, kiểm định trên tương lai (Chương **??**)

Bảng 11.2: Các bộ chia dữ liệu trong scikit-learn

Listing 11.2: Kiểm định chéo với cross_val_score và cross_validate

```
from sklearn.model_selection import cross_val_score, cross_validate,
    StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter
    =1000))
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X_tmp, y_tmp, cv=cv, scoring="
    accuracy")
print(scores.round(4), scores.mean().round(4), scores.std().round(4))

res = cross_validate(model, X_tmp, y_tmp, cv=cv,
                     scoring=["accuracy", "f1", "roc_auc"],
                        return_train_score=True)
print({k: v.mean().round(4) for k, v in res.items() if "test" in k})
```

Quan trọng: bộ tiền xử lý (scaler, encoder, PCA, …) phải nằm **trong pipeline** để được `fit` lại trên từng fold huấn luyện. Nếu chuẩn hóa toàn bộ dữ liệu trước rồi mới kiểm định chéo, thông tin của fold kiểm định đã rò rỉ vào quá trình huấn luyện (xem Chương 5).

11.3 Độ đo cho bài toán phân lớp

11.3.1 Ma trận nhầm lẫn

Với bài toán hai lớp (lớp **dương** là lớp ta quan tâm: có bệnh, thư rác, gian lận …), mọi dự đoán rơi vào một trong bốn ô:

	Dự đoán dương	**Dự đoán âm**
Thực tế dương	TP (True Positive)	FN (False Negative)
Thực tế âm	FP (False Positive)	TN (True Negative)

Bảng 11.3: Ma trận nhầm lẫn (confusion matrix)

Độ đo	**Công thức**	**Ý nghĩa**
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Tỉ lệ dự đoán đúng
Precision	$\frac{TP}{TP+FP}$	Trong các mẫu báo dương, bao nhiêu là dương thật
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Trong các mẫu dương thật, tìm ra được bao nhiêu
Specificity	$\frac{TN}{TN+FP}$	Trong các mẫu âm thật, nhận đúng bao nhiêu
F1-score	$\frac{2 \cdot P \cdot R}{P+R}$	Trung bình điều hòa của precision và recall

Bảng 11.4: Các độ đo phân lớp

Ví dụ 11.1 (Tính tay các độ đo)**.** *Một mô hình sàng lọc bệnh được thử trên 100 người: 50 người có bệnh, 50 người không. Kết quả:* $TP = 40$, $FN = 10$, $FP = 5$, $TN = 45$.

$$\textit{Accuracy} = \tfrac{40+45}{100} = 0{,}85, \qquad \textit{Precision} = \tfrac{40}{45} = 0{,}8889,$$
$$\textit{Recall} = \tfrac{40}{50} = 0{,}80, \qquad \textit{Specificity} = \tfrac{45}{50} = 0{,}90,$$
$$F_1 = \tfrac{2 \times 0{,}8889 \times 0{,}80}{0{,}8889+0{,}80} = 0{,}8421.$$

Mô hình bỏ sót 10 trong 50 người bệnh (recall 80%) — với bài toán y tế, đây là điều cần cải thiện dù accuracy khá cao.

Nghịch lý accuracy. Với dữ liệu mất cân bằng (ví dụ 1% giao dịch gian lận), mô hình luôn đoán "không gian lận" đạt accuracy 99% nhưng recall bằng 0. Khi đó phải dùng precision, recall, F1 hoặc AUC. Việc ưu tiên precision hay recall tùy vào **chi phí** của từng loại lỗi: lọc thư rác ưu tiên precision (không muốn chặn nhầm thư quan trọng), sàng lọc ung thư ưu tiên recall (không muốn bỏ sót).

Bài toán nhiều lớp. Tính độ đo cho từng lớp (một lớp là dương, còn lại là âm) rồi lấy trung bình: **macro** (trung bình cộng, mỗi lớp như nhau), **weighted** (trọng số theo số mẫu), **micro** (cộng dồn TP, FP, FN của mọi lớp).

11.3.2 Đường ROC và AUC

Hầu hết bộ phân lớp đưa ra **điểm số** hoặc xác suất (`predict_proba`), sau đó so với **ngưỡng** (mặc định 0,5). Thay đổi ngưỡng sẽ đổi precision lấy recall. Đường **ROC** (Receiver Operating Characteristic) — bắt nguồn từ lý thuyết phát hiện tín hiệu radar trong Thế chiến II — vẽ $TPR =$ Recall theo $FPR = 1 -$ Specificity khi ngưỡng chạy từ cao đến thấp. **AUC** là diện tích dưới đường ROC: bằng 1 với bộ phân lớp hoàn hảo, 0,5 với đoán ngẫu nhiên. AUC có một diễn giải xác suất đẹp (Hanley & McNeil, 1982): *AUC là xác suất một mẫu dương ngẫu nhiên được chấm điểm cao hơn một mẫu âm ngẫu nhiên.*

Ví dụ 11.2 (Tính AUC bằng cách đếm cặp)**.** *Bốn mẫu có nhãn* $y = (0, 0, 1, 1)$ *và điểm số* $s = (0{,}1;\ 0{,}4;\ 0{,}35;\ 0{,}8)$. *Có* $2 \times 2 = 4$ *cặp (dương, âm):*

- $(0{,}35$ *vs* $0{,}1)$*: đúng;* $(0{,}35$ *vs* $0{,}4)$*:* ***sai*** *thứ tự;*
- $(0{,}8$ *vs* $0{,}1)$*: đúng;* $(0{,}8$ *vs* $0{,}4)$*: đúng.*

$AUC = 3/4 = 0{,}75$.

Listing 11.3: Ma trận nhầm lẫn, báo cáo phân lớp và ROC

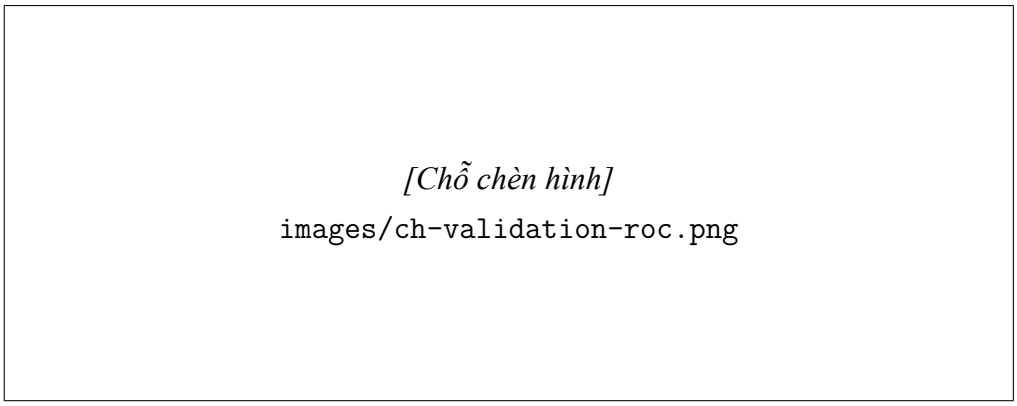
```
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, RocCurveDisplay,
                                ConfusionMatrixDisplay)

print(roc_auc_score([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8]))   # 0.75

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_score = model.predict_proba(X_test)[:, 1]

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=4))
print("AUC:", roc_auc_score(y_test, y_score))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
RocCurveDisplay.from_predictions(y_test, y_score)
```



Hình 11.3: Đường ROC của mô hình Logistic Regression trên tập ung thư vú so với đường chéo của bộ đoán ngẫu nhiên.

Với dữ liệu mất cân bằng nặng, đường **Precision – Recall** và diện tích dưới nó (Average Precision) thường phản ánh chất lượng mô hình rõ hơn ROC (Saito & Rehmsmeier, 2015).

11.4 Độ đo cho bài toán hồi quy

Độ đo	Công thức	Ghi chú
MAE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	Cùng đơn vị với y, ít nhạy với ngoại lai
MSE	$\frac{1}{n}\sum (y_i - \hat{y}_i)^2$	Phạt nặng sai số lớn
RMSE	$\sqrt{\text{MSE}}$	Cùng đơn vị với y
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Tỉ lệ phương sai được giải thích; có thể âm
MAPE	$\frac{1}{n}\sum \left\lvert \frac{y_i - \hat{y}_i}{y_i} \right\rvert$	Sai số tương đối; không dùng khi $y_i \approx 0$

Bảng 11.5: Các độ đo hồi quy

Ví dụ 11.3 (Tính tay độ đo hồi quy)**.** $y = (3;\ -0{,}5;\ 2;\ 7)$, $\hat{y} = (2{,}5;\ 0;\ 2;\ 8)$. *Sai số:* $(0{,}5;\ -0{,}5;\ 0;\ -1)$.

$$\textit{MAE} = \frac{0{,}5+0{,}5+0+1}{4} = 0{,}5, \quad \textit{MSE} = \frac{0{,}25+0{,}25+0+1}{4} = 0{,}375, \quad \textit{RMSE} = 0{,}6124.$$

$\bar{y} = 2{,}875$, $\sum (y_i - \bar{y})^2 = 29{,}1875$, *nên* $R^2 = 1 - 1{,}5/29{,}1875 = 0{,}9486$.

Listing 11.4: Độ đo hồi quy trong scikit-learn

```
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             root_mean_squared_error, r2_score)
y_true, y_hat = [3, -0.5, 2, 7], [2.5, 0.0, 2, 8]
print(mean_absolute_error(y_true, y_hat))       # 0.5
print(mean_squared_error(y_true, y_hat))        # 0.375
print(root_mean_squared_error(y_true, y_hat))   # 0.6124
print(r2_score(y_true, y_hat))                  # 0.9486
```

Lưu ý: trong `cross_val_score`, scikit-learn quy ước "điểm càng lớn càng tốt", nên các độ đo sai số có dạng âm như `scoring="neg_root_mean_squared_error"`.

11.5 Tìm siêu tham số

Tham số (trọng số hồi quy, ngưỡng chia của cây) được học từ dữ liệu; **siêu tham số** (`alpha` của Ridge, `n_neighbors` của KNN, `max_depth` của cây) phải được chọn trước khi huấn luyện. Ta chọn siêu tham số bằng cách tối ưu điểm kiểm định chéo.

- **Grid search**: thử mọi tổ hợp trên lưới giá trị — đầy đủ nhưng chi phí tăng theo cấp số nhân với số siêu tham số.
- **Random search**: lấy mẫu ngẫu nhiên N tổ hợp từ các phân phối. Bergstra và Bengio (2012) chỉ ra rằng với cùng ngân sách, random search thường tốt hơn vì chỉ vài siêu tham số thực sự quan trọng.
- **Tối ưu Bayes** (Optuna, scikit-optimize): dùng kết quả các lần thử trước để chọn điểm thử tiếp theo.

Listing 11.5: GridSearchCV và RandomizedSearchCV

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.svm import SVC
from scipy.stats import loguniform

pipe = make_pipeline(StandardScaler(), SVC())
param_grid = {"svc__C": [0.1, 1, 10, 100],
              "svc__gamma": [0.001, 0.01, 0.1, 1],
              "svc__kernel": ["rbf"]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="f1", n_jobs=-1)
grid.fit(X_tmp, y_tmp)                       # 16 to hop x 5 fold = 80 lan fit
print(grid.best_params_, round(grid.best_score_, 4))
print("Test F1:", round(grid.score(X_test, y_test), 4))   # danh gia 1 lan cuoi

rand = RandomizedSearchCV(pipe, {"svc__C": loguniform(1e-2, 1e3),
                                 "svc__gamma": loguniform(1e-4, 1e0)
                                    },
                          n_iter=30, cv=5, scoring="f1", random_state
                             =0, n_jobs=-1)
rand.fit(X_tmp, y_tmp)
print(rand.best_params_)
```

[Chỗ chèn hình]
`images/ch-validation-grid-heatmap.png`

Hình 11.4: Bản đồ nhiệt điểm F1 kiểm định chéo theo `C` và `gamma` của SVM.

Kiểm định chéo lồng nhau. `best_score_` của GridSearchCV là ước lượng **lạc quan**, vì chính nó đã được dùng để chọn siêu tham số (Cawley & Talbot, 2010). Để ước lượng khách quan khi không có tập test riêng, dùng **nested CV**: vòng ngoài đánh giá, vòng trong tìm siêu tham số.

Listing 11.6: Nested cross-validation

```
outer = cross_val_score(GridSearchCV(pipe, param_grid, cv=3),
                        X, y, cv=5, scoring="f1")
print(outer.mean().round(4), outer.std().round(4))
```

11.5.1 Đường cong học và đường cong kiểm định

- **Learning curve**: điểm train và validation theo **kích thước tập huấn luyện**. Hai đường cách xa nhau là dấu hiệu quá khớp, thêm dữ liệu sẽ có ích; hai đường hội tụ ở mức thấp là dưới khớp.
- **Validation curve**: điểm theo **một siêu tham số**, cho thấy vùng dưới khớp – quá khớp.

Listing 11.7: Learning curve và validation curve

```
from sklearn.model_selection import LearningCurveDisplay,
    ValidationCurveDisplay
from sklearn.tree import DecisionTreeClassifier

LearningCurveDisplay.from_estimator(pipe, X, y, cv=5,
                                    train_sizes=np.linspace(0.1, 1.0,
                                        8))
ValidationCurveDisplay.from_estimator(DecisionTreeClassifier(
    random_state=0), X, y,
                                      param_name="max_depth",
                                      param_range=range(1, 15), cv=5)
```

[Chỗ chèn hình]
`images/ch-validation-curves.png`

Hình 11.5: Trái: learning curve; phải: validation curve theo `max_depth` của cây quyết định.

11.6 Những sai lầm thường gặp

1. **Rò rỉ dữ liệu**: tiền xử lý, chọn đặc trưng hoặc cân bằng lớp (SMOTE) trên toàn bộ dữ liệu trước khi chia. Cách tránh: mọi bước đặt trong `Pipeline`.
2. **Đặc trưng chứa thông tin tương lai**: ví dụ dùng "ngày xuất viện" để dự đoán "có nhập viện không".
3. **Xáo trộn chuỗi thời gian** khi kiểm định chéo.
4. **Trùng lặp mẫu** giữa train và test (cùng một bệnh nhân, ảnh gần giống nhau).
5. **Nhìn tập test nhiều lần** để chỉnh mô hình.
6. **Chỉ báo accuracy** với dữ liệu mất cân bằng; không báo độ lệch chuẩn giữa các fold.

11.7 Tóm tắt chương

- Mục tiêu là lỗi tổng quát hóa; lỗi huấn luyện luôn lạc quan. Cân bằng bias và variance.
- Chia train / validation / test; tập test chỉ dùng một lần. Kiểm định chéo k-fold cho ước lượng ổn định hơn; chọn bộ chia phù hợp (stratified, group, time series).
- Phân lớp: ma trận nhầm lẫn, precision, recall, F1, ROC – AUC; hồi quy: MAE, MSE, RMSE, R^2.
- Tìm siêu tham số bằng Grid/Random search trên pipeline; dùng nested CV để có ước lượng khách quan.

11.8 Bài tập

1. Một bộ lọc thư rác trên 1000 email (100 thư rác) cho $TP = 80$, $FP = 30$. Lập ma trận nhầm lẫn và tính accuracy, precision, recall, F1, specificity.
2. Tính AUC bằng cách đếm cặp cho $y = (1, 0, 1, 0, 1)$, $s = (0{,}9;\ 0{,}8;\ 0{,}7;\ 0{,}3;\ 0{,}2)$, rồi kiểm tra bằng `roc_auc_score`.

3. Chứng minh rằng R^2 của mô hình luôn dự đoán $\bar{y}$ bằng 0. Khi nào $R^2 < 0$?

4. So sánh điểm accuracy của KNN ($K = 5$) trên tập Iris khi đánh giá bằng hold-out với 10 seed khác nhau và bằng 10-fold CV. Nhận xét về độ biến động.

5. Chứng minh bằng thực nghiệm hiện tượng rò rỉ dữ liệu: sinh dữ liệu ngẫu nhiên X (100 mẫu, 10 000 cột) và nhãn ngẫu nhiên; chọn 20 cột tương quan cao nhất với y *trên toàn bộ dữ liệu* rồi mới kiểm định chéo. So sánh với việc đặt `SelectKBest` trong pipeline.

6. Trên Ames Housing, dùng `GridSearchCV` để chọn `alpha` cho Ridge và Lasso (thang log), báo cáo RMSE kiểm định chéo và RMSE trên tập test.

Tài liệu tham khảo của chương

1. Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.

2. Cawley, G. C., & Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, 2079–2107.

3. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

4. Geisser, S. (1975). The Predictive Sample Reuse Method with Applications. *Journal of the American Statistical Association*, 70(350), 320–328.

5. Geman, S., Bienenstock, E., & Doursat, R. (1992). Neural Networks and the Bias/Variance Dilemma. *Neural Computation*, 4(1), 1–58.

6. Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36.

7. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Chapter 7. Springer.

8. Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of IJCAI '95*, 1137–1143.

9. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE*, 10(3), e0118432.

10. Stone, M. (1974). Cross-Validatory Choice and Assessment of Statistical Predictions. *Journal of the Royal Statistical Society: Series B*, 36(2), 111–147.

Chương 12: Mạng nơ-ron nhân tạo (Neural Network)

Mạng nơ-ron nhân tạo là họ mô hình đứng sau hầu hết thành tựu nổi bật của trí tuệ nhân tạo hiện đại: nhận dạng ảnh, dịch máy, nhận dạng giọng nói và các mô hình ngôn ngữ lớn. Chương cuối này trình bày nền tảng của mạng nơ-ron: từ perceptron đơn lẻ đến mạng nhiều lớp (Multilayer Perceptron – MLP), thuật toán lan truyền ngược, các kỹ thuật huấn luyện, và cách cài đặt bằng scikit-learn cùng một thư viện học sâu.
Sau khi học xong chương này, người học có khả năng:

- Trình bày lịch sử phát triển của mạng nơ-ron và lý do của các "mùa đông AI".
- Tính tay đầu ra của perceptron và giải thích vì sao perceptron không học được XOR.
- Tính tay một bước lan truyền xuôi và lan truyền ngược trên mạng nhỏ.
- Huấn luyện `MLPClassifier`/`MLPRegressor` và một mạng Keras đơn giản, lựa chọn hàm kích hoạt, hàm mất mát, bộ tối ưu và kỹ thuật điều chuẩn.

12.1 Lịch sử phát triển

Năm	Sự kiện
1943	McCulloch và Pitts đề xuất mô hình toán học đầu tiên của nơ-ron sinh học: nơ-ron ngưỡng với đầu vào nhị phân.
1949	Donald Hebb nêu quy tắc học "các nơ-ron cùng kích hoạt thì liên kết mạnh lên".
1958	Frank Rosenblatt công bố **Perceptron** — mô hình nơ-ron đầu tiên có thuật toán học trọng số từ dữ liệu, cài đặt trên phần cứng Mark I.
1969	Minsky và Papert (sách *Perceptrons*) chứng minh perceptron một lớp không biểu diễn được hàm XOR $\Rightarrow$ "mùa đông AI" thứ nhất.
1986	Rumelhart, Hinton và Williams phổ biến thuật toán **lan truyền ngược** (backpropagation) cho mạng nhiều lớp (ý tưởng đã có từ Linnainmaa 1970, Werbos 1974).
1989	Cybenko, Hornik chứng minh **định lý xấp xỉ vạn năng**. LeCun áp dụng mạng tích chập (CNN) nhận dạng chữ số viết tay.
1997	Hochreiter và Schmidhuber đề xuất LSTM cho dữ liệu chuỗi.
2006	Hinton và cộng sự huấn luyện mạng sâu bằng tiền huấn luyện từng lớp — khởi đầu thuật ngữ "deep learning".
2012	AlexNet (Krizhevsky, Sutskever, Hinton) thắng ImageNet với cách biệt lớn nhờ GPU, ReLU, Dropout.
2017	Vaswani và cộng sự giới thiệu kiến trúc **Transformer** ("Attention Is All You Need"), nền tảng của các mô hình ngôn ngữ lớn.
2018	Bengio, Hinton, LeCun nhận giải Turing cho những đóng góp về học sâu.

Bảng 12.1: Các mốc lịch sử của mạng nơ-ron

12.2 Perceptron

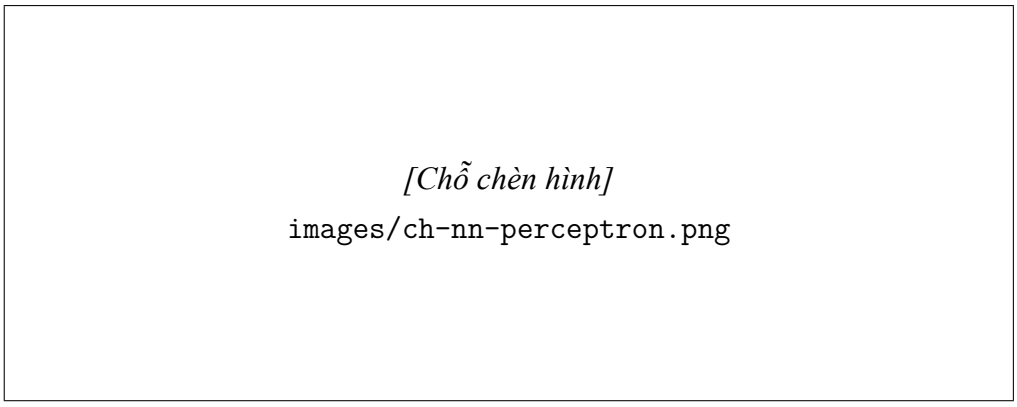
Định nghĩa 12.1 (Perceptron)**.** *Perceptron nhận vector đầu vào* $\mathbf{x} \in \mathbb{R}^d$*, tính tổng có trọng số rồi đưa qua hàm bước:*

$$\hat{y} = \text{step}(\mathbf{w}^\top \mathbf{x} + b), \qquad \text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases} \tag{12.1}$$

Quy tắc học của Rosenblatt: với mỗi mẫu bị phân lớp sai, cập nhật

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \qquad b \leftarrow b + \eta\,(y - \hat{y}). \tag{12.2}$$

Định lý hội tụ perceptron (Novikoff, 1962): nếu dữ liệu **tách được tuyến tính**, thuật toán dừng sau hữu hạn bước cập nhật.



Hình 12.1: Cấu trúc perceptron: đầu vào, trọng số, tổng có trọng số, hàm kích hoạt và đầu ra.

Ví dụ 12.1 (Perceptron cho hàm AND)**.** *Chọn* $\mathbf{w} = (1, 1)$*,* $b = -1{,}5$*:*

x_1	x_2	$z = x_1 + x_2 - 1{,}5$	$\hat{y}$	AND	XOR
0	0	$-1{,}5$	0	0	0
0	1	$-0{,}5$	0	0	1
1	0	$-0{,}5$	0	0	1
1	1	$0{,}5$	1	1	0

Bảng 12.2: Perceptron biểu diễn hàm AND

Đường thẳng $x_1 + x_2 = 1{,}5$ tách điểm $(1, 1)$ khỏi ba điểm còn lại. Nhưng với **XOR**, hai điểm lớp 1 là $(0, 1)$, $(1, 0)$ nằm trên một đường chéo và hai điểm lớp 0 nằm trên đường chéo kia: **không đường thẳng nào** tách được. Thật vậy, giả sử tồn tại $\mathbf{w}, b$: từ $(0, 0) \mapsto 0$ có $b < 0$; từ $(0, 1), (1, 0) \mapsto 1$ có $w_2 + b \geq 0$, $w_1 + b \geq 0$, cộng lại được $w_1 + w_2 + b \geq -b > 0$, mâu thuẫn với $(1, 1) \mapsto 0$. Lời giải: thêm một **lớp ẩn** — XOR = AND(OR, NAND).

Listing 12.1: Cài đặt perceptron từ đầu

```
import numpy as np

class Perceptron:
    def __init__(self, lr=1.0, epochs=10):
        self.lr, self.epochs = lr, epochs

    def fit(self, X, y):
        self.w = np.zeros(X.shape[1]); self.b = 0.0
        for _ in range(self.epochs):
            errors = 0
            for xi, yi in zip(X, y):
                update = self.lr * (yi - self.predict(xi))
                self.w += update * xi
                self.b += update
                errors += int(update != 0)
            if errors == 0:          # hoi tu
                break
        return self

```

```
    def predict(self, X):
        return (np.dot(X, self.w) + self.b >= 0).astype(int)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
p_and = Perceptron().fit(X, np.array([0, 0, 0, 1]))
print(p_and.w, p_and.b, p_and.predict(X))   # hoc dung AND
p_xor = Perceptron(epochs=100).fit(X, np.array([0, 1, 1, 0]))
print(p_xor.predict(X))                     # khong bao gio dung ca 4
```

12.3 Mạng nơ-ron nhiều lớp (MLP)

12.3.1 Kiến trúc

MLP gồm **lớp đầu vào**, một hoặc nhiều **lớp ẩn** và **lớp đầu ra**; mỗi nơ-ron nối với mọi nơ-ron ở lớp kế tiếp (lớp kết nối đầy đủ – dense). Với lớp thứ l:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = g^{(l)}\big(\mathbf{z}^{(l)}\big), \qquad \mathbf{a}^{(0)} = \mathbf{x}. \tag{12.3}$$

Hàm kích hoạt g phải **phi tuyến**: nếu không, nhiều lớp tuyến tính chồng lên nhau vẫn chỉ là một phép biến đổi tuyến tính.



Hình 12.2: Mạng MLP với lớp đầu vào, hai lớp ẩn và lớp đầu ra.

Định lý 12.1 (Xấp xỉ vạn năng – Cybenko 1989, Hornik 1991)**.** *Mạng một lớp ẩn với đủ nhiều nơ-ron và hàm kích hoạt phi tuyến (không phải đa thức) có thể xấp xỉ mọi hàm liên tục trên tập compact với độ chính xác tùy ý.*

Định lý chỉ khẳng định **tồn tại**, không nói cách tìm trọng số hay cần bao nhiêu nơ-ron. Trong thực tế, mạng **sâu** thường biểu diễn hiệu quả hơn mạng rộng một lớp.

12.3.2 Hàm kích hoạt

Hàm	Công thức	Đạo hàm	Ghi chú
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(1-\sigma)$	Đầu ra $(0,1)$; bão hòa $\Rightarrow$ triệt tiêu gradient
Tanh	$\tanh z$	$1 - \tanh^2 z$	Đầu ra $(-1,1)$, tâm 0
ReLU	$\max(0,z)$	$\mathbb{1}[z>0]$	Mặc định cho lớp ẩn (Nair & Hinton, 2010); nhanh
Leaky ReLU	$\max(0{,}01z, z)$	1 hoặc $0{,}01$	Tránh nơ-ron "chết"
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	—	Lớp đầu ra phân lớp nhiều lớp

Bảng 12.3: Các hàm kích hoạt phổ biến

[Chỗ chèn hình]

`images/ch-nn-activations.png`

Hình 12.3: Đồ thị các hàm kích hoạt sigmoid, tanh, ReLU và Leaky ReLU.

12.3.3 Hàm mất mát và lớp đầu ra

Bài toán	Lớp đầu ra	Hàm mất mát
Hồi quy	1 nơ-ron, tuyến tính	MSE
Phân lớp nhị phân	1 nơ-ron, sigmoid	Binary cross-entropy
Phân lớp K lớp	K nơ-ron, softmax	Categorical cross-entropy $-\sum_k y_k \log \hat{y}_k$

Bảng 12.4: Ghép cặp lớp đầu ra và hàm mất mát

12.4 Lan truyền ngược (Backpropagation)

Huấn luyện mạng là cực tiểu hàm mất mát L theo mọi trọng số bằng gradient descent (Chương **??**). Lan truyền ngược là cách tính **mọi** đạo hàm riêng $\partial L/\partial w$ hiệu quả bằng **quy tắc dây chuyền**, đi từ lớp đầu ra về lớp đầu vào và tái sử dụng kết quả trung gian. Chi phí chỉ cỡ một lần lan

truyền xuôi. Định nghĩa sai số tại lớp l là $\boldsymbol{\delta}^{(l)} = \partial L / \partial \mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} L \odot g'\big(\mathbf{z}^{(L)}\big), \tag{12.4}$$

$$\boldsymbol{\delta}^{(l)} = \big(\mathbf{W}^{(l+1)}\big)^{\top} \boldsymbol{\delta}^{(l+1)} \odot g'\big(\mathbf{z}^{(l)}\big), \tag{12.5}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \big(\mathbf{a}^{(l-1)}\big)^{\top}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}. \tag{12.6}$$

Ví dụ 12.2 (Tính tay một bước lan truyền xuôi và ngược)**.** *Mạng 2–2–1, mọi nơ-ron dùng sigmoid, mất mát* $L = \frac{1}{2}(t-o)^2$*. Đầu vào* $\mathbf{x} = (1;\ 0{,}5)$*, nhãn* $t = 1$*,*

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0{,}1 & 0{,}2 \\ 0{,}3 & 0{,}4 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0{,}1 \\ 0{,}1 \end{pmatrix}, \quad \mathbf{w}^{(2)} = (0{,}5;\ 0{,}6), \quad b^{(2)} = 0{,}1, \quad \eta = 0{,}5.$$

Lan truyền xuôi:

$$\begin{aligned} z_1 &= 0{,}1 \cdot 1 + 0{,}2 \cdot 0{,}5 + 0{,}1 = 0{,}3, & h_1 &= \sigma(0{,}3) = 0{,}5744, \\ z_2 &= 0{,}3 \cdot 1 + 0{,}4 \cdot 0{,}5 + 0{,}1 = 0{,}6, & h_2 &= \sigma(0{,}6) = 0{,}6457, \\ z_o &= 0{,}5 \cdot 0{,}5744 + 0{,}6 \cdot 0{,}6457 + 0{,}1 = 0{,}7746, & o &= \sigma(0{,}7746) = 0{,}6845, \end{aligned}$$

mất mát $L = \frac{1}{2}(1 - 0{,}6845)^2 = 0{,}0498$.

Lan truyền ngược:

$$\begin{aligned} \delta_o &= (o - t)\, o(1-o) = (-0{,}3155)(0{,}6845)(0{,}3155) = -0{,}0681, \\ \frac{\partial L}{\partial \mathbf{w}^{(2)}} &= \delta_o\, \mathbf{h} = (-0{,}0391;\ -0{,}0440), \\ \delta_{h_1} &= \delta_o \cdot 0{,}5 \cdot h_1(1-h_1) = -0{,}0083, \qquad \delta_{h_2} = \delta_o \cdot 0{,}6 \cdot h_2(1-h_2) = -0{,}0094, \\ \frac{\partial L}{\partial \mathbf{W}^{(1)}} &= \boldsymbol{\delta}_h\, \mathbf{x}^{\top} = \begin{pmatrix} -0{,}0083 & -0{,}0042 \\ -0{,}0094 & -0{,}0047 \end{pmatrix}. \end{aligned}$$

Cập nhật: $\mathbf{w}^{(2)} \leftarrow (0{,}5;\ 0{,}6) - 0{,}5 \times (-0{,}0391;\ -0{,}0440) = (0{,}5196;\ 0{,}6220)$. Gradient âm nên trọng số tăng, đẩy o lên gần nhãn $t = 1$. Chú ý gradient ở lớp ẩn nhỏ hơn lớp ra khoảng 8 lần, vì mỗi lần nhân với $\sigma' \le 0{,}25$ — đây chính là hiện tượng **triệt tiêu gradient** khiến mạng sâu dùng sigmoid khó huấn luyện, và là lý do ReLU được ưa chuộng.

Listing 12.2: Kiểm tra ví dụ lan truyền ngược bằng NumPy

```
sigmoid = lambda z: 1 / (1 + np.exp(-z))
x = np.array([1.0, 0.5]); t = 1.0
W1 = np.array([[0.1, 0.2], [0.3, 0.4]]); b1 = np.array([0.1, 0.1])
w2 = np.array([0.5, 0.6]); b2 = 0.1; lr = 0.5

# Lan truyen xuoi
h = sigmoid(W1 @ x + b1)            # [0.5744 0.6457]
o = sigmoid(w2 @ h + b2)            # 0.6845
loss = 0.5 * (t - o) ** 2           # 0.0498

# Lan truyen nguoc
d_o = (o - t) * o * (1 - o)         # -0.0681
d_h = d_o * w2 * h * (1 - h)        # [-0.0083 -0.0094]
w2 -= lr * d_o * h;  b2 -= lr * d_o
W1 -= lr * np.outer(d_h, x);  b1 -= lr * d_h
print(w2)                           # [0.5196 0.622 ]
```

12.5 Huấn luyện mạng nơ-ron trong thực tế

12.5.1 Bộ tối ưu

- **SGD theo lô nhỏ** (mini-batch): mỗi bước dùng 32–256 mẫu; một **epoch** là một lượt duyệt hết dữ liệu.
- **Momentum**: cộng dồn hướng các bước trước, giúp vượt vùng phẳng và giảm dao động.
- **Adam** (Kingma & Ba, 2015): tốc độ học thích nghi cho từng tham số; lựa chọn mặc định tốt (`solver="adam"` trong scikit-learn).

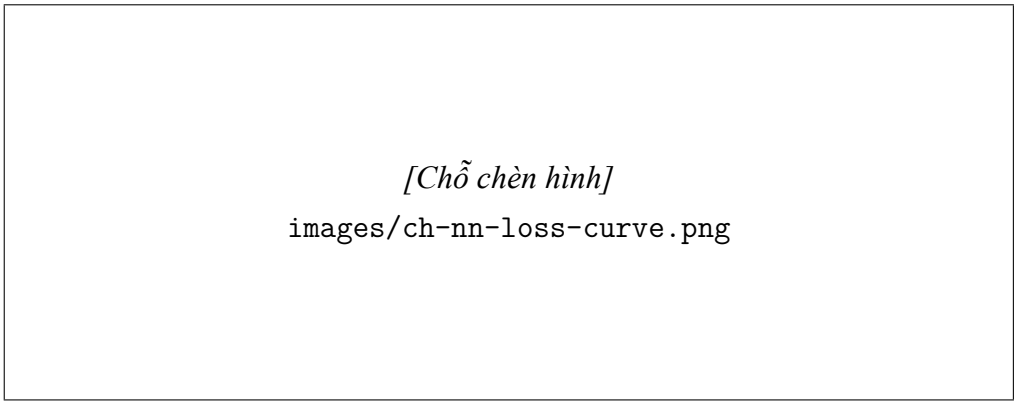
Tốc độ học η là siêu tham số quan trọng nhất: quá lớn thì phân kỳ, quá nhỏ thì hội tụ chậm.

12.5.2 Khởi tạo và chuẩn hóa

- Không khởi tạo mọi trọng số bằng 0 (các nơ-ron sẽ học giống hệt nhau). Dùng khởi tạo Xavier/Glorot (2010) cho sigmoid/tanh, He (2015) cho ReLU.
- **Chuẩn hóa đầu vào** (`StandardScaler`) là bắt buộc — mạng nơ-ron rất nhạy với thang đo.
- **Batch Normalization** (Ioffe & Szegedy, 2015) chuẩn hóa đầu ra các lớp ẩn, giúp huấn luyện nhanh và ổn định.

12.5.3 Chống quá khớp

- **Điều chuẩn L2** (weight decay): tham số `alpha` trong `MLPClassifier`.
- **Dropout** (Srivastava và cộng sự, 2014): mỗi bước huấn luyện tắt ngẫu nhiên một tỉ lệ nơ-ron.
- **Dừng sớm** (early stopping): theo dõi lỗi validation, dừng khi không còn cải thiện.
- Tăng cường dữ liệu (data augmentation) với ảnh, âm thanh.



Hình 12.4: Đường cong mất mát huấn luyện và kiểm định theo epoch; điểm dừng sớm tại nơi lỗi kiểm định nhỏ nhất.

12.6 MLP với scikit-learn

Listing 12.3: MLPClassifier giải bài toán XOR

```
from sklearn.neural_network import MLPClassifier

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]); y = np.array([0, 1, 1, 0])
mlp = MLPClassifier(hidden_layer_sizes=(4,), activation="tanh",
                    solver="lbfgs", random_state=1, max_iter=1000)
mlp.fit(X, y)
print(mlp.predict(X))     # [0 1 1 0] -- mot lop an du de hoc XOR
```

Listing 12.4: Phân lớp chữ số viết tay với MLPClassifier

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state
                                             =42)
pipe = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(128, 64), activation="relu",
                  solver="adam", alpha=1e-4, learning_rate_init=1e-3,
                  batch_size=64, max_iter=300, early_stopping=True,
                  random_state=42))
pipe.fit(X_tr, y_tr)
print(classification_report(y_te, pipe.predict(X_te), digits=4))

mlp = pipe[-1]
plt.plot(mlp.loss_curve_, label="train loss")
plt.plot(mlp.validation_scores_, label="validation accuracy")
plt.legend(); plt.xlabel("epoch"); plt.show()

# Tim sieu tham so (Chuong Validation)
grid = GridSearchCV(pipe, {"mlpclassifier__hidden_layer_sizes":
    [(64,), (128, 64)],
                           "mlpclassifier__alpha": [1e-4, 1e-2, 1]},
                              cv=3, n_jobs=-1)
grid.fit(X_tr, y_tr); print(grid.best_params_, grid.score(X_te, y_te)
    )
```

Listing 12.5: MLPRegressor dự đoán giá nhà Ames

```
from sklearn.neural_network import MLPRegressor
from sklearn.compose import TransformedTargetRegressor
```

```
3
4 # preprocess: ColumnTransformer da xay dung o Chuong Tien xu ly du
    lieu
5 reg = make_pipeline(preprocess,
6                     MLPRegressor(hidden_layer_sizes=(64, 32), alpha=1
                        e-3,
7                                  early_stopping=True, max_iter=1000,
                                     random_state=0))
8 model = TransformedTargetRegressor(regressor=reg, func=np.log1p,
    inverse_func=np.expm1)
9 model.fit(X_train, y_train)
```

Giới hạn: `MLPClassifier` chỉ chạy trên CPU và chỉ hỗ trợ lớp dense — phù hợp dữ liệu dạng bảng cỡ vừa và mục đích học tập. Với ảnh, văn bản, âm thanh và mạng sâu, cần các thư viện học sâu chuyên dụng.

12.7 Giới thiệu thư viện học sâu

TensorFlow/Keras (Google, 2015) và **PyTorch** (Meta, 2016) là hai thư viện phổ biến nhất. Cả hai cung cấp tensor chạy trên GPU, **tự động tính đạo hàm** (automatic differentiation — tổng quát hóa của lan truyền ngược) và các lớp dựng sẵn: Dense, Conv2D (CNN cho ảnh), LSTM/GRU (dữ liệu chuỗi), Attention/Transformer (văn bản).

Listing 12.6: Cùng bài toán digits với Keras

```
1 import keras
2 from keras import layers
3
4 scaler = StandardScaler().fit(X_tr)
5 model = keras.Sequential([
6     keras.Input(shape=(64,)),
7     layers.Dense(128, activation="relu"),
8     layers.Dropout(0.2),
9     layers.Dense(64, activation="relu"),
10    layers.Dense(10, activation="softmax"),
11 ])
12 model.compile(optimizer=keras.optimizers.Adam(1e-3),
13               loss="sparse_categorical_crossentropy", metrics=["
                    accuracy"])
14 model.summary()
15 stop = keras.callbacks.EarlyStopping(patience=5, restore_best_weights
    =True)
16 history = model.fit(scaler.transform(X_tr), y_tr, epochs=50,
    batch_size=64,
17                     validation_split=0.1, callbacks=[stop])
18 print(model.evaluate(scaler.transform(X_te), y_te))
```

Listing 12.7: Vòng lặp huấn luyện tối giản với PyTorch

```
1 import torch
2 from torch import nn
3
```

```
net = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10)
    )
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
Xt = torch.tensor(scaler.transform(X_tr), dtype=torch.float32)
yt = torch.tensor(y_tr)
for epoch in range(100):
    opt.zero_grad()
    loss = loss_fn(net(Xt), yt)     # lan truyen xuoi
    loss.backward()                 # lan truyen nguoc (autograd)
    opt.step()                      # cap nhat trong so
```

Loại dữ liệu	Kiến trúc phù hợp	Ví dụ ứng dụng
Dạng bảng	MLP; thường thua Gradient Boosting (Chương 8)	Dự đoán giá, rủi ro tín dụng
Ảnh	CNN, Vision Transformer	Phân loại ảnh, phát hiện đối tượng
Chuỗi, văn bản	RNN/LSTM, Transformer	Dịch máy, phân tích cảm xúc, LLM

Bảng 12.5: Lựa chọn kiến trúc theo loại dữ liệu

12.8 Tóm tắt chương

- Perceptron là bộ phân lớp tuyến tính, không học được XOR; thêm lớp ẩn với hàm kích hoạt phi tuyến cho MLP có khả năng xấp xỉ vạn năng.
- Lan truyền xuôi tính đầu ra; lan truyền ngược dùng quy tắc dây chuyền tính gradient cho mọi trọng số; cập nhật bằng SGD/Adam.
- ReLU cho lớp ẩn; sigmoid/softmax cho lớp ra phân lớp; ghép đúng với cross-entropy.
- Chuẩn hóa đầu vào, khởi tạo hợp lý, điều chuẩn L2, Dropout và dừng sớm là các kỹ thuật thiết yếu.
- scikit-learn cho MLP cơ bản; Keras/PyTorch cho học sâu trên GPU.

12.9 Bài tập

1. Tìm trọng số perceptron cho hàm OR và NAND. Ghép chúng với perceptron AND (Bảng 12.2) thành mạng hai lớp biểu diễn XOR; kiểm tra trên cả 4 đầu vào.
2. Chạy lại ví dụ lan truyền ngược thêm 100 bước. Vẽ mất mát theo số bước. Lặp lại với hàm kích hoạt ReLU ở lớp ẩn.
3. Chứng minh $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ và $\sigma'(z) \leq 0{,}25$. Ước lượng độ lớn gradient ở lớp đầu tiên của mạng 10 lớp sigmoid.
4. Với softmax và cross-entropy, chứng minh $\partial L/\partial z_k = \hat{y}_k - y_k$.

5. Huấn luyện `MLPClassifier` trên tập `digits` có và không có `StandardScaler`. So sánh độ chính xác và số vòng lặp.

6. So sánh `MLPRegressor`, Ridge và Gradient Boosting trên Ames Housing bằng RMSE kiểm định chéo 5-fold.

Tài liệu tham khảo của chương

1. Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.

2. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of AISTATS 2010*, 249–256.

3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

4. Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257.

5. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.

6. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.

7. McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5, 115–133.

8. Minsky, M., & Papert, S. (1969). *Perceptrons*. MIT Press.

9. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.

10. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

11. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.

12. Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30*.